INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE SÃO PAULO
CAMPUS SÃO PAULO

TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

JOAO ANTÔNIO CHAMUSCA MARTINS
KELVYN GABRIEL MILU DE OLIVEIRA
LUCAS VIEIRA DE SOUZA
MATHEUS PEREIRA PEREIRA

SISTEMA INTELIGENTE DE MONITORAMENTO E SEGURANÇA CIBERNÉTICA: APOIO À PROTEÇÃO E ORIENTAÇÃO SOBRE A VULNERABILIDADE DIGITAL (SafeID)

SÃO PAULO
2026

JOAO ANTÔNIO CHAMUSCA MARTINS
KELVYN GABRIEL MILU DE OLIVEIRA
LUCAS VIEIRA DE SOUZA
MATHEUS PEREIRA PEREIRA

SISTEMA INTELIGENTE DE MONITORAMENTO E SEGURANÇA CIBERNÉTICA: APOIO À PROTEÇÃO E ORIENTAÇÃO SOBRE A VULNERABILIDADE DIGITAL (SafeID)

Projeto apresentado ao Programa do Curso de Tecnologia em Análise e Desenvolvimento de Sistemas, do Instituto Federal de Educação, Ciência e Tecnologia de São Paulo, Campus São Paulo, como requisito parcial para aprovação na disciplina SPOPIE2 – Projeto Integrado de Extensão 2, sob orientação dos Professores Marcelo Tavares de Santana e Johnata Souza Santicioli.

SÃO PAULO

2026

RESUMO

A crescente dependência de serviços digitais por indivíduos e organizações tem gerado um rastro massivo de informações pessoais e de acesso restrito, tornando-as vulneráveis a incidentes de segurança. No cenário atual, a exfiltração de dados em larga escala e a subsequente compilação de bases de credenciais expostas em repositórios de *threat intelligence* representam um risco crítico à privacidade e à integridade dos titulares. Diante dessa realidade, este trabalho apresenta o SafeID, um sistema desenvolvido para permitir ao usuário cadastrado monitorar exposições registradas e oferecer apoio e orientação sobre sua vulnerabilidade digital.

O objetivo central do projeto é consolidar uma plataforma que atue no processamento e na triagem de informações provenientes de fontes externas que catalogam esses incidentes de segurança. Ao contrário de ferramentas de detecção em tempo real, o SafeID foca no mapeamento de dados que já foram comprometidos e estão circulando em repositórios de ameaças conhecidos. O sistema busca democratizar o acesso a essa informação técnica sobre o contexto da vulnerabilidade digital, permitindo que o cidadão comum compreenda se seus dados cadastrais ou credenciais de acesso fazem parte de vazamentos documentados, facilitando a gestão de sua própria identidade digital.

Metodologicamente, a pesquisa utiliza uma abordagem de desenvolvimento ágil, com ciclos iterativos de análise, implementação e validação. A solução é composta por uma interface desenvolvida em React e por serviços de *back-end* em Node.js com NestJS, com consulta a fontes de *threat intelligence* e cálculo de um *score* ponderado, implementados na PI-1, e com integração a um modelo de linguagem de grande escala (LLM), implementada na PI-2, que converte metadados técnicos em orientações acessíveis. A aplicação está implantada em contêineres na Amazon Web Services (AWS) e monitora continuamente os e-mails dos usuários que autorizam esse acompanhamento. O diagnóstico é apoiado por um *score* ponderado segundo a criticidade dos dados expostos. A validação funcional foi feita por testes automatizados com dados sintéticos, sem tratamento desnecessário de dados pessoais reais; em 03/11/2026, a validação não funcional incluiu uma campanha de carga em produção, sem erros, um teste de penetração sem achados de severidade alta ou crítica em aberto e testes de jornada no navegador. Ao final, o projeto indica que a combinação dessas tecnologias pode apoiar a prevenção e a comunicação de riscos, desde que acompanhada de governança, controles técnicos e observância das diretrizes da LGPD.

Palavras-chave: Segurança cibernética. Inteligência de Ameaças. LGPD. Inteligência Artificial. SafeID.

ABSTRACT

The growing reliance on digital services by individuals and organizations has generated a massive trail of personal and restricted access information, making them vulnerable to security incidents. In the current scenario, large-scale data exfiltration and the subsequent compilation of exposed credential databases in threat intelligence repositories represent a critical risk to the privacy and integrity of data subjects. Given this reality, this paper presents SafeID, a system developed to allow registered users to monitor recorded exposures and offer support and guidance on their digital vulnerability.

The central objective of the project is to consolidate a platform that operates in the processing and screening of information from external sources that catalog these security incidents. Unlike real-time detection tools, SafeID focuses on mapping data that has already been compromised and is circulating in known threat repositories. The system seeks to democratize access to this technical information regarding the context of digital vulnerability, allowing ordinary citizens to understand if their registration data or access credentials are part of documented leaks, facilitating the management of their own digital identity.

Methodologically, the research uses an agile development approach to create an application based on React and Node.js with NestJS as back-end framework. The system's differential lies in the combination of a weighted risk score, implemented in PI-1, with a large language model (LLM), integrated in PI-2, to translate technical incident records into practical mitigation guides. Furthermore, SafeID delivers a visual risk index — the "exposure score" — together with personalized action plans and continuous monitoring with e-mail alerts, running in containers on Amazon Web Services (AWS). Functional validation relied on automated tests with synthetic data; on 3 November 2026, non-functional validation included a load test in production with no errors, a penetration test with no open high- or critical-severity findings and end-to-end browser tests. In this way, the platform is designed in alignment with the guidelines of the General Data Protection Law (LGPD) by strengthening informative self-determination and the culture of preventive security in Brazil.

Keywords: Cybersecurity. Threat Intelligence. LGPD. Artificial Intelligence. SafeID.

LISTA DE FIGURAS

Figura 1 – Diagrama de atividades do processo de consulta e diagnóstico.....34
Figura 2 – Diagrama de casos de uso do SafeID.....37
Figura 3 – Diagrama de classes conceitual do domínio.....41
Figura 4 – Diagrama de arquitetura do SafeID.....47
Figura 5 – Diagrama de componentes do SafeID.....48
Figura 6 – Modelo Entidade-Relacionamento (MER) do SafeID.....52
Figura 7 – Diagrama Entidade-Relacionamento (DER) do SafeID.....53
Figura 8 – Diagrama de classes de projeto do SafeID.....54
Figura 9 – Representação esquemática (wireframe) do painel de resultado e do estado de indisponibilidade.....57
Figura 10 – Diagrama de sequência da consulta de exposição (UC01).....59
Figura 11 – Diagrama de estados do controle de acesso.....69
Figura 12 – Diagrama de estados do Circuit Breaker de integração externa.....70
Figura 13 – Quadro de atividades do SafeID no GitHub Projects (recorte).....75
Figura 14 – Diagrama de pacotes da aplicação.....78
Figura 15 – Diagrama de implantação do MVP da PI-1 (PaaS).....80
Figura 16 – Diagrama de implantação da arquitetura em produção na AWS.....81
Figura 17 – Roadmap de desenvolvimento das atividades do SafeID.....112

LISTA DE QUADROS

Quadro 1 – Comparativo de funcionalidades entre concorrentes.....23
Quadro 2 – Atores do sistema e seus papéis.....35
Quadro 3 – Requisitos funcionais (RF).....42
Quadro 4 – Requisitos não funcionais (RNF).....43
Quadro 5 – Dicionário de dados do SafeID.....54
Quadro 6 – Matriz de rastreabilidade (requisitos versus testes).....88
Quadro 7 – Matriz de responsabilidades e atribuições da equipe.....104
Quadro 8 – Matriz de riscos do projeto SafeID.....105
Quadro 9 – Backlog do produto (PI-1 e PI-2).....107
Quadro 10 – Cronograma consolidado e macrofases de engenharia do projeto.....111

LISTA DE TABELAS

Tabela 1 – Categorias e pesos dos metadados obtidos da API.....29
Tabela 2 – Metas e resultados de desempenho, carga e resiliência (k6 e Opossum).....85
Tabela 3 – Matriz de pesos e criticidade do score de risco (RN02 e RN03).....87
Tabela 4 – Cobertura da suíte unitária do back-end (medição de 04/11/2026).....90
Tabela 5 – Custos operacionais mensais (OpEx) estimados por categoria.....113
Tabela 6 – Cenários financeiros sobre base comum de usuários ativos.....116
Tabela 7 – Projeção de 36 meses e análise de sensibilidade.....117

LISTA DE ABREVIATURAS E SIGLAS

ABNT – Associação Brasileira de Normas Técnicas

ACID – Atomicidade, Consistência, Isolamento e Durabilidade

ACM – AWS Certificate Manager

AES – *Advanced Encryption Standard* (Padrão de Criptografia Avançada)

ALB – *Application Load Balancer*

ANPD – Autoridade Nacional de Proteção de Dados

API – *Application Programming Interface* (Interface de Programação de Aplicações)

AWS – *Amazon Web Services*

B2B2C – *Business-to-Business-to-Consumer*

B2C – *Business-to-Consumer*

CAC – Custo de Aquisição de Cliente

CDN – *Content Delivery Network* (Rede de Distribuição de Conteúdo)

CI/CD – *Continuous Integration / Continuous Deployment* (Integração Contínua / Implantação Contínua)

CNPJ – Cadastro Nacional da Pessoa Jurídica

CNH – Carteira Nacional de Habilitação

CPF – Cadastro de Pessoas Físicas

CPU – *Central Processing Unit* (Unidade Central de Processamento)

CRUD – *Create, Read, Update, Delete* (Criar, Ler, Atualizar, Excluir)

CSP – *Content Security Policy* (Política de Segurança de Conteúdo)

CSS – *Cascading Style Sheets* (Folhas de Estilo em Cascata)

CVSS – *Common Vulnerability Scoring System*

DBA – *Database Administrator* (Administrador de Banco de Dados)

DDL – *Data Definition Language* (Linguagem de Definição de Dados)

DER – Diagrama Entidade-Relacionamento

DML – *Data Manipulation Language* (Linguagem de Manipulação de Dados)

DTO – *Data Transfer Object* (Objeto de Transferência de Dados)

E2E – *End-to-End* (Ponta a Ponta)

ECS – *Elastic Container Service*

ECR – Elastic Container Registry

GCM – *Galois/Counter Mode*

HIBP – *Have I Been Pwned*

HMAC – Hash-based Message Authentication Code (Código de Autenticação de Mensagem Baseado em Hash)

HTTPS – *Hypertext Transfer Protocol Secure*

IA – Inteligência Artificial

IAM – *Identity and Access Management*

IEC – Comissão Eletrotécnica Internacional

IFSP – Instituto Federal de Educação, Ciência e Tecnologia de São Paulo

IP – *Internet Protocol*

ISO – Organização Internacional para Padronização

JSON – *JavaScript Object Notation* (Notação de Objetos JavaScript)

JWT – *JSON Web Token*

LGPD – Lei Geral de Proteção de Dados Pessoais

LLM – *Large Language Model* (Modelo de Linguagem de Grande Escala)

LTV – *Lifetime Value*

MER – Modelo Entidade-Relacionamento

MFA – *Multi-Factor Authentication* (Autenticação em Múltiplos Fatores)

MVP – *Minimum Viable Product* (Produto Mínimo Viável)

NAT – Network Address Translation (Tradução de Endereços de Rede)

NIST – *National Institute of Standards and Technology*

NLP – *Natural Language Processing* (Processamento de Linguagem Natural)

ORM – *Object-Relational Mapping* (Mapeamento Objeto-Relacional)

OSINT – *Open Source Intelligence* (Inteligência de Fontes Abertas)

OWASP – Open Worldwide Application Security Project

P95 – Percentil 95

PaaS – *Platform as a Service* (Plataforma como Serviço)

PI-1 / PI-2 – Projeto Integrado de Extensão 1 / Projeto Integrado de Extensão 2

PII – *Personally Identifiable Information* (Informações Pessoais Identificáveis)

PO – *Product Owner* (Dono do Produto)

PoC – *Proof of Concept* (Prova de Conceito)

RAM – *Random Access Memory* (Memória de Acesso Aleatório)

RDS – *Relational Database Service*

REST – *Representational State Transfer*

RF – Requisito Funcional

RN – Regra de Negócio

RNF – Requisito Não Funcional

RPM – Requisições por Minuto

SaaS – *Software as a Service* (Software como Serviço)

SAST – Static Application Security Testing (Teste Estático de Segurança de Aplicações)

SCA – Software Composition Analysis (Análise de Composição de Software)

SES – Simple Email Service (Amazon)

SGBD – Sistema Gerenciador de Banco de Dados

SGD – Secretaria de Governo Digital

SHA – *Secure Hash Algorithm*

SPA – *Single Page Application* (Aplicação de Página Única)

SQL – *Structured Query Language*

SSM – AWS Systems Manager (Parameter Store)

TLS – *Transport Layer Security*

UC – Caso de Uso

UDF – *User Defined Functions* (Funções Definidas pelo Usuário)

UML – *Unified Modeling Language* (Linguagem de Modelagem Unificada)

US – *User Story* (História de Usuário)

UX – *User Experience* (Experiência do Usuário)

VPC – *Virtual Private Cloud*

XSS – *Cross-Site Scripting*

SUMÁRIO

1 INTRODUÇÃO.....13
1.1 Caracterização do Projeto.....13
1.2 Objetivos.....14
1.2.1 Objetivo Geral.....15
1.2.2 Objetivos Específicos.....15
1.3 Justificativa.....15
1.4 Escopo do Projeto.....16
1.5 Estrutura do Documento.....17
2 CONTEXTO DO PROJETO.....19
2.1 Cenário.....19
2.1.1 A Economia da Informação e a Evolução das Ameaças Digitais.....19
2.1.2 Custódia de Dados, Fadiga de Segurança e o "Gap" de Detecção.....20
2.1.3 Marco Regulatório (LGPD) e a Literacia Digital no Brasil.....21
2.1.4 Parceiro do Projeto e Modelo de Atuação.....22
2.1.5 Soluções Existentes no Mercado.....22
2.2 Público-Alvo.....23
2.3 Necessidades Identificadas.....24
2.3.1 O Problema da Interpretação das Exposições.....24
2.3.2 A Necessidade de uma Medida Explicável de Risco.....24
2.3.3 Síntese e Lacuna.....25
2.4 Regras de Negócio.....26
2.4.1 Modelo de Avaliação de Risco e Ponderação de Exposição (Risk Score).....28
2.4.1.1 Fundamentação teórica e normativa.....28
2.4.1.2 Taxonomia e tabela de pesos por categoria de dado (data classes).....29
2.4.1.3 Modelagem matemática do algoritmo.....31
2.4.1.4 Exemplo numérico prático e auditável.....32
2.4.2 Fluxo de Atividades das Regras de Negócio.....33
3 ANÁLISE DO SISTEMA.....35
3.1 Visão Geral do Sistema.....35
3.2 Atores Envolvidos.....35
3.3 Casos de Uso.....36
3.4 Modelo Conceitual.....40
3.5 Requisitos Funcionais.....42

3.6 Requisitos Não Funcionais....43
3.7 Restrições do Sistema....44
4 PROJETO DO SISTEMA....46
4.1 Arquitetura do Sistema....46
4.2 Projeto dos Componentes....49
4.3 Modelo de Dados....51
4.3.1 Modelo Entidade-Relacionamento (MER)....51
4.3.2 Diagrama Entidade-Relacionamento (DER)....52
4.3.3 Diagrama de Classes de Projeto....53
4.3.4 Dicionário de Dados....54
4.4 Projeto da Interface do Usuário....56
4.5 Diagramas de Interação....57
4.6 Tecnologias Utilizadas....60
4.6.1 Front-end: React.js e Tailwind CSS....60
4.6.2 Back-end: Node.js, NestJS e Prisma ORM....61
4.6.3 Banco de Dados: PostgreSQL e Redis....62
4.6.4 Inteligência Artificial e APIs Externas....63
4.6.5 Infraestrutura em Nuvem....64
4.7 Segurança e Controle de Acesso....66
4.7.1 Critérios de Segurança e Privacidade....66
4.7.2 Autenticação, Autorização e Disponibilidade....67
4.7.3 Registro de Eventos (Logs) e Auditoria....71
4.7.4 Observância à Legislação....71
5 IMPLEMENTAÇÃO....74
5.1 Ambiente de Desenvolvimento....74
5.2 Estrutura da Aplicação....77
5.3 Implementação dos Componentes....78
5.4 Integração dos Módulos....79
5.5 Testes Realizados....81
5.5.1 Plano de Testes....82
5.5.2 Testes Funcionais....82
5.5.2.1 Testes unitários....82
5.5.2.2 Testes de componente....83
5.5.2.3 Testes de integração....83
5.5.2.4 Testes de ponta a ponta (E2E)....84

5.5.3 Testes Não Funcionais.....84
5.5.3.1 Estratégia integrada de validação.....85
5.5.3.2 Validação de desempenho, carga, resiliência e segurança.....85
5.5.3.3 Matriz do score de risco e regras RN02 e RN03.....87
5.5.3.4 Exemplo prático de cálculo para verificação.....87
5.5.3.5 Matriz de rastreabilidade.....88
5.5.3.6 Governança, conformidade e manutenibilidade.....89
5.5.4 Testes Automatizados.....89
5.5.4.1 Fluxo de desenvolvimento orientado a testes.....89
5.5.4.2 Cobertura de testes.....90
5.6 Resultados Obtidos.....91
5.7 Limitações da Implementação.....92
6 CONCLUSÃO.....94
6.1 Considerações Finais.....94
6.2 Dificuldades Encontradas.....95
6.2.1 Dificuldades Técnicas.....95
6.2.2 Escolhas e Descartes.....96
6.3 Trabalhos Futuros.....98
REFERÊNCIAS.....100
APÊNDICE A – GESTÃO DO PROJETO.....103
A.1 Organização da Equipe.....103
A.2 Metodologia de Gestão e Desenvolvimento (Scrum).....104
A.3 Gestão de Riscos do Projeto.....105
A.4 Backlog do Produto.....106
A.5 Detalhamento das Sprints (PI-1 – março a junho).....108
A.6 Planejamento para a PI-2 (agosto a novembro).....108
APÊNDICE B – CRONOGRAMA DO PROJETO.....110
APÊNDICE C – VIABILIDADE FINANCEIRA.....113
C.1 Custos.....113
C.2 Receitas.....114
C.3 Cenário Realista.....115
C.4 Cenário Otimista.....115
C.5 Cenário Pessimista.....116
C.6 Métricas SaaS: CAC e LTV.....117
ANEXOS.....119

1 INTRODUÇÃO

Este capítulo apresenta o SafeID de forma geral, estabelecendo sua caracterização, seus objetivos, sua justificativa, seu escopo e a organização do documento. Sua função é oferecer ao leitor uma visão inicial do propósito e da abrangência do trabalho antes da apresentação do contexto e dos aspectos técnicos da solução.

1.1 Caracterização do Projeto

A ampliação da circulação de dados pessoais em serviços digitais elevou a exposição dos titulares a incidentes cibernéticos. Em 2022, o Brasil registrou mais de 30 bilhões de tentativas de ataques cibernéticos, o segundo maior volume da América Latina (FORTINET, 2022). Esse indicador dimensiona a superfície de ataque, mas não informa ao titular quais de seus dados foram comprometidos, qual é a gravidade da exposição ou como reagir.

Essa lacuna caracteriza uma assimetria informacional: enquanto especialistas e agentes maliciosos operam com registros técnicos e bases de credenciais expostas, o usuário comum recebe alertas fragmentados, linguagem especializada e recomendações genéricas. Mesmo quando toma conhecimento de um vazamento, pode não compreender o risco nem priorizar medidas de mitigação (CONTI; DARGAHI; DEHGHANTANHA, 2018a; CHEN; WANG, 2024).

O problema enfrentado pelo SafeID, portanto, não se limita à identificação de vazamentos. Ele envolve transformar evidências técnicas em comunicação de risco compreensível e acionável, considerando barreiras como a reutilização de senhas (CONTI *et al.*, 2018b), a baixa adoção de autenticação em múltiplos fatores e o desconhecimento sobre exposições documentadas (NEMEC ZLATOLAS; WELZER; LHOTSKA, 2024). Sem essa mediação, a informação disponível tende a produzir inércia, e não prevenção.

Para enfrentar essa assimetria, este trabalho propõe o SafeID, plataforma web de apoio à decisão voltada à fase pós-incidente, cuja arquitetura prevê a consulta de exposições registradas em fontes de inteligência de ameaças, o cálculo de um índice de risco e a tradução dos resultados em orientações acessíveis. O projeto foi desenvolvido no Curso de Tecnologia em Análise e Desenvolvimento de Sistemas do Instituto Federal de Educação, Ciência e Tecnologia de São Paulo (IFSP), Campus São Paulo, no âmbito das disciplinas SPOPIE1 e SPOPIE2 – Projeto Integrado de Extensão 1 e 2, em parceria com a empresa Enetsec, que contribui com consultoria especializada em segurança da informação.

Neste documento, a PI-1 designa a etapa realizada de março a junho de 2026, na qual foram implementados a consulta às exposições e a avaliação determinística do risco; a PI-2 designa a etapa realizada de agosto a novembro de 2026, encerrada com a apresentação final em 13/11/2026, que integrou inteligência artificial para comunicar a gravidade de cada incidente e indicar ações de mitigação compatíveis com os dados comprometidos (REUBEN-OWOH; HAIG, 2025). Na PI-1, foram implementados o processamento de metadados provenientes de APIs de monitoramento de exposições, como o Have I Been Pwned (HIBP), e os componentes de consulta e de cálculo do risco; a PI-2 consolidou a autenticação com verificação de e-mail, o histórico criptografado, o monitoramento ativo com alertas, a assistência cognitiva, a implantação na AWS e a validação por testes de carga, de segurança e de jornada no navegador. A solução organiza-se em três componentes centrais:

1. **Mapeamento Automatizado (implementado na PI-1 e estendido ao monitoramento contínuo na PI-2):** realiza a triagem de incidentes vinculados ao identificador do usuário em repositórios de ameaças;
2. **Cálculo de *Risk Score* (implementado na PI-1):** aplica um algoritmo ponderado que quantifica a gravidade da exposição com base na natureza do dado, atribuindo, por exemplo, peso superior às senhas em relação aos nomes;
3. **Assistente Cognitivo (implementado na PI-2):** utiliza um modelo de linguagem (LLM) para converter diagnósticos técnicos em planos de ação personalizados e compreensíveis, com resposta padrão determinística quando o modelo não responde.

A aplicação encontra-se disponível publicamente no endereço https://www.safeid.app.br, com código-fonte mantido no repositório https://github.com/Safe-ID/safeid-main. Nela, o usuário cria uma conta, confirma o e-mail e recebe o diagnóstico de risco e o plano de ação gerado pela IA, podendo ainda recuperar ou trocar a senha, baixar uma cópia dos seus dados e excluir a conta; os recursos não implementados (gestão de múltiplos identificadores e painel administrativo) estão descritos na Seção 6.3.

1.2 Objetivos

Esta seção apresenta os objetivos que orientam o desenvolvimento do SafeID. Estabelece-se primeiro o objetivo geral e, em seguida, os objetivos específicos, cada um identificado pela fase do projeto (PI-1 ou PI-2) para permitir a verificação de seu status na Seção 5.6.

1.2.1 Objetivo Geral

Desenvolver a plataforma SafeID, um sistema de apoio à decisão voltado à segurança cibernética preventiva, capaz de processar informações de repositórios de *threat intelligence* sobre incidentes de exposição de dados, diagnosticar a exposição de dados pessoais do usuário e fornecer diretrizes de mitigação personalizadas por meio de inteligência artificial.

1.2.2 Objetivos Específicos

Os objetivos específicos a seguir detalham as etapas necessárias para o cumprimento do objetivo geral e indicam a fase do projeto (PI-1 ou PI-2) em que cada um é tratado:

- **Investigar, selecionar e consumir** APIs de monitoramento de exposições de incidentes de segurança *Open Source Intelligence* (OSINT) que permitam a triagem automatizada de credenciais e dados cadastrais expostos (Fase PI-1, concluído);
- **Modelar e implementar** um algoritmo de cálculo de *Risk Score* (índice de risco) que pondere a criticidade do vazamento com base na natureza dos dados comprometidos reportados pelas APIs (Fase PI-1, concluído);
- **Desenvolver** uma interface web responsiva que utilize elementos de visualização de dados (como o termômetro de risco) para facilitar a interpretação dos usuários (Fase PI-1, concluído);
- **Projetar** a arquitetura de persistência para o histórico de consultas e os metadados dos incidentes (base implementada na PI-1; funções de conta e histórico completo implementadas na PI-2, exceto múltiplos identificadores por usuário);
- **Integrar** um modelo de linguagem de grande escala (LLM) para atuar como assistente cognitivo, traduzindo metadados técnicos em planos de ação práticos e acessíveis (Fase PI-2, concluído);
- **Aplicar** os princípios de *Privacy by Design* e as diretrizes da Lei Geral de Proteção de Dados (LGPD), assegurando o tratamento ético e a proteção das informações coletadas durante o ciclo de uso do sistema (Fase PI-2, concluído).

1.3 Justificativa

A relevância do projeto está na necessidade de reduzir as barreiras técnicas que dificultam ao cidadão exercer sua autodeterminação informativa, princípio assegurado pela Lei Geral de Proteção de Dados (BRASIL, 2018). A literatura aponta que práticas inadequadas dos próprios titulares, como a reutilização de senhas e a baixa adoção de

autenticação em múltiplos fatores, ampliam o risco decorrente de vazamentos (NEMEC ZLATOLAS; WELZER; LHOTSKA, 2024).

A parceria com a Enetsec aproxima o desenvolvimento acadêmico da experiência do setor privado. A empresa contribui com consultoria especializada e com a validação da proposta por profissionais de segurança da informação, o que apoia a verificação dos requisitos do projeto e a análise de viabilidade técnica. Essa colaboração também sustenta a evolução prevista para um modelo B2B2C, no qual organizações parceiras disponibilizariam a seus clientes recursos de monitoramento, avaliação de risco e orientação preventiva.

A justificativa organiza-se em quatro dimensões:

- **Social:** a ferramenta oferece orientação clara em um cenário em que o desconhecimento sobre vazamentos facilita fraudes financeiras e o roubo de identidade;
- **Tecnológica:** o projeto combina a consulta a APIs de segurança e o processamento de linguagem natural para tratar um problema de comunicação de risco, em linha com a defesa de mensagens personalizadas feita por Reuben-Owoh e Haig (2025);
- **Normativa:** ao facilitar o diagnóstico de exposições históricas, o sistema apoia o direito de acesso à informação, pois a mera divulgação de fatores de risco, sem estrutura interpretativa, não informa adequadamente o titular (CHEN; HENRY; JIANG, 2023);
- **Formativa:** o desenvolvimento integra, em um problema real apresentado por um parceiro externo, conteúdos de engenharia de requisitos, modelagem, arquitetura de software, segurança da informação, testes automatizados e gestão ágil, contribuindo para a formação técnica e extensionista dos estudantes envolvidos.

O diferencial do SafeID está na combinação de uma experiência centrada no usuário, da apresentação acessível do nível de risco e da tradução de dados técnicos em orientações práticas, aspecto confirmado pela análise das soluções existentes no mercado apresentada na Seção 2.1.5.

1.4 Escopo do Projeto

O escopo do projeto SafeID compreende o desenvolvimento de uma aplicação web voltada à literacia e à proteção de dados pessoais, atuando especificamente na camada de pós-incidente de segurança. A solução propõe-se a servir como uma ponte inteligível entre bases de dados técnicas de vazamentos (*data breaches*) e o usuário final, utilizando inteligência artificial para personalizar orientações de mitigação.

O que será desenvolvido (inclusões):

- **Módulo de Consulta:** verificação automática do e-mail informado no cadastro (e, futuramente, de outros identificadores) em bases de dados OSINT (*Open Source Intelligence*), utilizando integração via API com provedores consolidados no mercado de cibersegurança.
- **Motor de Análise de Risco:** algoritmo que processa a natureza do vazamento (por exemplo, exposição de senhas, endereços IP e nomes completos) e atribui uma métrica de gravidade ponderada (*Risk Score*).
- **Assistente de Orientação (IA):** integração com modelos de linguagem de grande escala (LLM) para traduzir metadados técnicos em planos de ação preventivos e compreensíveis para o público leigo.
- **Painel de Controle (*Dashboard*):** visualização histórica das consultas realizadas e evolução do nível de exposição do usuário.

O que não será desenvolvido (exclusões):

- **Recuperação de Contas:** o sistema não realizará ações diretas em contas de terceiros (por exemplo, o SafeID não trocará automaticamente a senha do e-mail do usuário).
- **Monitoramento de Rede Local:** a aplicação não atuará como antivírus ou *firewall* residente no dispositivo do usuário.
- **Venda de Proteção Financeira:** o projeto não oferece seguros ou garantias financeiras contra fraudes, limitando-se à orientação e ao monitoramento informativo.

O projeto é entregue em duas etapas: no primeiro semestre, um **Protótipo Funcional (PoC)** para validar a comunicação com APIs e a lógica de risco; no segundo semestre, a **aplicação completa**, com interface finalizada, sistema de autenticação, assistente cognitivo integrado, monitoramento contínuo e implantação na AWS. As premissas e restrições que delimitam a operação do sistema são detalhadas na Seção 3.7.

1.5 Estrutura do Documento

O documento organiza-se em seis capítulos, seguidos dos elementos pós-textuais. Após esta introdução, o Capítulo 2 apresenta o contexto do projeto, caracterizando o cenário das ameaças digitais e do marco regulatório, o parceiro e as soluções existentes no mercado, o público-alvo, as necessidades identificadas e as regras de negócio, incluindo o modelo de avaliação de risco. O Capítulo 3 dedica-se à análise do sistema, com a visão geral da solução, os atores, os casos de uso, o modelo conceitual, os requisitos funcionais e não funcionais e as

restrições. O Capítulo 4 descreve o projeto do sistema: arquitetura, componentes, modelo de dados, interface, interações, tecnologias e mecanismos de segurança e controle de acesso.

O Capítulo 5 trata da implementação, abrangendo o ambiente de desenvolvimento, a estrutura da aplicação, a implementação e a integração dos componentes, os testes realizados, os resultados obtidos e as limitações. O Capítulo 6 apresenta a conclusão, com as considerações finais, as dificuldades encontradas e os trabalhos futuros. Por fim, as Referências relacionam as fontes utilizadas; o Apêndice A documenta a gestão do projeto (equipe, metodologia, riscos e *backlog*); o Apêndice B apresenta o cronograma; e o Apêndice C reúne a análise de viabilidade financeira.

2 CONTEXTO DO PROJETO

Este capítulo apresenta o contexto real no qual o SafeID está inserido, caracterizando o cenário das ameaças digitais e da proteção de dados, a área de atuação do parceiro e as soluções disponíveis no mercado, o público envolvido, as necessidades identificadas e as regras de negócio do domínio. Sua função é estabelecer a compreensão do problema sob a perspectiva da comunidade atendida antes da apresentação da solução computacional. Os fundamentos teóricos e normativos que sustentam o projeto, originalmente reunidos em uma revisão de literatura, são aqui incorporados à caracterização do cenário e das necessidades.

2.1 Cenário

O SafeID aplica-se ao cenário da segurança da informação voltada ao cidadão, especificamente à etapa posterior aos incidentes de vazamento de dados. As subseções a seguir percorrem a evolução das ameaças na economia da informação (2.1.1), os desafios de custódia e comunicação de incidentes (2.1.2), a relação entre a LGPD e a literacia digital no Brasil (2.1.3), a área de atuação do parceiro e o modelo de atuação previsto (2.1.4) e as soluções existentes no mercado (2.1.5).

2.1.1 A Economia da Informação e a Evolução das Ameaças Digitais

A economia digital elevou os dados pessoais à condição de ativos estratégicos, distribuídos entre serviços, dispositivos e organizações. Von Solms e Van Niekerk (2013) interpretam essa mudança como uma ampliação do próprio campo da cibersegurança: a proteção deixa de se limitar à infraestrutura corporativa e passa a abranger a identidade e a conduta do indivíduo. Essa perspectiva fornece o fundamento de governança pessoal do problema, mas, isoladamente, não explica como os dados comprometidos continuam produzindo risco após a violação.

Conti, Dargahi e Dehghantanha (2018a) deslocam a análise para o ciclo posterior ao comprometimento: dados exfiltrados deixam de constituir episódios isolados e passam a alimentar mercados secundários e novas campanhas abusivas. Nesse ponto, sua abordagem de *Threat Intelligence* complementa Von Solms e Van Niekerk (2013), pois transforma a responsabilidade ampla de proteção em atividade baseada em evidências. Há, contudo, uma tensão relevante: enquanto a inteligência de ameaças tende a privilegiar capacidade analítica e antecipação técnica, a governança pessoal exige que seus resultados sejam compreensíveis e

acionáveis pelo titular. A efetividade da defesa depende, portanto, tanto da qualidade da evidência coletada quanto de sua tradução para decisões individuais.

No Brasil, o volume de mais de 30 bilhões de tentativas de ataque registrado em 2022 (FORTINET, 2022), já mencionado na Introdução, dimensiona a superfície de ataque, mas não permite, por si só, avaliar a gravidade de cada exposição. No plano global, o custo médio de um vazamento de dados atingiu US$ 4,45 milhões em 2023, 15% acima do valor registrado três anos antes (IBM SECURITY, 2023). À luz de Von Solms e Van Niekerk (2013), esses indicadores evidenciam a necessidade de proteção orientada ao indivíduo; sob a perspectiva de Conti, Dargahi e Dehghantanha (2018a), demonstram a importância de organizar evidências dispersas para identificar padrões e priorizar respostas. Essa convergência sustenta ferramentas que não apenas localizem incidentes, mas também convertam seu conteúdo em informação útil ao usuário.

2.1.2 Custódia de Dados, Fadiga de Segurança e o "Gap" de Detecção

A proteção individual é condicionada pela forma como organizações custodiam, detectam e comunicam incidentes envolvendo dados pessoais. A IBM Security (2023) estima em aproximadamente 277 dias o ciclo médio entre a ocorrência e a contenção de um vazamento. Esse intervalo mostra que a governança defendida por Von Solms e Van Niekerk (2013) não pode ser transferida integralmente ao indivíduo: durante meses, o titular pode desconhecer uma exposição originada em sistemas que não controla. A inteligência baseada em evidências proposta por Conti, Dargahi e Dehghantanha (2018a) reduz parte dessa assimetria ao correlacionar registros de ameaças, mas sua contribuição permanece incompleta se o resultado chegar tarde ou em linguagem inacessível.

Nesse intervalo entre exposição, detecção e reação, a "fadiga de segurança" amplia a vulnerabilidade. Nemec Zlatolas, Welzer e Lhotska (2024) associam a gestão complexa de credenciais e a baixa adoção de autenticação em múltiplos fatores à dificuldade de incorporar práticas protetivas à rotina. A contribuição desses autores qualifica o diagnóstico de Conti, Dargahi e Dehghantanha (2018a): dispor de inteligência sobre ameaças não garante, por si só, mudança de comportamento. Se a evidência técnica aumenta a carga cognitiva, ela pode produzir inércia em vez de prevenção. Em contraste, a perspectiva centrada no indivíduo de Von Solms e Van Niekerk (2013) exige que controles e mensagens sejam compatíveis com a capacidade real de compreensão e ação do usuário.

Chen, Henry e Jiang (2023) aprofundam esse argumento ao observar que a divulgação de fatores de risco, desacompanhada de uma estrutura interpretativa, não informa

adequadamente o titular sobre o impacto da exposição. A posição converge com Nemec Zlatolas, Welzer e Lhotska (2024) quanto ao limite das respostas que apenas impõem tarefas ao usuário, porém desloca o foco da gestão de credenciais para a qualidade da comunicação. Santos e Pinto (2025) reforçam que barreiras cognitivas e linguísticas distorcem a percepção de risco, enquanto Chen e Wang (2024) defendem a integração entre resposta técnica e suporte educacional. Em conjunto, esses trabalhos indicam que reduzir o hiato de detecção requer uma cadeia completa: obtenção de evidências, avaliação de gravidade, comunicação inteligível e indicação de contramedidas. O SafeID posiciona-se nessa interface entre inteligência técnica e apoio efetivo à decisão.

2.1.3 Marco Regulatório (LGPD) e a Literacia Digital no Brasil

A Lei Geral de Proteção de Dados Pessoais (LGPD — Lei nº 13.709/2018; BRASIL, 2018) consolidou no Brasil princípios de transparência, prevenção e autodeterminação informativa. Entretanto, a existência do direito formal não assegura seu exercício material. Chen, Henry e Jiang (2023) sustentam que a simples divulgação de fatores de risco é insuficiente quando o titular não dispõe de uma estrutura para interpretar o impacto sobre sua privacidade. Essa crítica aproxima o marco regulatório da discussão de Nemec Zlatolas, Welzer e Lhotska (2024): tanto a sobrecarga operacional quanto a opacidade informacional podem impedir respostas adequadas, ainda que o usuário tenha recebido um alerta.

A mediação tecnológica torna-se, portanto, condição para aproximar rigor normativo e prática cotidiana. Reuben-Owoh e Haig (2025) defendem que padrões de cibersegurança precisam ser acompanhados por mecanismos de comunicação personalizada. Essa proposição converge com Chen, Henry e Jiang (2023), mas acrescenta uma dimensão operacional: não basta informar; é necessário adequar a mensagem ao contexto decisório. Em contraponto, a inteligência de ameaças discutida por Conti, Dargahi e Dehghantanha (2018a) prioriza a qualidade e a correlação das evidências. A integração das duas vertentes indica que conformidade, análise técnica e literacia digital devem operar de forma articulada, sem que uma substitua a outra.

Nesse contexto, o SafeID é concebido como instrumento de apoio à conformidade e à cidadania digital, e não como substituto das obrigações dos agentes de tratamento. Ao combinar monitoramento, pontuação explicável e orientação em linguagem acessível, a proposta busca tornar o direito à informação operacional para o cidadão. Chen e Wang (2024) defendem que a gestão social de incidentes exige integrar resposta técnica e suporte educacional; em diálogo com Von Solms e Van Niekerk (2013), essa integração amplia a

cibersegurança para além do perímetro organizacional e a situa no exercício cotidiano da autonomia informacional.

2.1.4 Parceiro do Projeto e Modelo de Atuação

O projeto conta com a parceria da Enetsec, empresa do setor de segurança da informação, que aproxima o desenvolvimento acadêmico da experiência do setor privado. A empresa contribui com consultoria especializada e com a validação da proposta por profissionais de segurança da informação, o que apoia a verificação dos requisitos do projeto e a análise de viabilidade técnica.

A atuação prevista para o SafeID combina duas frentes de mercado. Na frente B2C (*Business-to-Consumer*), o produto estrutura-se como um SaaS (*Software as a Service*) *Freemium*: consultas básicas gratuitas de utilidade pública, coerentes com o propósito de Responsabilidade Social Digital do projeto, e um plano recorrente com monitoramento proativo e orientações personalizadas. Na frente B2B2C (*Business-to-Business-to-Consumer*), sustentada pela colaboração com o parceiro, organizações disponibilizariam a seus próprios clientes recursos de monitoramento, avaliação de risco e orientação preventiva. A análise detalhada de custos, receitas e cenários desse modelo encontra-se no Apêndice C.

2.1.5 Soluções Existentes no Mercado

O mercado oferece desde agregadores de incidentes até suítes de proteção de identidade. A análise a seguir indica que a maioria das soluções entrega o dado bruto, sem uma camada de interpretação voltada ao usuário leigo.

Have I Been Pwned (HIBP). O HIBP é um agregador de incidentes de referência e a principal fonte de dados do SafeID (HAVE I BEEN PWNED, [s. d.]). Sua interface é técnica e informativa. Como limitação, o serviço opera no modelo “consulta e resposta”, sem métrica de gravidade (*score*) nem plano de mitigação além de sugestões genéricas, como a troca de senhas.

Mozilla Monitor. O serviço da Mozilla atua como camada de interface sobre os dados do HIBP, com experiência mais simples para o usuário. Como limitação, embora ofereça alertas por e-mail, as recomendações permanecem estáticas, sem análise cognitiva dos vazamentos para gerar orientações específicas a cada perfil de incidente.

Soluções de identidade (por exemplo, LifeLock e Aura). São plataformas comerciais pagas que oferecem monitoramento extensivo, incluindo pontuação de crédito e vigilância contínua. Como limitação, além do custo, apresentam configuração mais complexa

e costumam estar integradas a serviços bancários e de crédito, o que as distancia de uma ferramenta ágil e focada na literacia em cibersegurança do cidadão comum.

A classificação dos concorrentes foi realizada por meio de análise comparativa da documentação oficial de cada solução e de testes manuais das funcionalidades disponíveis em suas versões gratuitas, quando aplicável. Foram observados critérios comuns de consulta a incidentes, alertas, acessibilidade da interface, cálculo de risco, uso de assistência cognitiva e foco em educação digital. Por tratar-se de verificação qualitativa e sujeita à evolução das ferramentas, o Quadro 1 deve ser lido como retrato do momento da análise.

Quadro 1 – Comparativo de funcionalidades entre concorrentes

Funcionalidade	**HIBP**	**Mozilla Monitor**	**Soluções Pagas**	**SafeID**
Consulta de incidentes (OSINT)	Sim	Sim	Sim	**Sim**
Alertas de novas exposições	Não*	Sim	Sim	**Sim**
Interface acessível	Não	Sim	Parcial	**Sim**
Score de risco ponderado	Não	Não	Sim	**Sim**
Assistente cognitivo (LLM)	Não	Não	Não	**Sim**
Foco em educação digital	Não	Parcial	Não	**Sim**

*A funcionalidade gratuita de consulta do HIBP não inclui alertas ativos para o usuário comum sem registro de domínio. Fonte: elaborado pelos autores (2026).

2.2 Público-Alvo

O público-alvo do SafeID é composto, prioritariamente, por cidadãos brasileiros usuários de serviços digitais que não possuem formação técnica em segurança da informação. Trata-se de pessoas que mantêm contas em lojas virtuais, redes sociais, aplicativos e serviços financeiros e que, embora expostas aos efeitos de vazamentos, dificilmente interpretam registros técnicos de incidentes ou priorizam contramedidas. Para esse público, a plataforma deve oferecer linguagem simples, indicação visual do nível de risco e orientações práticas.

A partir das histórias de usuário elaboradas para o projeto (Seção 3.3), distinguem-se os seguintes perfis:

- **Usuário cadastrado:** pessoa que deseja acompanhar a evolução de sua exposição ao longo do tempo, manter histórico de consultas e receber alertas; a consulta exige conta com e-mail verificado, o que garante que cada pessoa consulte apenas o próprio endereço (RN07);
- **Responsável familiar:** usuário cadastrado que pretende monitorar identificadores de dependentes em um único painel (US07, não implementado);

- **Administrador do sistema:** responsável pela operação da plataforma, que consulta métricas agregadas e anonimizadas para aprimorar o serviço (US10, não implementado).

Indiretamente, também são afetados pelo projeto as organizações parceiras do modelo B2B2C descrito na Seção 2.1.4 e a comunidade acadêmica do IFSP, que se beneficia da aproximação entre extensão, pesquisa aplicada e demandas reais do setor de segurança da informação.

2.3 Necessidades Identificadas

Esta seção apresenta as necessidades observadas no cenário descrito, que motivaram a proposição da solução, sem ainda detalhar os requisitos do sistema, especificados no Capítulo 3.

2.3.1 O Problema da Interpretação das Exposições

O problema tratado pelo SafeID é a distância entre a existência de informações sobre vazamentos e a capacidade do titular de interpretá-las. Bases de credenciais exfiltradas circulam em repositórios de *threat intelligence* e em mercados secundários, e o custo médio global de um vazamento de dados atingiu US$ 4,45 milhões em 2023 (IBM SECURITY, 2023). Do lado do titular, contudo, as ferramentas de consulta existentes frequentemente entregam resultados em formato técnico bruto, que não permite interpretação imediata do risco (CONTI; DARGAHI; DEHGHANTANHA, 2018a).

Essa lacuna gera inércia: mesmo ciente do vazamento, o indivíduo não compreende a gravidade da exposição nem quais contramedidas priorizar. A questão que orienta este trabalho é, portanto, como converter registros técnicos de exposição em uma medida de risco compreensível e em orientações acionáveis para o usuário comum.

2.3.2 A Necessidade de uma Medida Explicável de Risco

A pontuação de risco constitui um mecanismo de síntese: converte múltiplos atributos de um incidente em uma medida ordinal ou numérica destinada a apoiar a priorização. Em segurança cibernética, essa quantificação não equivale a uma previsão exata do dano; trata-se de uma representação comparável, construída a partir de dimensões como criticidade do dado, possibilidade de exploração, extensão da exposição e temporalidade. A abordagem de inteligência orientada por evidências de Conti, Dargahi e Dehghantanha (2018a) oferece a base para selecionar atributos observáveis, enquanto a concepção ampliada de cibersegurança

de Von Solms e Van Niekerk (2013) impede que a métrica seja reduzida à severidade técnica: o impacto sobre identidade, privacidade e capacidade de reação do titular também deve compor a avaliação.

Modelos de referência, como o *Common Vulnerability Scoring System* (FIRST, 2019), ilustram a lógica de ponderar atributos e normalizar o resultado em uma escala fixa, de 0 a 10. Entretanto, esse tipo de modelo destina-se a medir a severidade técnica de vulnerabilidades, e não a exposição de dados pessoais de um titular, o que justifica um índice próprio para o problema do SafeID. De modo geral, esses modelos atribuem pesos diferenciados aos fatores de risco e normalizam o resultado em faixas interpretáveis. Essa lógica permite distinguir, por exemplo, a exposição de credenciais de autenticação de metadados menos diretamente exploráveis, bem como considerar que a recência pode alterar a probabilidade de uso abusivo. Nemec Zlatolas, Welzer e Lhotska (2024) oferecem suporte comportamental para essa diferenciação ao relacionar reutilização de credenciais, complexidade de gestão e baixa adoção de MFA; Chen, Henry e Jiang (2023), por sua vez, alertam que a pontuação só cumpre função informativa se seus critérios e consequências forem comunicados de modo compreensível. Assim, pesos e limiares devem ser explícitos, justificáveis e auditáveis, para que a aparência de precisão matemática não oculte escolhas normativas do modelo.

Para o SafeID, essa literatura fundamenta um índice explicável de exposição, no qual metadados heterogêneos são categorizados, ponderados e apresentados em níveis de prioridade. O modelo deve funcionar como apoio à decisão, e não como diagnóstico absoluto: uma mesma pontuação pode demandar respostas distintas conforme o tipo de dado comprometido e o contexto do usuário. Os pesos, fatores temporais, a normalização e as faixas do modelo são definidos como regras de negócio na Seção 2.4.

2.3.3 Síntese e Lacuna

A literatura examinada oferece contribuições consistentes, porém fragmentadas:

1. Von Solms e Van Niekerk (2013) ampliam a cibersegurança para a governança pessoal;
2. Conti, Dargahi e Dehghantanha (2018a) estruturam a análise baseada em inteligência de ameaças;
3. Nemec Zlatolas, Welzer e Lhotska (2024) evidenciam as barreiras comportamentais à adoção de práticas protetivas;
4. Chen, Henry e Jiang (2023) e Chen e Wang (2024) demonstram a insuficiência da comunicação técnica sem mediação educacional;

5. o CVSS (FIRST, 2019) oferece um modelo de pontuação consolidado, mas voltado à severidade de vulnerabilidades.

Permanece pouco explorada, contudo, a integração desses eixos em uma solução voltada ao usuário comum brasileiro que correlacione exposições já documentadas, quantifique sua criticidade por um modelo transparente e explique prioridades de mitigação em linguagem acessível, sob os princípios da LGPD. Essa é a lacuna que o SafeID se propõe a preencher, articulando detecção pós-incidente, *risk scoring* explicável e apoio cognitivo à decisão em uma única plataforma. Os objetivos da Seção 1.2 derivam diretamente dessa lacuna.

Dessa síntese decorrem as necessidades que orientam a solução:

- **N1 – Conhecer as exposições:** permitir que o titular verifique, de forma simples, se seus identificadores constam em incidentes documentados;
- **N2 – Compreender a gravidade:** traduzir a exposição em uma medida de risco única, explicável e visualmente interpretável;
- **N3 – Saber como agir:** indicar contramedidas priorizadas e compatíveis com o tipo de dado comprometido;
- **N4 – Acompanhar a evolução:** permitir o monitoramento contínuo e o registro histórico da exposição para usuários que assim desejarem;
- **N5 – Preservar a privacidade:** realizar todo o processo com minimização de dados e em alinhamento com a LGPD, evitando que a própria ferramenta se torne nova fonte de exposição.

2.4 Regras de Negócio

As regras de negócio a seguir definem as restrições e o funcionamento lógico do SafeID, assegurando que o sistema opere em conformidade com as diretrizes de cibersegurança e proteção de dados. Elas são enunciadas em nível conceitual, independentemente da implementação.

- **RN01 – Identificador Único de Consulta:** o sistema deve aceitar como identificador primário para consulta o endereço de e-mail informado no cadastro, cuja titularidade é comprovada por um link de verificação antes de qualquer consulta. Em versões futuras, o sistema poderá ser expandido para outros identificadores, desde que validados por máscaras específicas.

- **RN02 – Cálculo do *score* de risco:** é o peso da classe de dado ; é o conjunto de classes expostas no incidente ; é o fator temporal. Para cada incidente , . O valor bruto é . O resultado normalizado é , em escala contínua , com .
- **RN03 – Fator temporal:** é a idade do incidente , expressa em meses. Define-se para ; para ; e para .
- **RN04 – Limitação de Requisições (Rate Limiting):** para evitar abusos e ataques de força bruta, a API limita as requisições por endereço IP, com limite geral de 100 requisições a cada 15 minutos e limites específicos para rotas sensíveis (login: 5 por minuto; cadastro e reenvio do link de verificação: 5 a cada 15 minutos). Ao exceder o limite, o usuário recebe a resposta HTTP 429 com a mensagem “Muitas requisições. Aguarde alguns minutos e tente novamente.”. As consultas à HIBP obedecem, adicionalmente, ao limite de requisições por minuto do plano contratado, aplicado de forma global a todas as instâncias (Seção 4.6.4).
- **RN05 – Anonimização de Resultados Sensíveis:** o sistema nunca deverá exibir a senha vazada em texto claro. Caso o metadado do vazamento inclua a senha, o SafeID deve apenas informar a existência da exposição e orientar a troca imediata, preservando a segurança do usuário mesmo dentro da plataforma.
- **RN06 – Personalização via IA:** o plano de ação gerado pela Inteligência Artificial deve ser obrigatoriamente contextualizado ao tipo de vazamento encontrado. Se o vazamento for de “Dados de Cartão”, a recomendação prioritária deve ser o bloqueio bancário; se for de “Senha”, a troca de credenciais e a ativação de MFA. Se o modelo de linguagem não responder no tempo-limite, o sistema apresenta uma resposta padrão determinística, construída a partir das mesmas classes de dados.
- **RN07 – Histórico e Retenção:** o histórico de consultas é mantido apenas para usuários cadastrados com e-mail verificado, com os dados de incidentes cifrados; contas não verificadas em 7 dias são eliminadas automaticamente, e a exclusão da conta pelo titular remove o histórico em cascata e limpa o cache (Seção 4.7.4). Não há consulta anônima: a exigência de conta verificada impede que o SafeID seja usado para consultar e-mails de terceiros, em observância ao princípio da necessidade da LGPD e como prevenção à enumeração de endereços.
- **RN08 – Faixas de Classificação do Risco:** o Risk Score final, arredondado ao inteiro mais próximo, é classificado em quatro faixas fechadas: 0 a 25, Baixo (LOW); 26 a 50, Médio (MODERATE); 51 a 75, Alto (HIGH); e 76 a 100, Crítico (CRITICAL). A classificação é sempre aplicada ao mesmo valor inteiro que é persistido e exibido, de

modo que o valor e a faixa nunca divergem; os subscores de cada incidente, com duas casas decimais, permanecem disponíveis para auditoria (Seção 2.4.1.3).

2.4.1 Modelo de Avaliação de Risco e Ponderação de Exposição (Risk Score)

A quantificação do risco de segurança e exposição digital no SafeID fundamenta-se na necessidade de converter metadados heterogêneos de incidentes cibernéticos (*Open Source Intelligence* – OSINT) em uma métrica unificada, intuitiva e acionável para o cidadão leigo. A ausência de padronização na comunicação de vazamentos frequentemente gera passividade ou alarmismo desproporcional; assim, o modelo matemático do SafeID, que detalha as regras RN02, RN03 e RN08, estabelece um rigor técnico e normativo para a avaliação de danos potenciais.

2.4.1.1 Fundamentação teórica e normativa

A atribuição dos pesos de criticidade e a arquitetura da equação matemática baseiam-se em quatro pilares do ecossistema de cibersegurança e proteção de dados:

1. **Diretrizes de Identidade Digital e Autenticação (NIST SP 800-63B-4):** o *National Institute of Standards and Technology, na revisão 4 das diretrizes de identidade digital, publicada em 2025 em substituição à versão de 2017* (NIST, 2025), estabelece que segredos de autenticação (como senhas em texto claro ou *hashes* de baixa entropia) possuem o maior potencial de comprometimento de contas por meio de ataques automatizados de *Credential Stuffing* e *Password Spraying*. Portanto, essas categorias recebem o peso máximo de criticidade. No plano da governança, o NIST Cybersecurity Framework 2.0 (NIST, 2024) situa a avaliação de risco e a resposta a incidentes como funções contínuas, que o SafeID apoia do lado do titular.
2. **Métricas de Impacto de Vulnerabilidades (CVSS v3.1 / v4.0):** inspirando-se no *Common Vulnerability Scoring System* (FIRST, 2019; FIRST, 2023) e na OWASP Risk Rating Methodology (OWASP, [s. d.]), a taxonomia do SafeID avalia os vetores de confidencialidade e o potencial de exploração secundária. Atributos que permitem fraudes financeiras diretas ou usurpação de identidade legal possuem peso superior a metadados puramente técnicos (como o endereço IP).
3. **Lei Geral de Proteção de Dados (LGPD – Lei nº 13.709/2018):** com base no art. 5º, incisos I e II, e no art. 52 da Lei (BRASIL, 2018), a classificação diferencia dados pessoais comuns de dados sensíveis ou cadastrais de alto impacto, como CPF, dados

financeiros e dados biométricos, alinhando a ponderação à severidade do dano potencial ao titular.

4. **Literatura científica em *Threat Intelligence*:** conforme demonstrado por Conti, Dargahi e Dehghantanha (2018a), dados exfiltrados alimentam mercados secundários de cibercrime. Adicionalmente, Nemec Zlatolas, Welzer e Lhotska (2024) observam que a reutilização de senhas torna os vazamentos recentes exponencialmente mais perigosos, justificando a incorporação de um fator de atenuação temporal () na modelagem.

2.4.1.2 Taxonomia e tabela de pesos por categoria de dado (*data classes*)

Os metadados retornados pelas fontes de inteligência de ameaças e de *Open Source Intelligence* (OSINT – Inteligência de Fontes Abertas) são mapeados e categorizados em quatro níveis de criticidade para o cálculo do risco, conforme a Tabela 1. As classes são identificadas pelos nomes exatos que a API do HIBP devolve no campo DataClasses, os mesmos usados pelo Motor de Risco; classes não listadas recebem o peso padrão 3. A última coluna relaciona cada nível ao impacto correspondente na OWASP Risk Rating Methodology (escala de 0 a 9, em que valores de 6 a 9 indicam impacto alto) e ao impacto de confidencialidade no sistema vulnerável (VC) do CVSS 4.0, tornando a ordenação dos pesos auditável frente a referências externas.

Tabela 1 – Categorias e pesos dos metadados obtidos da API

Nível de Criticidade	**Classe de dado (nome no HIBP)**	**Peso (w_i)**	**Justificativa teórica e regulatória**	**Correspondência externa (OWASP RR / CVSS 4.0)**
Nível 4: Altíssima	Dados financeiros (Credit cards, Credit card CVV, Bank account numbers, Financial transactions)	10	Dano patrimonial direto e fraude financeira automatizada (BRASIL, 2018, art. 52).	Perda de confidencialidade de dado crítico e dano financeiro: impacto 9 (alto) / VC:H
Nível 4: Altíssima	Senhas (Passwords); o HIBP não distingue texto claro de hash, e o modelo adota o pior caso	10	Acesso direto às contas por credential stuffing e password spraying (NIST, 2025).	Credencial de acesso exposta: impacto 9 (alto), com exploração trivial / VC:H
Nível 3: Alta	Documentos oficiais (Government issued IDs, Social security numbers, Passport numbers, Driver's licenses) e dados parciais de cartão (Partial credit card data)	8	Personificação ilícita, abertura de contas fraudulentas e falsidade ideológica.	Dado crítico de identificação: impacto 7 (alto) / VC:H
Nível 3: Alta	Tokens, biometria e segredos de recuperação (Auth tokens, Biometric data, Security questions and answers)	8	Quebra do segundo fator e dos mecanismos de recuperação de conta; dado biométrico é sensível e irrevogável (BRASIL, 2018, art. 5º, II).	Dado crítico de autenticação: impacto 7 (alto) / VC:H
Nível 3: Alta	Dicas de senha (Password hints)	7	Reduzem o espaço de busca da senha e favorecem a adivinhação	Dado crítico parcial: impacto 6 (alto) / VC:L

Nível de Criticidade	Classe de dado (nome no HIBP)	Peso (w_i)	Justificativa teórica e regulatória	Correspondência externa (OWASP RR / CVSS 4.0)
			direcionada.	
Nível 2: Média	Telefone e endereço (Phone numbers, Physical addresses)	5	Engenharia social direcionada, troca fraudulenta de chip (SIM swapping) e phishing por SMS ou mensageiros.	Dado pessoal não crítico em extensão: impacto 5 (médio) / VC:L
Nível 2: Média	Data de nascimento (Dates of birth)	4	Usada em perguntas de validação de segurança e processos cadastrais.	Impacto 4 (médio) / VC:L
Nível 1: Baixa	Nome e nome de usuário (Names, Usernames); peso padrão para classes não mapeadas	3	Informação pública ou semipública; isoladamente, baixo potencial ofensivo.	Dado não crítico mínimo: impacto 3 (médio-baixo) / VC:L
Nível 1: Baixa	E-mail, IP e localização (Email addresses, IP addresses, Geographic locations)	2	O e-mail já é o identificador consultado; IP e localização mudam com frequência.	Impacto 2 (baixo) / VC:L
Nível 1: Baixa	Telemetria (User agent details, Browser user agent details, Device information)	1	Telemetria técnica sem vínculo direto com credenciais.	Impacto 1 (baixo) / VC:N

Fonte: elaborado pelos autores (2026), com base em NIST (2025), FIRST (2023), OWASP ([s. d.]) e Brasil (2018); pesos conforme o Motor de Risco do repositório (server/src/core/engines/risk.engine.ts).

A correspondência externa não foi usada para gerar os pesos, que foram definidos pela equipe a partir das referências da Seção 2.4.1.1; ela serve para verificar, a posteriori, que a ordenação do modelo é coerente com escalas consolidadas: todos os pesos 8 a 10 correspondem a impacto alto na OWASP Risk Rating Methodology e a VC:H no CVSS 4.0, e nenhum dado de impacto baixo recebe peso superior a 3. Os valores numéricos dentro de cada nível continuam sendo escolhas normativas do modelo, explicitadas para auditoria.

Justificativa da hierarquia entre dados biométricos e financeiros. Dados biométricos são irrevogáveis: não podem ser trocados como uma senha nem cancelados como um cartão, o que, em horizonte longo, eleva seu impacto. O peso do SafeID, porém, mede a urgência de exploração e de ação no curto prazo, que é o que o painel comunica. Senhas e dados financeiros permitem exploração imediata e automatizada em larga escala (credential stuffing, compras fraudulentas), e é para eles que existe uma contramedida eficaz e imediata ao alcance do titular (trocar a senha, bloquear o cartão). Já o reaproveitamento de um registro biométrico vazado depende, em geral, de capturar ou contornar o sensor do serviço que o utiliza, e o titular não dispõe de ação corretiva equivalente. Por isso, a biometria permanece no Nível 3 (peso 8), e a irrevogabilidade deve ser tratada na comunicação do plano de ação (RN06), explicando que o dado não pode ser substituído e orientando o monitoramento de usos indevidos. As instruções atuais do modelo de linguagem ainda não trazem orientação específica para biometria; sua inclusão e a reavaliação da hierarquia com dados de uso real estão registradas nos trabalhos futuros (Seção 6.3).

2.4.1.3 Modelagem matemática do algoritmo

Seja o número total de incidentes de segurança (*data breaches*) nos quais o identificador do usuário foi localizado. Para cada incidente pertencente ao conjunto , define-se como o conjunto de categorias de dados comprometidos.

1. Fator de atenuação temporal . Dado o tempo decorrido , em meses, desde a data de publicação ou ocorrência do incidente , o coeficiente de relevância temporal é definido pela função degrau:

2. Pontuação do incidente . A pontuação de severidade do incidente corresponde à soma dos pesos das categorias expostas, ajustada pelo fator temporal, em que representa o peso atribuído à classe de dado :

3. Pontuação bruta acumulada . O somatório do impacto de todos os vazamentos associados ao titular é calculado por:

4. Normalização e obtenção do *Risk Score* final. Para mapear o resultado em um índice contínuo de 0 a 100, utiliza-se uma função de saturação linear com limite :

A escolha de resulta de uma análise de sensibilidade determinística com . Enquanto , cada ponto bruto corresponde a 2 pontos na escala final: uma contribuição crítica recente de peso 10 gera escore 20, e duas contribuições equivalentes geram 40; a saturação em 100 ocorre apenas quando a soma ajustada alcança 50. Limites 30 e 40 saturariam o modelo após, respectivamente, três e quatro contribuições de peso 10, ao passo que 50 preserva discriminação por uma contribuição crítica adicional. Limites 60 e 75, por outro lado, comprimem os níveis intermediários. Trata-se de análise baseada em cenários do próprio modelo, não de evidência empírica de produção.

5. Faixas de mapeamento do termômetro de risco (RN08). O valor contínuo é arredondado ao inteiro mais próximo antes da classificação; a faixa é, portanto, sempre a do valor exibido e persistido, e os subscores de cada incidente, com duas casas decimais, ficam registrados para auditoria.

- De 0 a 25: nível de risco **Baixo** (cor verde na interface) — ação recomendada: manutenção de boas práticas e monitoramento passivo.

- De 26 a 50: nível de risco **Médio** (cor amarela na interface) — ação recomendada: atualização preventiva de senhas antigas e ativação de *Multi-Factor Authentication* (MFA – Autenticação de Múltiplos Fatores).
- De 51 a 75: nível de risco **Alto** (cor laranja na interface) — ação recomendada: troca imediata de credenciais e verificação de contas reutilizadas.
- De 76 a 100: nível de risco **Crítico** (cor vermelha na interface) — ação recomendada pela Inteligência Artificial (requisito funcional RF06): bloqueio prévio de cartões, auditoria de extratos e revogação imediata de acessos em aplicações vinculadas.

2.4.1.4 Exemplo numérico prático e auditável

Para fins de validação técnica e verificação de auditoria, considere-se a simulação de consulta de um e-mail cadastrado localizado em três incidentes de segurança (*data breaches*):

1. **Incidente 1 — vazamento no comércio eletrônico "LojaX":** ocorrido há 6 meses (;); dados expostos: senha (Passwords;), endereço de *Internet Protocol* (IP;) e nome completo ().
2. **Incidente 2 — vazamento no fórum "ServicoY":** ocorrido há 36 meses (;); dados expostos: *dicas* de senha (Password hints;) e nome de usuário (*username*;).
3. **Incidente 3 — vazamento no aplicativo "PlataformaZ":** ocorrido há 12 meses (;); dados expostos: cartão de crédito (Credit cards;) e número de telefone ().

A execução do cálculo, passo a passo, é a seguinte:

- Incidente 1: ; .
- Incidente 2: ; .
- Incidente 3: ; .
- Pontuação bruta: pontos.
- Normalização: .

O diagnóstico resultante apresenta *score* de 74/100, classificado como risco **alto** e representado em cor laranja no painel (*dashboard*). Embora o vazamento no fórum antigo tenha sofrido atenuação temporal, reduzindo seu peso de 10 para 7,0 pontos, a combinação de uma senha exposta em incidente recente com o comprometimento de dados de cartão eleva o risco individual para o nível alto, demandando alteração imediata de credenciais e bloqueio de cartões.

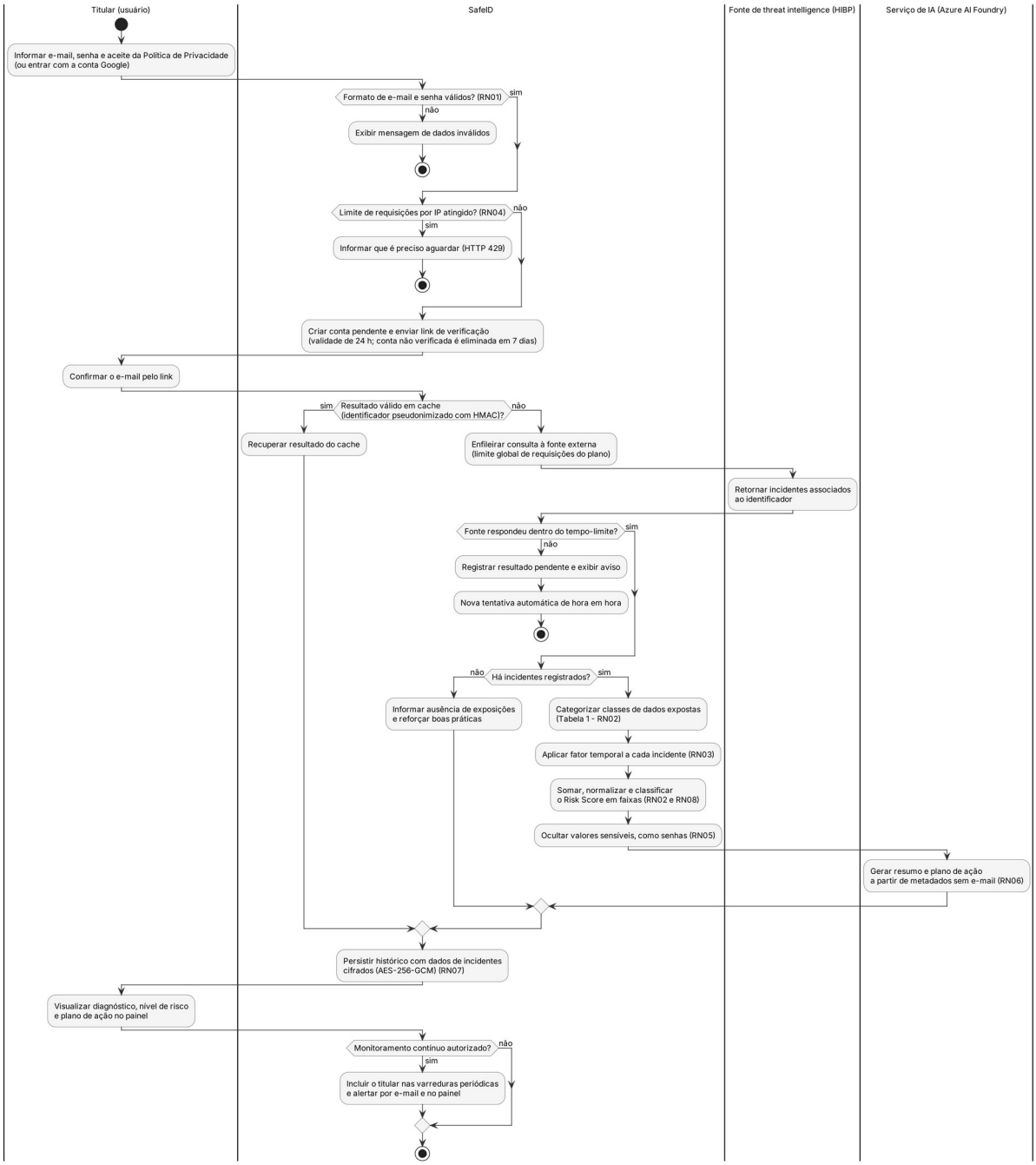
Para manter o exemplo legível, ele omite a classe Email addresses (peso 2), que o HIBP informa em praticamente todos os incidentes, já que a busca é feita pelo próprio e-mail. Incluída nos três incidentes, ela acrescentaria 5,4 pontos brutos (2,0 + 1,4 + 2,0), elevando a

soma para 42,4 e o score para 85, na faixa Crítico. O caso mostra por que o modelo também pondera classes de baixo peso: a recorrência de exposições, ainda que de dados simples, desloca o titular de faixa.

2.4.2 Fluxo de Atividades das Regras de Negócio

A Figura 1 representa, por meio de um diagrama de atividades da *Unified Modeling Language* (UML) (OMG, 2017), o processo de consulta e diagnóstico de exposição sob a perspectiva do domínio, do cadastro à exibição do plano de ação, evidenciando os pontos em que cada regra de negócio é aplicada: a validação do identificador (RN01), a limitação de requisições (RN04), a ponderação e a atenuação temporal do risco (RN02 e RN03), a classificação em faixas (RN08), a ocultação de dados sensíveis (RN05), a contextualização das orientações (RN06) e a política de retenção do histórico (RN07). O diagrama também representa o caminho de contingência: quando a fonte externa não responde, o resultado fica pendente e é refeito automaticamente.

Figura 1 – Diagrama de atividades do processo de consulta e diagnóstico



Fonte: elaborado pelos autores (2026).

3 ANÁLISE DO SISTEMA

Este capítulo transforma a compreensão do contexto e das necessidades em uma visão estruturada do sistema a ser desenvolvido. São definidos o que o SafeID deve fazer, quem interage com ele, quais conceitos compõem seu domínio e quais requisitos e restrições devem ser considerados, sem ainda detalhar sua implementação.

3.1 Visão Geral do Sistema

O SafeID é uma aplicação web de apoio à decisão em segurança cibernética preventiva, delimitada à fase posterior aos incidentes de vazamento. Seu propósito é receber um identificador do titular (o endereço de e-mail, conforme RN01), verificar sua presença em incidentes já catalogados por fontes externas de inteligência de ameaças, quantificar a gravidade da exposição por meio de um índice explicável e comunicar o resultado em linguagem acessível, acompanhado de orientações de mitigação.

Em visão macroscópica, a solução encadeia três funções: (i) o **mapeamento automatizado** das exposições, que consulta a fonte externa (HIBP) e categoriza os metadados retornados; (ii) o **cálculo do *Risk Score***, que aplica o modelo determinístico definido na Seção 2.4.1 e classifica o resultado em faixas; e (iii) a **assistência cognitiva**, implementada na PI-2, que utiliza um modelo de linguagem para converter o diagnóstico em um plano de ação. Em torno dessas funções, o sistema oferece aos usuários cadastrados autenticação com verificação de e-mail, histórico de consultas, monitoramento contínuo com alertas, exportação do plano de ação, recuperação e troca de senha, cópia dos dados da conta e exclusão da conta; a gestão de múltiplos identificadores não foi implementada. A fronteira do sistema exclui ações em contas de terceiros, monitoramento de rede local e oferta de proteção financeira, conforme o escopo definido na Seção 1.4.

3.2 Atores Envolvidos

Os atores que interagem diretamente com o SafeID e seus respectivos papéis são caracterizados no Quadro 2. Foram identificados atores humanos, derivados dos perfis de público-alvo (Seção 2.2), e atores sistêmicos, correspondentes aos serviços externos e ao mecanismo temporal que dispara rotinas automáticas.

Quadro 2 – Atores do sistema e seus papéis

Ator	Tipo	Papel no sistema
Usuário cadastrado	Humano (primário)	Titular autenticado com e-mail verificado; recebe o diagnóstico e o plano de ação, mantém histórico, recebe alertas, exporta o plano,

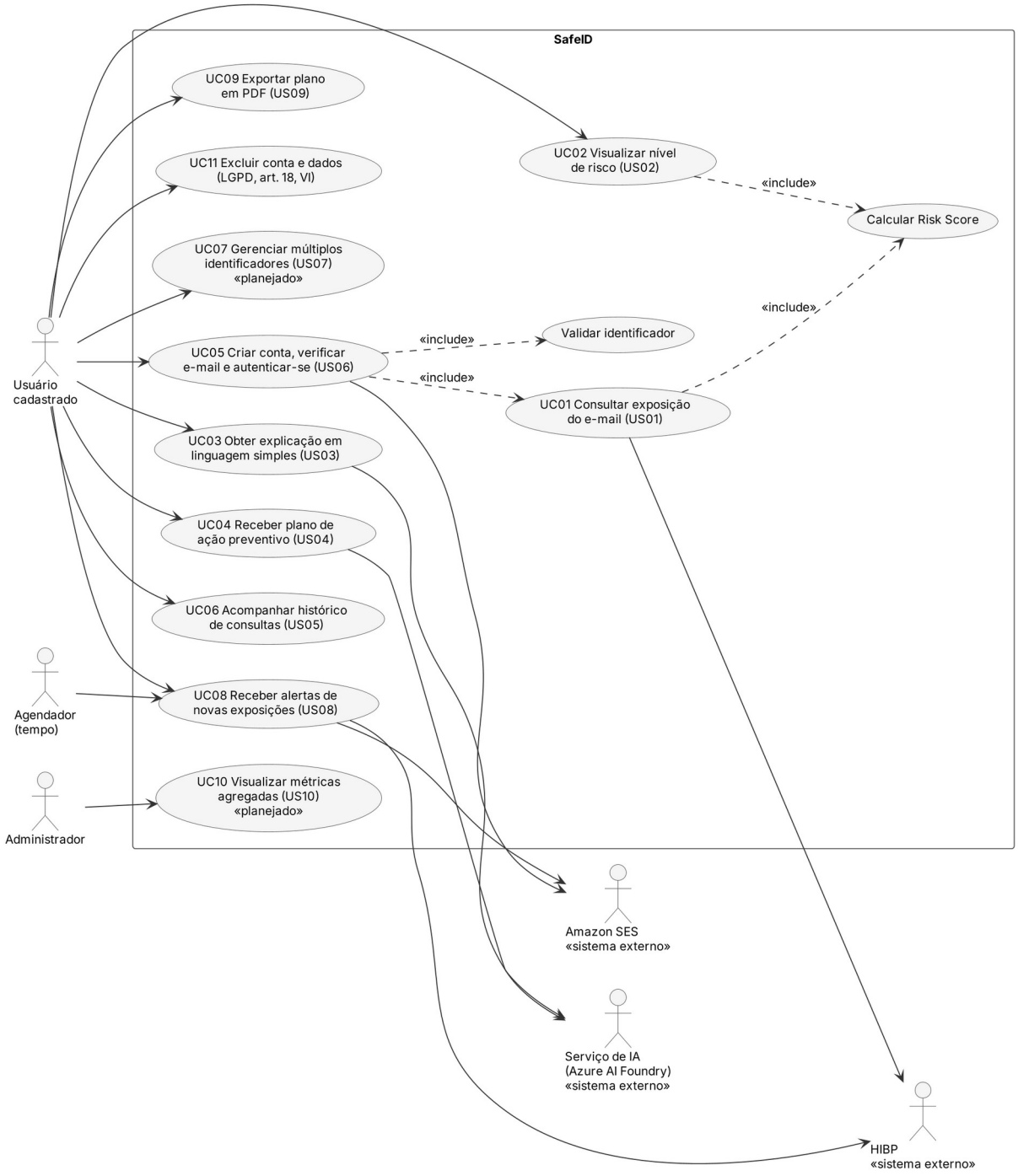
Ator	Tipo	Papel no sistema
		recupera ou troca a senha, baixa uma cópia dos próprios dados e pode excluir a conta. A gestão de múltiplos identificadores não foi implementada.
Administrador	Humano (primário)	Consulta métricas agregadas e anonimizadas de uso para apoiar a operação e a melhoria do serviço (não implementado; hoje restrito à rota de diagnóstico dos jobs, protegida por token).
HIBP	Sistema externo (secundário)	Fornece os registros de incidentes associados ao identificador consultado.
Serviço de IA (Azure AI Foundry)	Sistema externo (secundário)	Gera explicações e planos de ação em linguagem natural a partir de metadados sem identificador pessoal.
Amazon SES	Sistema externo (secundário)	Envia os e-mails de verificação de conta e de alerta de novas exposições.
Agendador (tempo)	Sistema (primário)	Dispara periodicamente as varreduras silenciosas que alimentam os alertas de novas exposições.

Fonte: elaborado pelos autores (2026).

3.3 Casos de Uso

As funcionalidades do SafeID foram levantadas por meio de histórias de usuário, que representam as necessidades dos diferentes perfis que interagem com o sistema e estabelecem, por meio de critérios de aceite, a correspondência entre as expectativas dos usuários e os comportamentos esperados. A Figura 2 consolida essas funcionalidades em um diagrama de casos de uso, no qual cada caso referencia a história de usuário correspondente; os casos marcados com o estereótipo «planejado» não foram implementados, e os demais estão em produção. O caso UC11, relativo à exclusão da conta e dos dados, decorre do direito do titular previsto no art. 18, VI, da LGPD, tratado na Seção 4.7.4. Os critérios de aceite foram escritos de forma verificável, com valores extraídos da implementação, para permitir a rastreabilidade com os testes do Capítulo 5.

Figura 2 – Diagrama de casos de uso do SafeID



Fonte: elaborado pelos autores (2026).

A seguir, descrevem-se as histórias de usuário e seus critérios de aceite, que detalham os fluxos e comportamentos esperados de cada caso de uso.

US01 – Consulta de Vulnerabilidade (PI-1, automatizada na PI-2; UC01). Como cidadão preocupado com minha segurança, eu quero que o e-mail da minha conta seja

verificado pelo sistema para saber se minhas informações foram expostas em vazamentos de dados conhecidos. Critérios de aceite:

- e-mail fora do formato RFC ou senha com menos de 8 caracteres é recusado com HTTP 400, antes de qualquer consulta externa (class-validator);
- a consulta é disparada automaticamente na confirmação do e-mail e enfileirada no BullMQ; o resultado é aguardado por até 20 s, e o painel exibe estado de carregamento nesse intervalo;
- se a HIBP não responder em 10 s ou o Circuit Breaker estiver aberto, o painel exibe a mensagem “Não conseguimos verificar o seu email agora”, marca o score como PENDENTE e a consulta é refeita automaticamente de hora em hora.

US02 – Entendimento do Nível de Risco (PI-1; UC02). Como usuário leigo em tecnologia, eu quero visualizar um índice de risco (*score*) de fácil compreensão para que eu saiba a urgência de tomar medidas de proteção. Critérios de aceite:

- o score de 0 a 100 é exibido em termômetro, com a faixa escrita por extenso (a cor apenas reforça), conforme RN08;
- para o mesmo conjunto de incidentes, o Motor de Risco devolve sempre o mesmo valor; o exemplo da Seção 2.4.1.4 deve resultar em 74 (Alto);
- uma legenda explica o significado dos níveis “Baixo”, “Médio”, “Alto” e “Crítico”.

US03 – Tradução de Metadados via IA (PI-2, implementado; UC03). Como usuário sem conhecimento técnico, eu quero que a inteligência artificial explique em linguagem simples o que significa cada termo técnico do vazamento (por exemplo, “SQL Injection” ou “Combolist”) para que eu entenda o impacto real na minha privacidade. Critérios de aceite:

- o payload enviado ao modelo contém apenas nome, data e classes de dados dos incidentes, além do score e da classificação; nenhum campo contém o e-mail do usuário (verificado em teste unitário do AIEngine);
- o resumo é escrito em português, sem jargão, com até 450 caracteres (limite instruído ao modelo);
- se o modelo não responder em 10 s ou devolver JSON inválido, o painel exibe a resposta padrão determinística, identificada internamente como fallback.

US04 – Plano de Ação Preventivo (PI-2, implementado; UC04). Como vítima de uma exposição de dados, eu quero receber orientações passo a passo do que fazer para mitigar o dano, para que eu possa proteger minhas contas de forma assertiva. Critérios de aceite:

- o plano contém de 4 a 6 passos, ordenados do mais urgente ao menos urgente, cada um iniciado por verbo no imperativo, e um nível de urgência (alta, média ou baixa);
- quando há senha exposta, o plano inclui a troca da senha e a ativação da verificação em duas etapas; quando há dado de cartão, o aviso ao banco (RN06);
- o plano nunca inclui links para clicar nem pede senhas ou códigos, para não reproduzir padrões de phishing.

US05 – Monitoramento de Histórico (PI-2, implementado; UC06). Como usuário cadastrado, eu quero salvar meu histórico de consultas para que eu possa acompanhar se as medidas que tomei reduziram meu *score* de vulnerabilidade ao longo do tempo. Critérios de aceite:

- o histórico só é acessível com token JWT válido (sem token, HTTP 401), e cada usuário vê apenas os próprios registros;
- os dados de incidentes são gravados cifrados com AES-256-GCM (prefixo enc:v1) e decifrados na leitura;
- o painel exibe um gráfico da evolução do score entre as consultas registradas.

US06 – Autenticação e Privacidade (PI-2, implementado; UC05). Como usuário recorrente, eu quero criar uma conta protegida para que minhas consultas e dados de histórico não fiquem expostos a terceiros que utilizem o mesmo dispositivo. Critérios de aceite:

- o cadastro exige aceite da Política de Privacidade e confirmação do e-mail por link de uso único, válido por 24 h; contas não confirmadas em 7 dias são eliminadas;
- senhas têm no mínimo 8 caracteres e são armazenadas com bcrypt (fator de custo 10);
- o token de acesso vale 24 h e o de renovação, 7 dias; o login aceita até 5 tentativas por minuto por IP.

US07 – Gestão de Múltiplos Identificadores (não implementado; UC07). Como responsável pela segurança digital da minha família, eu quero cadastrar múltiplos e-mails no meu painel para monitorar a vulnerabilidade de todos os meus dependentes em um único lugar. Critérios de aceite:

- o usuário deve conseguir adicionar e remover e-mails da sua lista de monitoramento;
- o *dashboard* deve separar os resultados por identificador.

US08 – Notificações de Alerta Ativo (PI-2, implementado; UC08). Como usuário preventivo, eu quero ser notificado via e-mail assim que um novo vazamento contendo meus dados for detectado, para que eu possa agir antes que ocorra uma invasão real. Critérios de aceite:

- o catálogo público da HIBP é sincronizado a cada 6 h e verificado de hora em hora; quando surge um vazamento novo, os usuários monitorados são conferidos;
- cada vazamento novo gera uma única notificação por usuário (chave de deduplicação), no painel e por e-mail, com link para o painel;
- todo e-mail de alerta permite descadastramento em um clique, e o monitoramento pode ser desligado a qualquer momento nas preferências.

US09 – Exportação de Plano de Mitigação (PI-2, implementado; UC09). Como usuário que deseja se organizar *offline*, eu quero exportar o plano de ação gerado pela IA em formato PDF para que eu possa segui-lo como um *checklist* de segurança. Critérios de aceite:

- o documento contém score, classificação, urgência, resumo, passos de mitigação e a lista de vazamentos com a gravidade de cada um;
- o PDF é gerado pelo próprio navegador (“Salvar como PDF”), sem enviar dados a serviços adicionais.

US10 – Administração e Auditoria (não implementado; UC10). Como administrador do sistema, eu quero visualizar métricas agregadas (anonimizadas) de consultas para entender quais são os vazamentos mais comuns e treinar melhor o modelo de IA. Critérios de aceite:

- o painel administrativo deve exibir o volume de consultas por período;
- o sistema deve aplicar pseudonimização (HMAC-SHA256 com *chave* secreta) em informações analíticas, estabelecendo como meta que nenhum e-mail identificável seja exposto nesta visão (*Privacy by Design*).

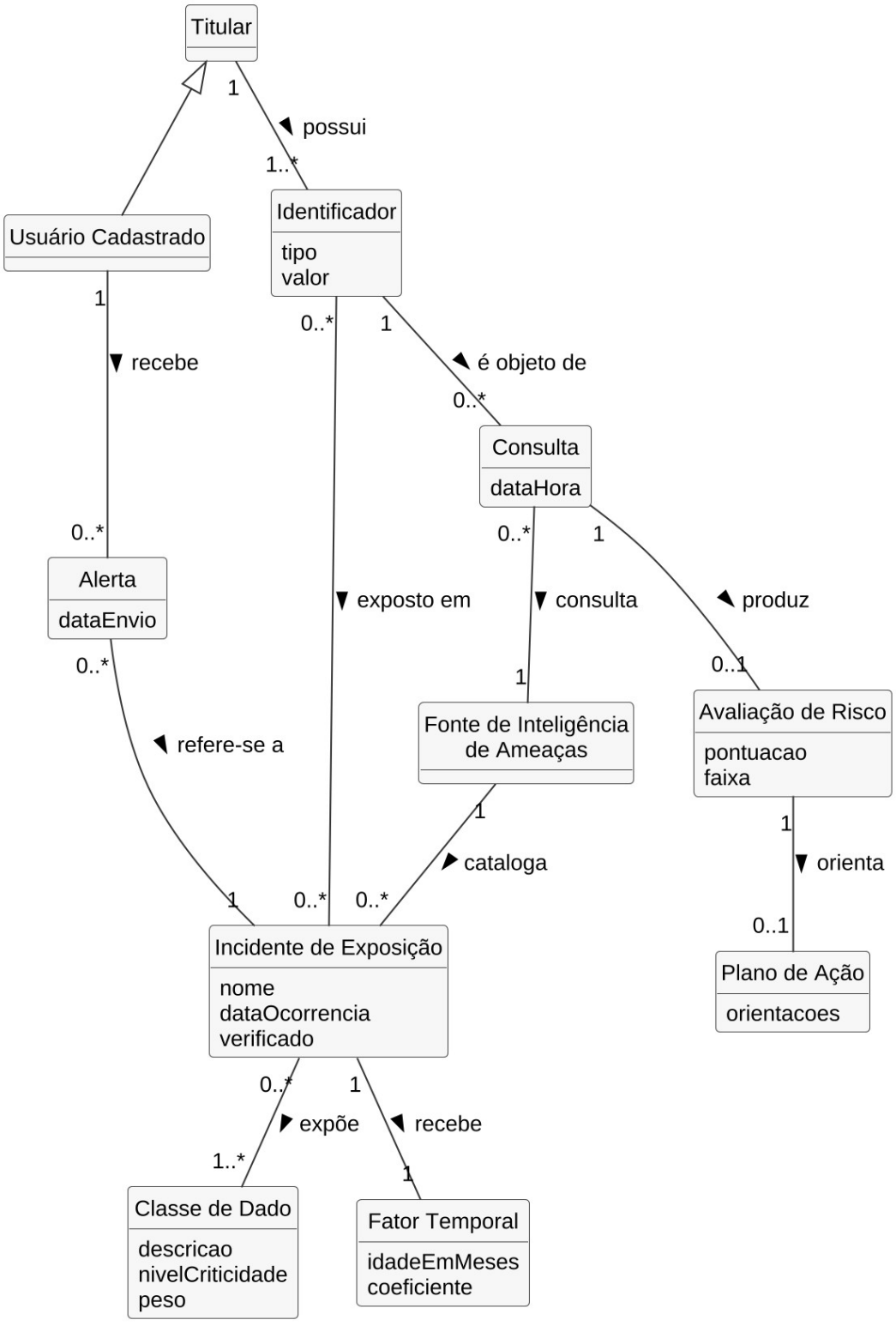
3.4 Modelo Conceitual

O modelo conceitual representa os principais conceitos do domínio do SafeID, suas características e relacionamentos, sem incorporar decisões de implementação, como tipos de dados, chaves ou tecnologias de persistência. A Figura 3 apresenta o diagrama de classes conceitual.

No domínio, um **Titular** possui um ou mais **Identificadores** (na versão atual, apenas o e-mail da conta; a multiplicidade 1..* antecipa a US07). Cada identificador pode ser objeto de diversas **Consultas**, realizadas junto a uma **Fonte de Inteligência de Ameaças**, que cataloga **Incidentes de Exposição**. Um incidente expõe uma ou mais **Classes de Dado**, cada qual com nível de criticidade e peso (Tabela 1), e recebe um **Fator Temporal** conforme sua idade (RN03). Cada consulta produz uma **Avaliação de Risco**, com pontuação e faixa, que

orienta um **Plano de Ação**. O **Usuário Cadastrado**, especialização do titular, pode ainda receber **Alertas** referentes a novos incidentes.

Figura 3 – Diagrama de classes conceitual do domínio



Fonte: elaborado pelos autores (2026).

3.5 Requisitos Funcionais

Os requisitos funcionais descrevem as funções que o software deve executar para atender às necessidades dos usuários e às regras de negócio estabelecidas. Para o SafeID, os requisitos foram divididos entre o que foi entregue no protótipo (PI-1), o que foi implementado na PI-2 e o que não foi implementado. Os requisitos funcionais (RF) estão listados no Quadro 3, que traz, para cada um, um critério objetivo de verificação e a rastreabilidade com as regras de negócio (RN) e os requisitos não funcionais (RNF) relacionados. O antigo RF07 (diagnóstico cognitivo) foi incorporado ao RF06, que passa a ter duas saídas distintas: o diagnóstico, que explica o que aconteceu, e o plano de ação, que diz o que fazer.

Quadro 3 – Requisitos funcionais (RF)

ID	Requisito funcional	Descrição	Critério de verificação	Rastreabilidade	Fase
RF01	Consulta por identificador (OSINT)	Ao confirmar o e-mail (ou no primeiro login com Google), o sistema enfileira no BullMQ a consulta à HIBP para o e-mail da conta.	Resultado ou estado pendente gravado em até 20 s; nenhuma chamada à HIBP fora do worker da fila.	RN01, RN04; RNF02	PI-1 (automatizada na PI-2)
RF02	Categorização de incidentes	Extrai do retorno da HIBP nome, data, classes de dados e verificação de cada incidente, e mantém cópia local do catálogo público.	Classe desconhecida recebe peso padrão 3, sem erro (teste de fuzzing).	RN02; RNF07	PI-1
RF03	Cálculo e exibição do Risk Score	O Motor de Risco aplica pesos, fator temporal e normalização e classifica o resultado em faixas.	Exemplo da Seção 2.4.1.4 resulta em 74 (Alto); limites 25/26, 50/51 e 75/76 classificados conforme RN08.	RN02, RN03, RN08	PI-1
RF04	Tratamento de erros e validação	Valida entradas com class-validator e protege a integração externa com Circuit Breaker, com mensagens amigáveis.	Entrada inválida: HTTP 400; excesso: HTTP 429; HIBP indisponível: estado “Verificação pendente”.	RN01, RN04; RNF04, RNF07	PI-1
RF05	Painel resumido	O front-end exibe score, faixa, incidentes e plano de ação em interface Mobile First.	Painel utilizável de 360 a 1.920 px de largura, sem rolagem horizontal.	RN05, RN08; RNF05	PI-1
RF06	Diagnóstico e plano de ação cognitivos	A partir de metadados sem e-mail, o modelo gera duas saídas: (a) diagnóstico em linguagem simples; (b) plano com 4 a 6 passos e nível de urgência.	Prompt sem e-mail (teste unitário); JSON válido ou resposta padrão em até 10 s.	RN05, RN06; RNF01, RNF03	PI-2
RF07	Verificação de titularidade do e-mail	A conta só é ativada após confirmação por link de uso único; contas não confirmadas são eliminadas.	Link expira em 24 h; token usado ou expirado é rejeitado; limpeza após 7 dias.	RN01, RN07; RNF09	PI-2
RF08	Gestão de contas e histórico	Autenticação por e-mail e senha ou Google, com histórico de consultas persistido e cifrado.	Rotas de dados do usuário exigem JWT (sem token: HTTP 401); dados de incidentes gravados cifrados.	RN07; RNF01, RNF09	PI-2
RF09	Monitoramento ativo e alertas	Jobs agendados sincronizam o catálogo da HIBP, verificam os usuários monitorados e notificam	Uma notificação por vazamento novo e usuário; descadastramento em um	RN07; RNF02, RNF10	PI-2

ID	Requisito funcional	Descrição	Critério de verificação	Rastreabilidade	Fase
		vazamentos novos no painel e por e-mail.	clique.		
RF10	Exportação do plano (PDF)	Gera versão para impressão do painel e do plano, salva em PDF pelo navegador.	Documento contém score, faixa, resumo, passos e vazamentos.	RN05	PI-2
RF11	Exclusão de conta	O titular exclui a própria conta; histórico, vazamentos conhecidos e notificações são removidos em cascata, e o cache é limpo.	Após a exclusão, nenhum registro do usuário permanece no banco nem no cache.	RN07; LGPD, art. 18, VI	PI-2
RF12	Múltiplos identificadores	O usuário cadastra e remove outros e-mails monitorados no mesmo painel (US07).	Resultados separados por identificador.	RN01	Não implementado
RF13	Recuperação e troca de senha	O titular recebe por e-mail um link de uso único para criar nova senha ou troca a senha na conta logada; nos dois casos, as demais sessões são encerradas.	Link expira em 60 min; mesma resposta exista a conta ou não; token de renovação anterior deixa de valer.	RN04; RNF09	PI-2
RF14	Cópia dos dados da conta	O titular baixa, nas configurações da conta, um arquivo JSON com cadastro, análises, vazamentos conhecidos e notificações.	O arquivo traz só os dados da própria conta (teste de autorização entre usuários).	LGPD, art. 18, II e V	PI-2

Fonte: elaborado pelos autores (2026).

3.6 Requisitos Não Funcionais

Enquanto os requisitos funcionais definem *o que* o sistema faz, os não funcionais descrevem *como* ele deve ser, em termos de qualidade, segurança e desempenho. Os requisitos não funcionais (RNF) estão listados no Quadro 4, cada um com uma meta verificável. Em relação à versão anterior, o RNF03 foi reclassificado de Usabilidade para Desempenho, por tratar de tempo de resposta, e foram incluídos requisitos específicos de autenticação e autorização (RNF09) e de disponibilidade (RNF10), este último em resposta ao risco de ponto único de falha discutido na Seção 4.6.3.

Quadro 4 – Requisitos não funcionais (RNF)

ID	Requisito não funcional	Categoria	Descrição	Meta verificável
RNF01	Privacidade por Design (LGPD)	Segurança	Fora do cadastro do usuário, o e-mail não é armazenado em texto claro: chaves de cache e histórico usam HMAC-SHA256 com chave secreta; dados de incidentes são cifrados com AES-256-GCM na aplicação.	Nenhum e-mail em texto claro em chaves do Redis, no histórico ou nos logs.
RNF02	Desempenho de consulta	Desempenho	Resultados recentes são servidos pelo cache Redis; consultas novas passam pela fila, respeitando o limite da HIBP.	P95 < 50 ms com cache; consulta nova concluída ou marcada como pendente em até 20 s.
RNF03	Tempo de resposta do motor cognitivo	Desempenho	O modelo de linguagem é acessado pelo SDK da OpenAI (versão 6.41) no Azure AI Foundry, com tempo-limite e resposta padrão.	Plano entregue, ou substituído pela resposta padrão, em até 10 s; meta de P95 de até 5 s, a medir em produção.
RNF04	Resiliência e tolerância a falhas	Confiabilidade	Circuit Breaker (Opossum) na integração com a HIBP, especificado na Seção 4.7.2.	Abre com taxa de erro ≥ 50% na janela de 10 s; semiabre

ID	Requisito não funcional	Categoria	Descrição	Meta verificável
				após 30 s.
RNF05	Compatibilidade e responsividade	Portabilidade	Interface Mobile First, compatível com navegadores atuais de desktop e dispositivos móveis.	Uso completo de 360 a 1.920 px nas versões atuais de Chrome, Firefox, Safari e Edge.
RNF06	Segurança de tráfego e API	Segurança	HTTPS em todo o tráfego, com TLS terminado no ALB e no CloudFront; Helmet aplica cabeçalhos de segurança, complementado por validação e codificação de saída.	HTTP redirecionado para HTTPS (301); cabeçalhos do Helmet em todas as respostas da API.
RNF07	Integridade e sanitização de dados	Segurança	Toda entrada passa por DTOs com class-validator, com rejeição de campos não declarados.	Campo não declarado ou e-mail fora do formato RFC: HTTP 400.
RNF08	Manutenibilidade e qualidade (DevOps)	Manutenibilidade	Testes automatizados (Jest no back-end, Vitest no front-end), CI no GitHub Actions e empacotamento em contêiner Docker multi-stage.	Cobertura unitária ≥ 80% de statements, branches e lines e ≥ 85% de functions, bloqueante no CI.
RNF09	Autenticação e autorização	Segurança	Senhas com bcrypt; conta ativa só após verificar o e-mail; tokens de acesso e de renovação com segredos distintos; rotas de dados protegidas por guard JWT; limite de tentativas de login.	Acesso: 24 h; renovação: 7 dias; login: 5 tentativas por minuto por IP; 100% das rotas de dados do usuário exigem JWT.
RNF10	Disponibilidade	Confiabilidade	Health check do ALB em /api/health (banco e Redis, tempo-limite de 2 s); o ECS recria a task que falha; backups automáticos do RDS e alarmes no CloudWatch.	Meta de 99% de disponibilidade mensal (sem SLA formal); task recriada em até 2 min; backups com retenção de 1 dia.

Fonte: elaborado pelos autores (2026).

3.7 Restrições do Sistema

As limitações impostas ao desenvolvimento e à operação do SafeID, de natureza técnica, institucional, legal e orçamentária, são registradas a seguir.

- **Premissa de disponibilidade externa (técnica):** o sistema depende da disponibilidade das APIs de terceiros (como o *Have I Been Pwned*). Caso o serviço externo esteja indisponível, o SafeID apresentará uma mensagem de indisponibilidade momentânea, não sendo responsabilidade da equipe a manutenção de bases de dados de terceiros.
- **Restrição ética de tratamento de credenciais (técnica e legal):** o sistema não realizará "quebra de *hashes*" ou descriptografia de senhas. O HIBP informa apenas que a classe Passwords foi exposta, sem a senha nem o tipo de hash; o sistema apenas comunica a exposição, cumprindo seu papel informativo e ético.
- **Limite de taxa do provedor (técnica):** a API HIBP impõe um limite de requisições por minuto que depende do plano contratado (no plano de entrada, 10 por minuto). O SafeID lê esse limite diariamente na própria API, usa 90% dele como margem de

segurança (no plano de entrada, uma chamada a cada 6,7 s) e o aplica de forma global às filas, o que impede chamadas síncronas diretas a partir do front-end.

- **Restrição contratual e orçamentária:** a API HIBP é paga e o acesso à sua base completa exige assinatura; a produção usa uma assinatura comercial, enquanto a suíte automatizada usa respostas simuladas e os e-mails de teste do provedor. Do mesmo modo, o uso da LLM introduz custo variável por volume de requisições, o que impõe limites de consumo e o uso de *cache*.
- **Restrição legal:** todo tratamento de dados pessoais deve observar a LGPD (BRASIL, 2018), em especial os princípios de necessidade, transparência, segurança e prevenção, e os direitos do titular, como a eliminação dos dados tratados com consentimento (art. 18, VI).
- **Restrição de escopo do identificador:** nesta etapa, o e-mail é o único identificador aceito para consulta (RN01).
- **Restrições institucionais e de cronograma:** o desenvolvimento está vinculado ao calendário acadêmico do IFSP, dividido entre a PI-1 (março a junho de 2026) e a PI-2 (agosto a novembro de 2026, com a apresentação final em 13/11), e à capacidade da equipe, composta por quatro integrantes com dedicação estimada de 10 a 12 horas semanais cada (Apêndice A).

4 PROJETO DO SISTEMA

Este capítulo define como a solução é estruturada para atender aos requisitos identificados na análise. São apresentados a arquitetura, o projeto dos componentes, o modelo de dados, a interface do usuário, as interações entre os elementos do sistema, as tecnologias adotadas e os mecanismos de segurança e controle de acesso.

4.1 Arquitetura do Sistema

O SafeID adota uma arquitetura de Monólito Modular (*Modular Monolith*). Diferentemente de uma arquitetura de microsserviços puros — que introduziria complexidade desnecessária de orquestração e rede (*over-engineering*) para o escopo deste MVP —, o monólito modular mantém a aplicação em uma base de código unificada por meio do *framework* NestJS, mas com limites de domínio estritamente separados. O desempenho mensurável e o isolamento de falhas não decorrem da fragmentação física da aplicação, mas do processamento assíncrono: a consulta à API OSINT externa, sujeita a limite de requisições, é desacoplada da requisição do usuário e enviada para workers de processamento via filas (Redis e BullMQ), e a inferência de Inteligência Artificial é chamada de forma não bloqueante, com tempo-limite e resposta padrão. Essa abordagem entrega a capacidade técnica de processamento necessária, mantendo a simplicidade operacional e de implantação de um monólito.

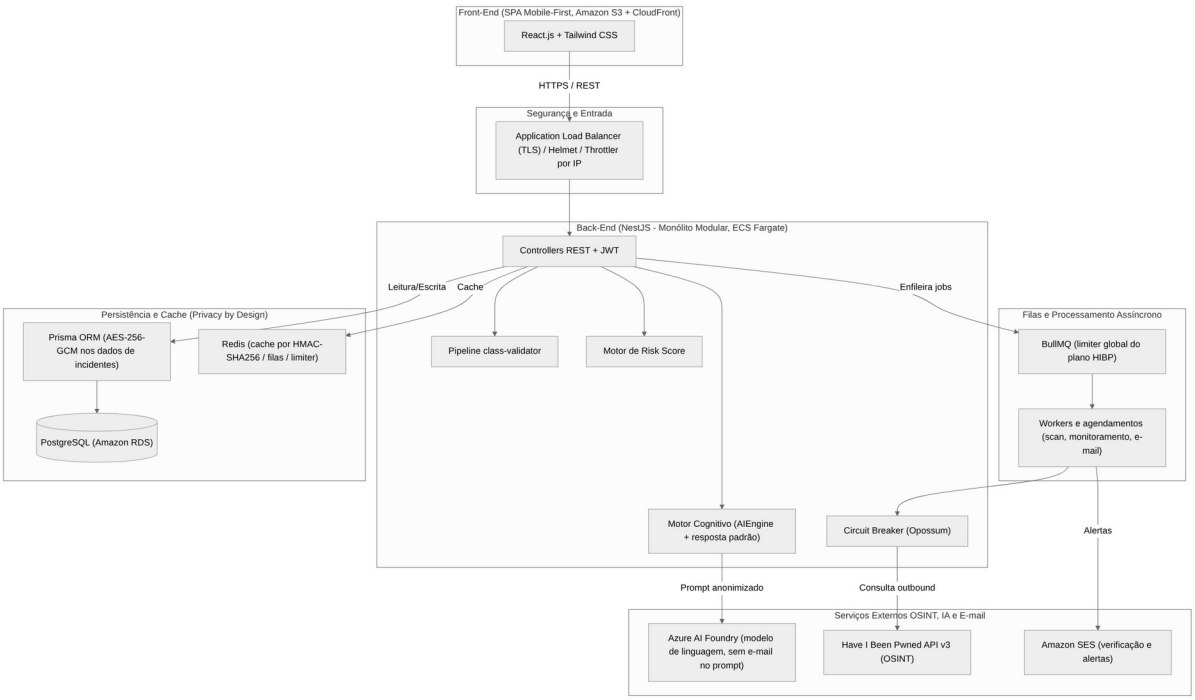
- ***Front-end* (React.js e Tailwind CSS):** camada de apresentação focada em prover uma experiência de usuário (UX) consistente, intuitiva e acessível para o público B2C. Utiliza a abordagem *Mobile First* para garantir responsividade entre dispositivos móveis e *desktops*. O Tailwind CSS é empregado para assegurar agilidade no desenvolvimento e consistência da identidade visual da marca. Toda a comunicação com o servidor ocorre de forma estritamente assíncrona, por meio de requisições HTTPS.
- ***Back-end* e serviços externos (Node.js, TypeScript, NestJS, HIBP, Azure AI Foundry e Amazon SES):** o *back-end* concentra o Motor de Risco e organiza as regras por módulos. DTOs e *class-validator* validam entradas; o Helmet aplica políticas de cabeçalhos que reduzem exposição técnica; Redis e BullMQ cadenciam tarefas em segundo plano; e o *Circuit Breaker*, implementado com Opossum, limita o efeito de falhas da HIBP (especificação completa na Seção 4.7.2). As integrações com

a HIBP e com o modelo de linguagem são mantidas em classes separadas para favorecer testes e substituição de provedores.

- **Persistência e *cache* (PostgreSQL, Prisma ORM e Redis):** o Redis guarda o cache de resultados, as filas e o limitador de requisições à HIBP; o PostgreSQL guarda contas, histórico de consultas, vazamentos já conhecidos de cada usuário e notificações. Os controles de pseudonimização (HMAC-SHA256) e de criptografia (AES-256-GCM) são especificados uma única vez, na Seção 4.7.1, à qual as demais seções remetem.

A Figura 4 detalha essa arquitetura em camadas: apresentação, segurança de entrada, processamento no *back-end*, filas, persistência e *cache*, e serviços externos.

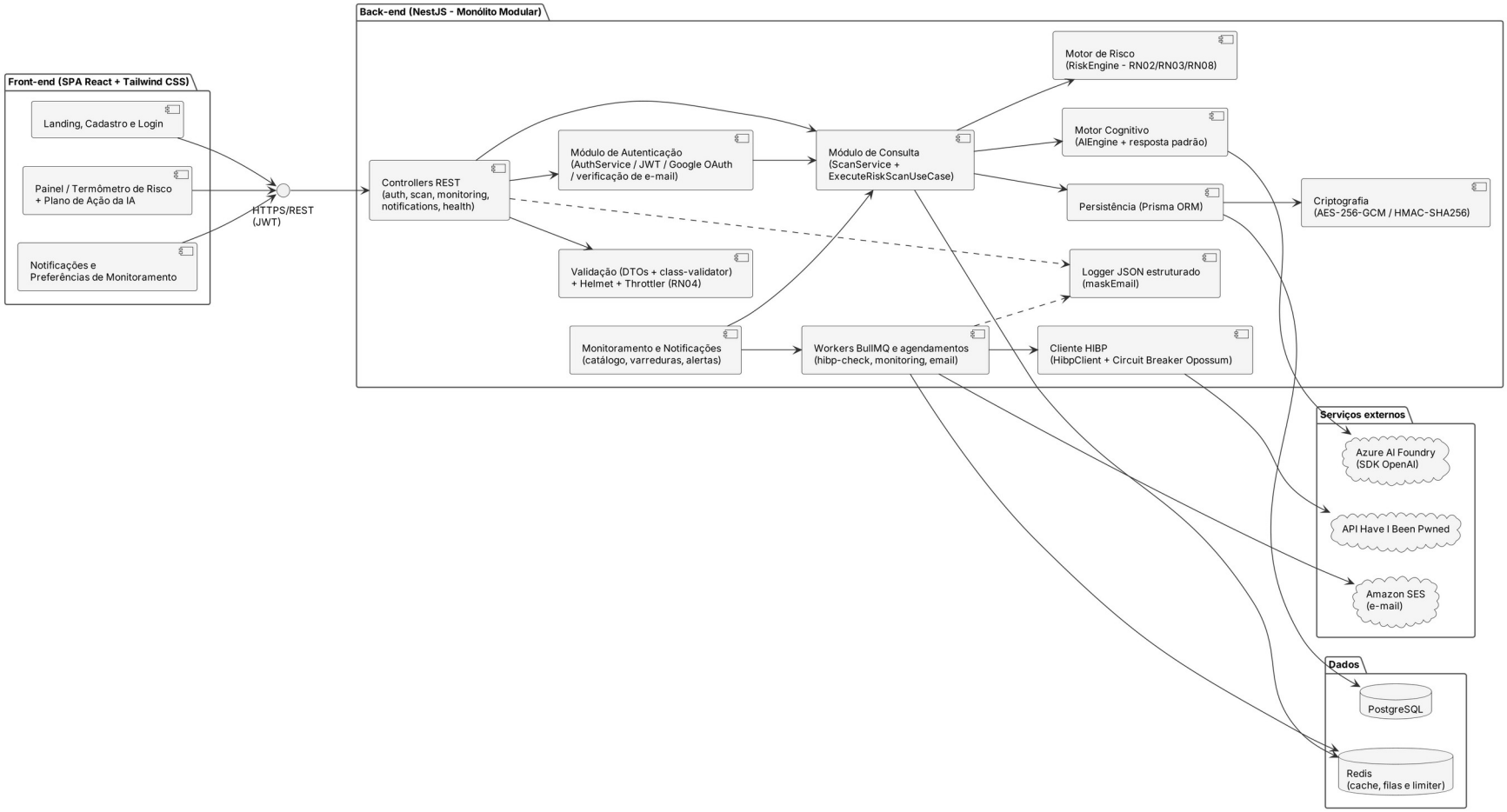
Figura 4 – Diagrama de arquitetura do SafeID



Fonte: elaborado pelos autores (2026).

Em notação UML, a mesma organização é representada no diagrama de componentes da Figura 5, que explicita os componentes do *front-end* e do *back-end*, a interface HTTPS/REST que os conecta e as dependências em relação aos serviços de dados (PostgreSQL e Redis) e aos serviços externos (HIBP, Azure AI Foundry e Amazon SES).

Figura 5 – Diagrama de componentes do SafeID



Fonte: elaborado pelos autores (2026).

4.2 Projeto dos Componentes

Os componentes seguem uma arquitetura em camadas inspirada na Clean Architecture, sem adotá-la de forma estrita. O núcleo de regras (pasta core, com entidades, repositórios abstratos, casos de uso e os motores de risco e de IA) é separado da infraestrutura (pasta infra, com banco, cache, filas, clientes HTTP e envio de e-mail) e da camada de entrada (módulos NestJS, com controladores e DTOs). A regra de dependência é cumprida pelo Motor de Risco, que não importa nenhum framework, mas tem duas exceções conhecidas: o caso de uso importa constantes de fila da infraestrutura, e o AIEngine, mantido no núcleo, depende diretamente do SDK da OpenAI. Por isso, o documento usa a denominação "arquitetura em camadas inspirada na Clean Architecture"; a verificação automática da regra de dependência no CI está registrada como trabalho futuro (Seção 6.3). O isolamento do Motor de Risco permite testá-lo sem acionar serviços pagos. As responsabilidades e interfaces dos principais componentes são as seguintes:

- **Componentes de apresentação do *front-end*:** as telas de cadastro, verificação de e-mail e login recebem as credenciais e exibem os estados de carregamento e de erro; o painel (*dashboard)* apresenta o termômetro de risco, a legenda das faixas, a lista de incidentes, o plano de ação, o histórico e as notificações. Comunicam-se com o *back-end* exclusivamente pela interface HTTPS/REST.
- **Controllers da API REST:** expõem os *endpoints* da aplicação, como as rotas de autenticação e verificação de e-mail (/api/v1/auth), de histórico (GET /api/v1/scan/history), de monitoramento e de notificações e a verificação de saúde (GET /api/health); delegam a validação à camada de DTOs e o processamento aos serviços. Não há rota para o usuário disparar uma consulta: ela ocorre no cadastro, no monitoramento e nas novas tentativas automáticas.
- **Validação e proteção de entrada:** DTOs com *class-validator* rejeitam entradas fora dos esquemas definidos (RNF07); o Helmet configura cabeçalhos de segurança (RNF06); e o *rate limit* aplica a regra RN04.
- **Módulo de Consulta (ScanService e ExecuteRiskScanUseCase):** os dois componentes têm papéis distintos. O ScanService é o serviço de aplicação do módulo NestJS: recebe o pedido de consulta (cadastro, monitoramento ou nova tentativa), registra os vazamentos já conhecidos do usuário (UserBreach), atualiza o resultado exibido no painel, grava um resultado pendente quando a consulta falha e responde às consultas de histórico. O ExecuteRiskScanUseCase é o caso de uso do núcleo,

independente do NestJS: verifica o cache, enfileira a consulta à HIBP, aguarda o job por até 20 s, aciona os motores de risco e de IA, persiste o ScanHistory por meio de um repositório abstrato e atualiza o cache. Em síntese, o caso de uso define como uma varredura é feita; o serviço define quando ela acontece e o que muda na conta do usuário.

- **Motor de Risco:** componente de domínio, sem dependências de infraestrutura, que implementa as regras RN02, RN03 e RN08 (pesos, fator temporal, normalização e classificação em faixas).
- **Cliente HIBP (HibpClient) com *Circuit Breaker*:** encapsula a comunicação com a API HIBP, o tratamento dos códigos HTTP (404, sem vazamentos; 429, limite de uso; 401 e 403, chave inválida; 5xx, indisponibilidade) e o disjuntor implementado com Opossum (RNF04, Seção 4.7.2).
- **Workers BullMQ:** consomem os *jobs* das três filas no Redis: hibp-check (única fila que chama a HIBP, para que o limite de requisições valha para todas as instâncias), monitoring (sincronização do catálogo, varreduras, reconciliação e limpeza de contas não verificadas) e email (envio pelo Amazon SES).
- **Motor Cognitivo (AIEngine):** encapsula o acesso ao modelo de linguagem pelo SDK da OpenAI, com modelo e tempo-limite configuráveis por variável de ambiente, validação da resposta em JSON, cache das recomendações por 7 dias e resposta padrão determinística.
- **Módulo de Autenticação (AuthService):** responsável por cadastro, verificação de e-mail, *login com senha ou Google*, emissão e renovação de *tokens* JWT e exclusão de conta.
- **Persistência (Prisma ORM) e Serviço de Criptografia:** mapeiam as entidades do modelo de dados (Seção 4.3) e aplicam AES-256-GCM aos dados de incidentes e HMAC-SHA256 aos identificadores pseudonimizados (Seção 4.7.1).
- **Logger estruturado:** registra eventos em formato JSON, sem e-mails em texto claro (Seção 4.7.3).
- **Monitoramento e Notificações:** sincronizam o catálogo público da HIBP, detectam vazamentos novos por comparação com os já conhecidos de cada usuário (UserBreach), criam a notificação do painel e enfileiram o e-mail de alerta.

4.3 Modelo de Dados

O modelo de dados do SafeID foi projetado para garantir rastreabilidade, segurança e flexibilidade no armazenamento das informações relacionadas ao monitoramento de exposições digitais. A modelagem segue o paradigma relacional, utilizando o PostgreSQL como sistema gerenciador, e foi implementada por meio do Prisma ORM, o que assegura tipagem forte e integração com o *back-end* desenvolvido em NestJS.

4.3.1 Modelo Entidade-Relacionamento (MER)

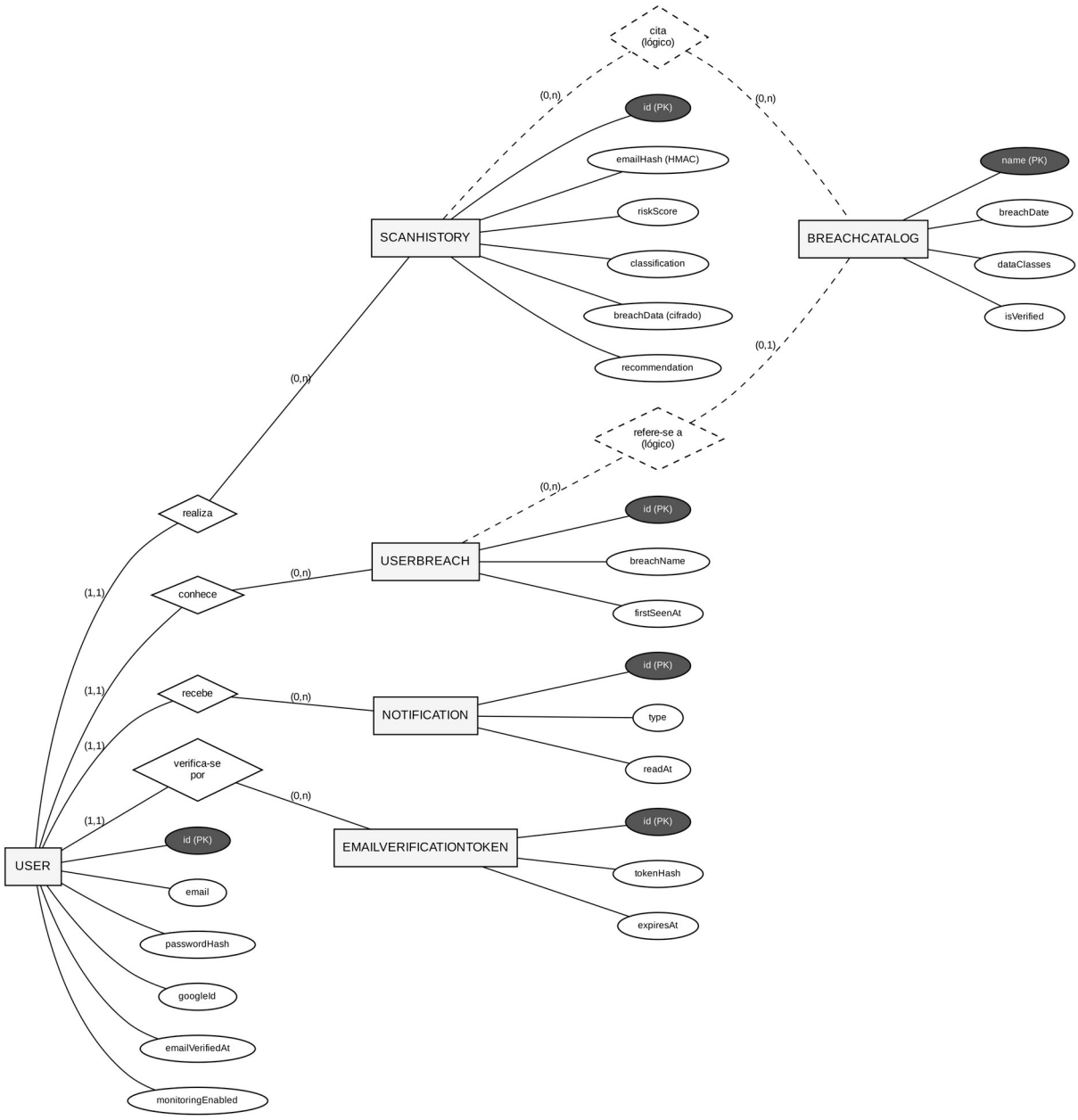
O MER do sistema, derivado do esquema Prisma em produção, é composto por seis entidades e uma tabela de controle:

- **User:** representa o usuário autenticado na plataforma. Cada registro armazena o e-mail, o *hash* da senha (ou o identificador Google), o último resultado exibido, as datas de aceite da política e de verificação do e-mail e as preferências de monitoramento.
- **ScanHistory:** armazena o histórico de varreduras realizadas pelo usuário, incluindo o *HMAC* do e-mail consultado, *score* de risco, classificação, número de incidentes, dados dos incidentes cifrados, resumo e passos gerados pela IA, origem da consulta e *timestamps*. Cada ScanHistory está vinculado a um User.
- **UserBreach:** registra os vazamentos já conhecidos de cada usuário, base para detectar os novos no monitoramento.
- **BreachCatalog:** cópia local do catálogo público da HIBP (nome, datas, classes de dados e indicadores de verificação), sincronizada a cada 6 horas.
- **Notification:** notificações de vazamento novo exibidas no painel, com o status do e-mail de alerta.
- **EmailVerificationToken:** hash dos tokens dos links de verificação de e-mail, com validade e uso único.
- **CatalogSyncState:** tabela de controle de linha única que registra a última sincronização completa do catálogo; por definição, não se relaciona com as demais.

Todos os relacionamentos de User são de um para muitos com cardinalidade mínima zero (0,n): um usuário recém-cadastrado ainda não tem consultas, vazamentos conhecidos, notificações ou, se entrou pelo Google, tokens de verificação. As chaves estrangeiras usam exclusão em cascata, o que implementa a eliminação dos dados do titular (Seção 4.7.4). O BreachCatalog relaciona-se logicamente com UserBreach (breachName = name) e com os incidentes contidos em ScanHistory.breachData. Esses vínculos não usam chave estrangeira

por decisão de projeto: o catálogo é uma cópia de dados externos, que pode ser recriada a qualquer momento, e um vazamento pode chegar ao usuário antes de a sincronização registrá-lo no catálogo. No MER e no DER, esses vínculos aparecem tracejados. O modelo ainda não contempla múltiplos identificadores por usuário, evolução prevista para a história de usuário US07. O modelo entidade-relacionamento está representado na Figura 6.

Figura 6 – Modelo Entidade-Relacionamento (MER) do SafeID



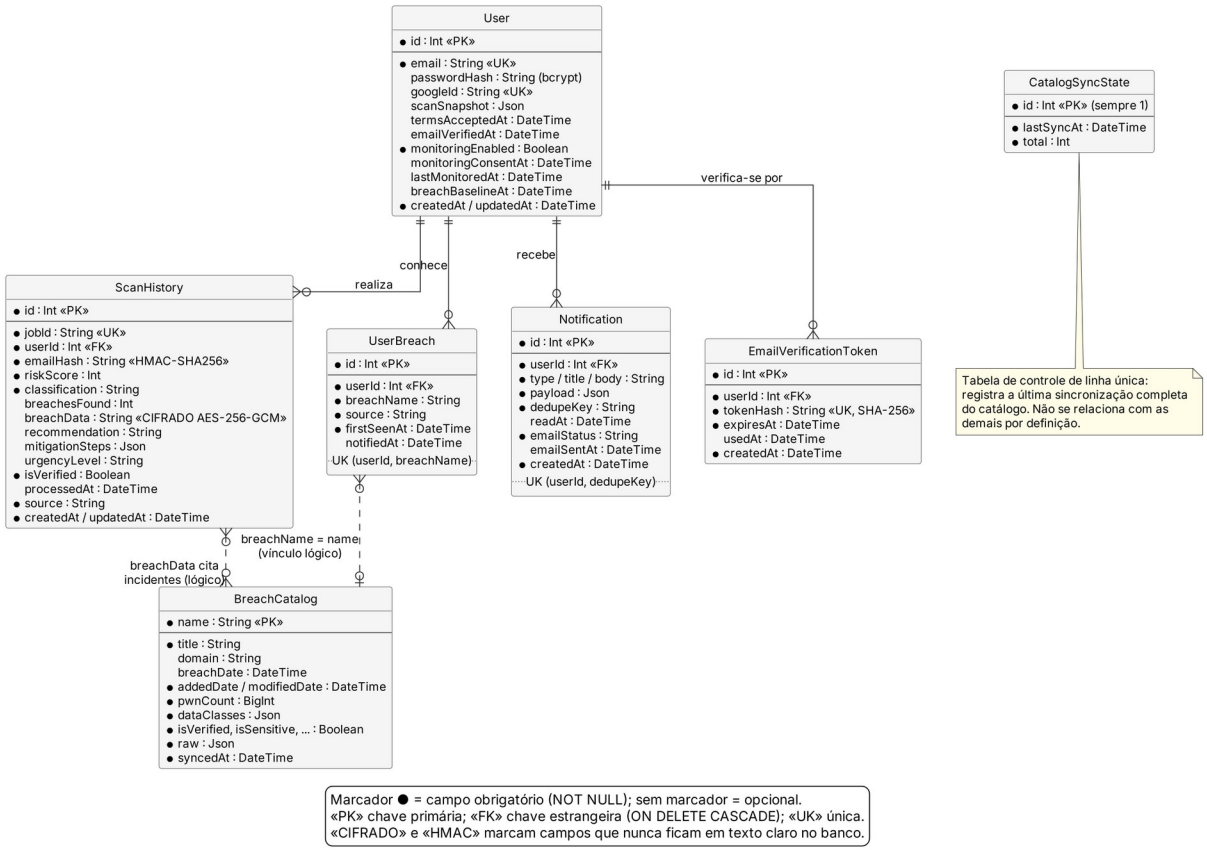
Fonte: elaborado pelos autores (2026).

4.3.2 Diagrama Entidade-Relacionamento (DER)

O DER da Figura 7 ilustra as entidades, seus atributos e seus relacionamentos, com a obrigatoriedade de cada campo e as chaves marcadas. Os campos que nunca ficam em texto

claro no banco são destacados no próprio diagrama: breachData, cifrado com AES-256-GCM, e emailHash, pseudonimizado com HMAC-SHA256.

Figura 7 – Diagrama Entidade-Relacionamento (DER) do SafeID

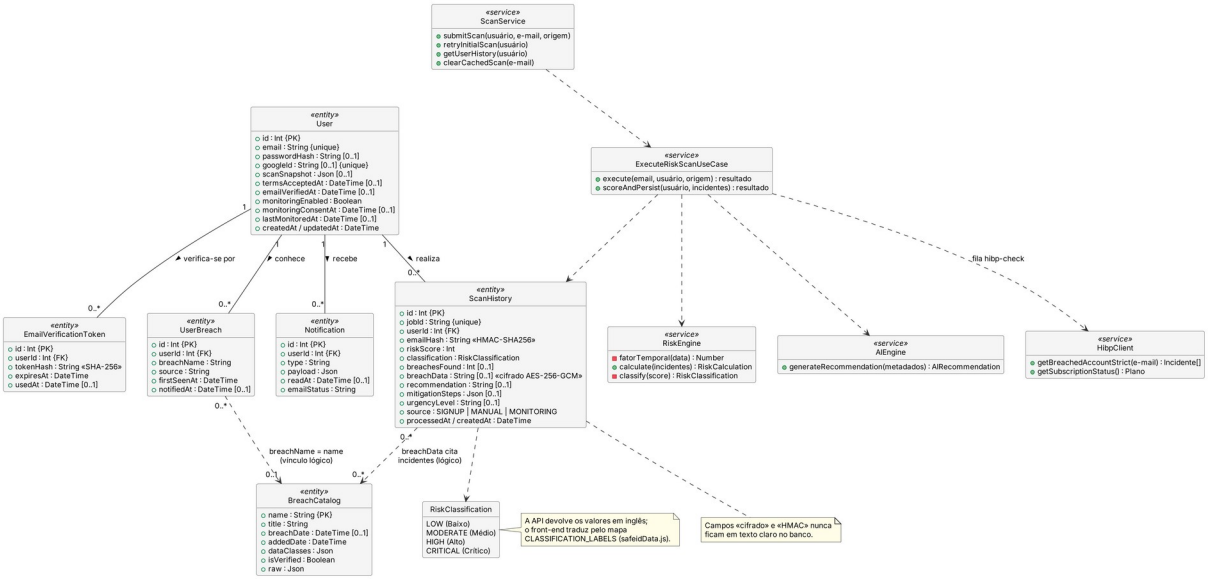


Fonte: elaborado pelos autores (2026).

4.3.3 Diagrama de Classes de Projeto

A Figura 8 apresenta o diagrama de classes de projeto, que refina o modelo conceitual da Seção 3.4 incorporando as decisões de implementação: as entidades persistentes, com os tipos definidos no esquema Prisma; a enumeração de classificação de risco, coerente com as faixas da Seção 2.4.1.3; e os principais serviços que manipulam essas entidades. As operações dos serviços são representativas das responsabilidades descritas na Seção 4.2. A enumeração RiskClassification usa termos em inglês, que são os valores trafegados pela API e persistidos; a interface os traduz para o português por um único mapa no front-end (CLASSIFICATION_LABELS: LOW, Baixo; MODERATE, Médio; HIGH, Alto; CRITICAL, Crítico), indicado em nota no diagrama. Os campos cifrados e pseudonimizados são marcados com os estereótipos «cifrado» e «HMAC».

Figura 8 – Diagrama de classes de projeto do SafeID



Fonte: elaborado pelos autores (2026).

4.3.4 Dicionário de Dados

O Quadro 5 apresenta o dicionário de dados do SafeID, derivado do esquema Prisma em produção. Para cada campo, informa o tipo, a obrigatoriedade (S, obrigatório; N, opcional) e a participação em chaves (PK, primária; FK, estrangeira; UK, única).

Quadro 5 – Dicionário de dados do SafeID

Entidade	Campo	Tipo	Obrig.	Chave	Descrição
User	id	Int	S	PK	Identificador do usuário (autoincremento)
	email	String	S	UK	E-mail do titular, usado no login e nos alertas
	passwordHash	String	N		Hash bcrypt da senha (nulo em contas só Google)
	googleId	String	N	UK	Identificador da conta Google
	scanSnapshot / scanSnapshotUpdatedAt	Json / DateTime	N		Último resultado exibido no painel e data da atualização
	termsAcceptedAt	DateTime	N		Aceite da Política de Privacidade
	emailVerifiedAt	DateTime	N		Confirmação do e-mail (sem ela, não há login nem consulta)
	monitoringEnabled	Boolean	S		Monitoramento contínuo ativo (padrão: verdadeiro, com opt-out)
	monitoringConsentAt / lastMonitoredAt / breachBaselineAt	DateTime	N		Consentimento, última verificação e definição da base de vazamentos conhecidos
	createdAt / updatedAt	DateTime	S		Datas de criação e atualização
ScanHistory	id	Int	S	PK	Identificador da consulta
	jobId	String	S	UK	Identificador do processamento
	userId	Int	S	FK	Usuário dono da consulta (exclusão em cascata)
	emailHash	String	S		HMAC-SHA256 do e-mail consultado (indexado)
	riskScore	Int	S		Score de 0 a 100, inteiro (RN08)

Entidade	Campo	Tipo	Obrig.	Chave	Descrição
	classification	String	S		LOW, MODERATE, HIGH ou CRITICAL (RN08)
	breachesFound	Int	N		Número de incidentes encontrados
	breachData	String	N		JSON dos incidentes cifrado com AES-256-GCM (prefixo enc:v1)
	recommendation / mitigationSteps / urgencyLevel	String / Json / String	N		Resumo, passos e urgência gerados pela IA
	isVerified	Boolean	S		Verdadeiro se o resultado veio da HIBP; falso se veio do cache
	source	String	S		Origem: SIGNUP, MANUAL ou MONITORING
	processedAt / createdAt / updatedAt	DateTime	N / S / S		Datas de processamento, criação e atualização
UserBreach	id	Int	S	PK	Identificador do registro
	userId	Int	S	FK	Usuário (exclusão em cascata); único com breachName
	breachName	String	S	UK*	Nome do vazamento no HIBP; vínculo lógico com BreachCatalog.name
	source / firstSeenAt / notifiedAt	String / DateTime / DateTime	S / S / N		Origem, primeira detecção e data da notificação
BreachCatalog	name	String	S	PK	Nome estável do vazamento no HIBP
	title / domain	String	S / N		Título e domínio do serviço afetado
	breachDate / addedDate / modifiedDate	DateTime	N / S / S		Data do incidente e datas de inclusão e alteração no HIBP
	pwnCount	BigInt	S		Contas afetadas
	dataClasses	Json	S		Classes de dados expostas (Tabela 1)
	isVerified, isFabricated, isSensitive, isRetired, isSpamList, isMalware, isStealerLog	Boolean	S		Indicadores de qualidade e natureza do incidente
	raw / syncedAt	Json / DateTime	S		Payload original e data da sincronização
Notification	id	Int	S	PK	Identificador da notificação
	userId	Int	S	FK	Usuário (exclusão em cascata); único com dedupeKey
	type / title / body / payload	String / Json	S		Tipo (NEW_BREACH), textos e dados do alerta
	dedupeKey	String	S	UK*	SHA-256 do tipo e dos nomes, evita alertas duplicados
	readAt / emailStatus / emailSentAt / createdAt	DateTime / String	N / S / N / S		Leitura, status e data do e-mail, criação
EmailVerification Token	id	Int	S	PK	Identificador do token
	userId	Int	S	FK	Usuário (exclusão em cascata)
	tokenHash	String	S	UK	SHA-256 do token; o token em claro só existe no e-mail
	expiresAt / usedAt / createdAt	DateTime	S / N / S		Validade (24 h), uso e criação
CatalogSyncState	id	Int	S	PK	Sempre 1 (linha única)
	lastSyncAt / total	DateTime / Int	S		Última sincronização completa e total de vazamentos no catálogo

Fonte: elaborado pelos autores (2026).

UK* indica unicidade composta com userId. O modelo foi validado para garantir conformidade com as práticas de segurança e privacidade, incluindo a pseudonimização de identificadores sensíveis e a criptografia dos dados de incidentes em repouso, conforme detalhado na Seção 4.7.

4.4 Projeto da Interface do Usuário

A interface do SafeID é concebida como o componente responsável por converter dados complexos em informações acionáveis pelo cidadão, reduzindo a assimetria de informação entre especialistas em cibersegurança e o público leigo. Seu projeto partiu de um protótipo de alta fidelidade elaborado no Figma (item 06 do *backlog*, Apêndice A) e segue a abordagem *Mobile First* (RNF05), com estilização por classes utilitárias do Tailwind CSS.

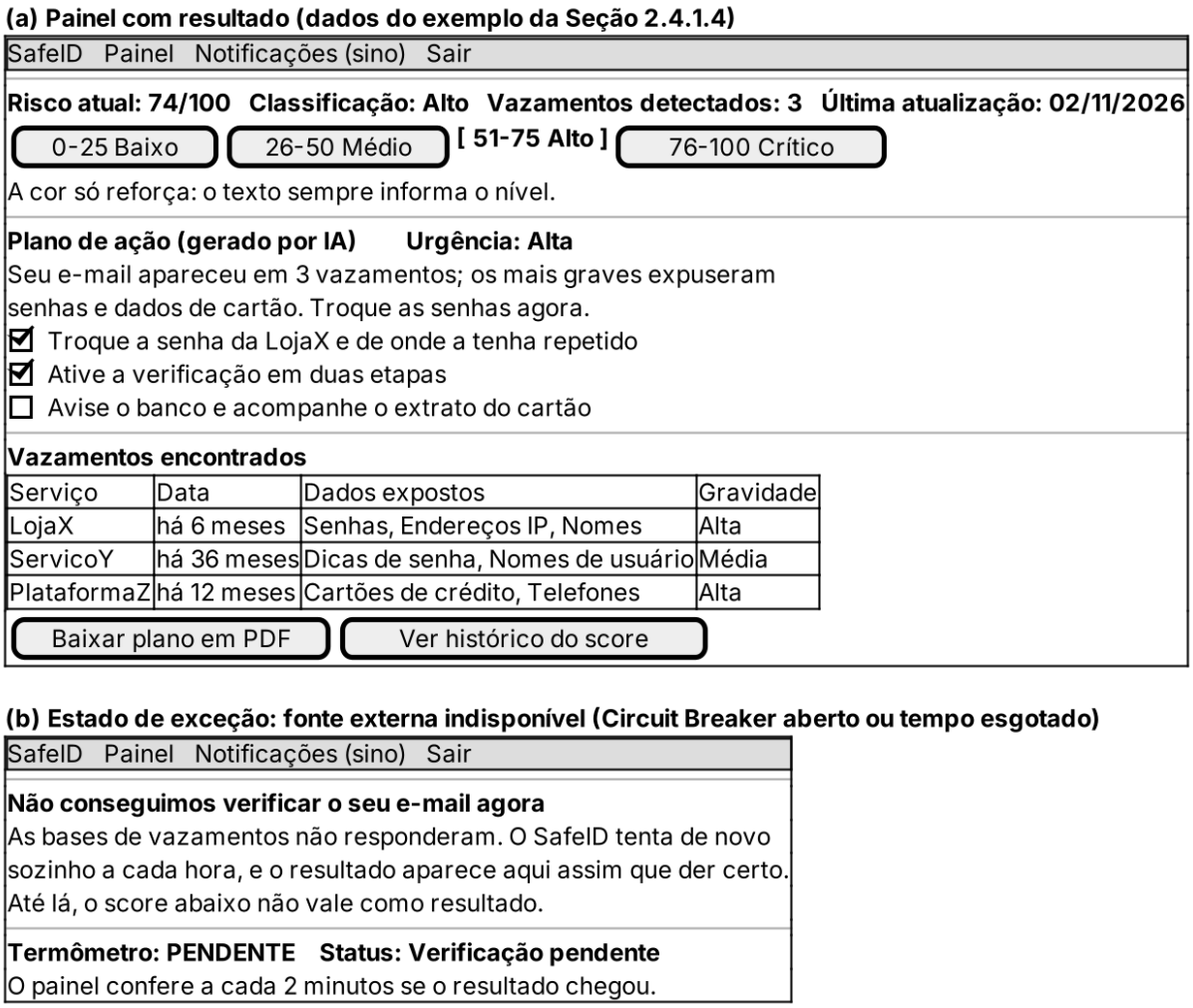
A organização das telas acompanha a jornada do usuário descrita nos casos de uso:

- **Telas de cadastro, verificação e login:** recebem e-mail e senha (ou o login com Google), exigem o aceite da Política de Privacidade, validam o formato da entrada antes do envio e orientam o usuário a confirmar o e-mail pelo link recebido;
- **Tela de resultado (*dashboard*):** exibe o *Risk Score* em um termômetro de risco com as cores de semáforo das faixas (verde, amarelo, laranja e vermelho) e o nível sempre escrito por extenso, a legenda que explica cada nível (US02), o plano de ação gerado pela IA, a lista de incidentes com as classes de dados expostas, traduzidas, sem jamais exibir senhas (RN05), o gráfico de evolução do score e o sino de notificações;
- **Estados de exceção:** mensagens amigáveis para e-mail inválido, ausência de vazamentos, limite de requisições atingido e indisponibilidade temporária da fonte externa, quando o score é marcado como pendente e a consulta é refeita automaticamente (*Figura* 9b);
- **Telas implementadas na PI-2:** cadastro com verificação de e-mail e login com Google, histórico com gráfico de evolução (US05), plano de ação gerado pela IA (US03 e US04), notificações e preferências de monitoramento (US08), exportação em PDF (US09), configurações da conta (troca de senha, cópia dos dados e exclusão), criação de nova senha pelo link do e-mail, Termos de Uso, Política de Privacidade e página de endereço não encontrado. Ficaram fora da entrega final a gestão de identificadores (US07), o painel administrativo (US10) e o tema claro: a interface atual usa apenas o tema escuro.

A Figura 9 apresenta uma representação esquemática (*wireframe*) do painel em dois estados: (a) resultado, preenchido com os dados do exemplo numérico da Seção 2.4.1.4; e (b)

estado de exceção, quando a fonte externa está indisponível (Circuit Breaker aberto ou tempo esgotado), com as mesmas mensagens exibidas pela aplicação. A lista de vazamentos mostra apenas as classes de dados expostas, traduzidas (por exemplo, “Senhas”), sem metadados técnicos de representação da senha, como o algoritmo de hash, que o HIBP não informa e que confundiriam o usuário leigo (RN05 e Seção 3.7).

Figura 9 – Representação esquemática (*wireframe*) do painel de resultado e do estado de indisponibilidade



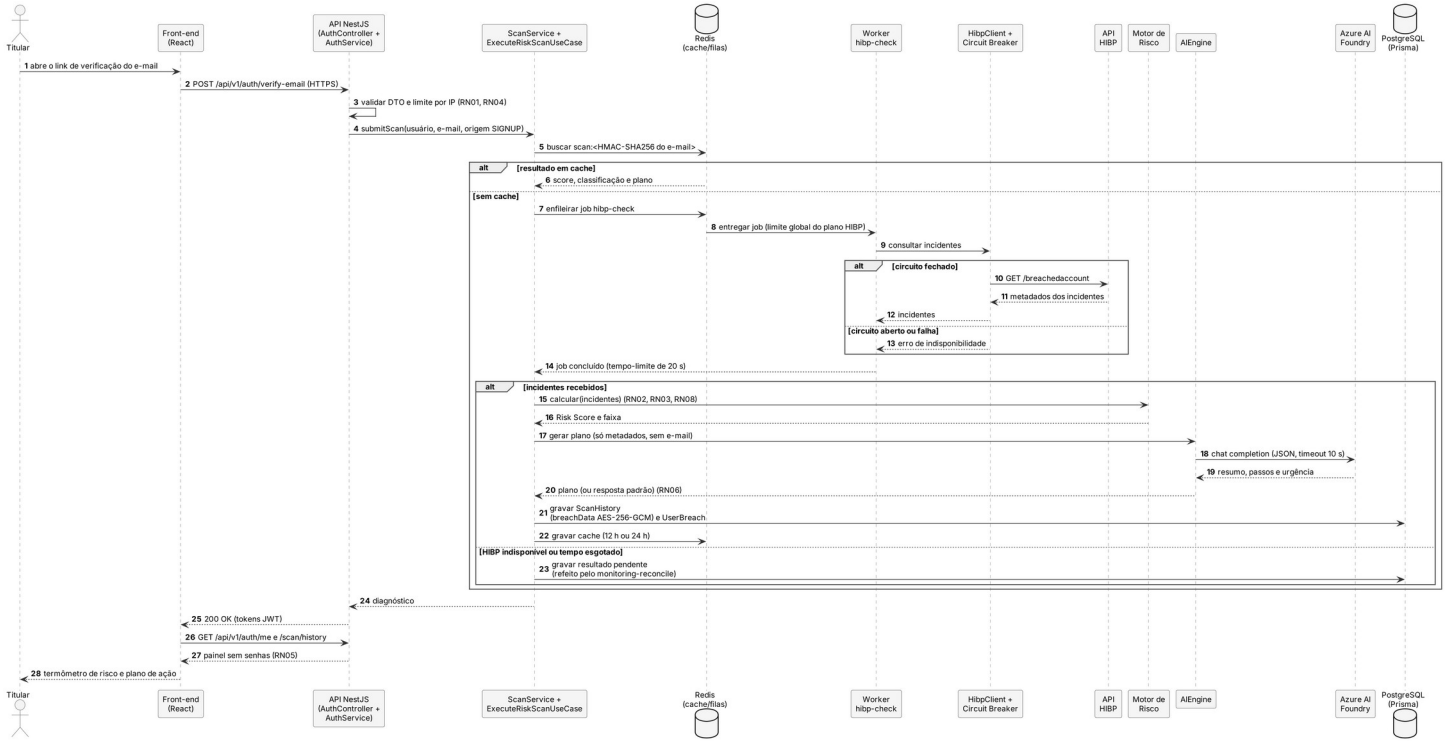
Fonte: elaborado pelos autores (2026).

4.5 Diagramas de Interação

O diagrama de sequência da Figura 10 detalha a colaboração entre os elementos do sistema na realização do caso de uso principal (UC01 – Consultar exposição do e-mail), disparado pela confirmação do e-mail no cadastro. Após a validação da entrada e do limite de requisições, o ScanService aciona o caso de uso, que verifica o *cache* pelo *HMAC*-SHA256 do identificador; na ausência de resultado, enfileira um *job* consumido pelo worker hibp-check, que respeita o limite global *do plano da HIBP*. O *worker* consulta a API por meio do

cliente protegido pelo *Circuit Breaker*. Com os incidentes recebidos, o Motor de Risco calcula o *score* e a faixa, e o AIEngine gera o plano de ação a partir de metadados sem identificador pessoal, ou a resposta padrão, se o modelo não responder. O histórico é persistido com os dados de incidentes cifrados, o cache é atualizado e o painel é exibido sem senhas. Se a HIBP estiver indisponível ou o tempo-limite de 20 s se esgotar, grava-se um resultado pendente, que a rotina de reconciliação refaz de hora em hora.

Figura 10 – Diagrama de sequência da consulta de exposição (UC01)



Fonte: elaborado pelos autores (2026).

4.6 Tecnologias Utilizadas

Esta seção apresenta as tecnologias selecionadas para a implementação e a operação do SafeID, considerando critérios de adequação ao escopo, segurança, desempenho e manutenibilidade. As subseções detalham as soluções adotadas no *front-end*, no *back-end*, na persistência de dados, na integração com inteligência artificial e serviços externos, bem como na infraestrutura em nuvem em que a aplicação está implantada.

4.6.1 Front-end: React.js e Tailwind CSS

Para viabilizar a camada de apresentação, utiliza-se a biblioteca React.js (META, [s. d.]), fundamentada em uma arquitetura baseada em componentes. Em comparação com Vue, o React oferece maior aderência ao ecossistema já adotado pela equipe e ampla disponibilidade de bibliotecas para gerenciamento de estado e testes; em relação ao Angular, reduz a imposição de uma estrutura completa de *framework*, adequada ao escopo incremental do MVP. A escolha também se apoia na eficiência do seu algoritmo de reconciliação (*Virtual DOM*), que permite atualizações granulares na interface sem a necessidade de recarregar a página integralmente. No contexto do SafeID, o React.js gerencia o estado de requisições assíncronas complexas — como o tempo de espera (*loading*) enquanto o *back-end* processa filas no BullMQ ou a exibição de *fallbacks* caso o *Circuit Breaker* seja acionado. Ao decompor a interface em unidades lógicas, elementos críticos, como o *dashboard* de vulnerabilidades e o indicador de *Risk Score*, funcionam de forma isolada e testável.

Integrado ao React, o Tailwind CSS (TAILWIND LABS, [s. d.]) gerencia a estilização pelo paradigma *utility-first*. A abordagem *Mobile First* favorece responsividade e acessibilidade em diferentes dispositivos. As classes utilitárias também permitem representar visualmente as faixas do Motor de Risco e controlar o volume dos artefatos de estilo; o efeito sobre métricas de carregamento deve ser verificado por testes de desempenho.

A combinação entre React e Tailwind CSS permite separar componentes funcionais e regras de apresentação, mantendo consistência visual no consumo de APIs e na exibição dos resultados. A camada de apresentação renderiza as explicações geradas pela Inteligência Artificial em formato didático e acessível. Dessa forma, a interface contribui para a transparência e a clareza exigidas pelas diretrizes da LGPD.

4.6.2 Back-end: Node.js, NestJS e Prisma ORM

A infraestrutura de serviços do SafeID, responsável por orquestrar a lógica de processamento e a comunicação com as bases de inteligência cibernética, foi desenvolvida sobre o ecossistema Node.js (NODE.JS, [s. d.]) utilizando TypeScript em modo estrito (o front-end, por sua vez, é escrito em JavaScript com JSX). A escolha dessa *stack* fundamenta-se no modelo de I/O não bloqueante e orientado a eventos do Node.js, que permite ao servidor gerenciar múltiplas requisições assíncronas, como as chamadas à API OSINT e as requisições de análise cognitiva ao modelo de linguagem, integrado por meio do SDK da OpenAI (versão 6.41) ao Azure AI Foundry. Para favorecer manutenibilidade, testabilidade e isolamento das regras de negócio, o *back-end* foi estruturado com o **NestJS**. Em comparação com o Express puro, o NestJS fornece organização modular, injeção de dependência e convenções arquiteturais integradas, reduzindo a necessidade de definir manualmente esses mecanismos no projeto. O *framework* organiza o sistema por meio de módulos, *controllers* e *providers* (serviços), implementando os princípios de Inversão de Controle (IoC) e Injeção de Dependência (DI).

O NestJS fornece a camada HTTP e o roteamento REST, além de interceptadores, *guards* e *middlewares*. DTOs e *class-validator* aplicam validação declarativa; o Helmet reduz a exposição de cabeçalhos e permite configurar políticas de segurança; Redis e BullMQ organizam filas em segundo plano; e o *Circuit Breaker*, com Opossum (NODESHIFT, 2026), limita a propagação de falhas de serviços externos (Seção 4.7.2). Esses controles reduzem riscos específicos, mas dependem de configuração, testes e monitoramento.

Na persistência, o Prisma ORM (PRISMA, [s. d.]) oferece uma abstração orientada a tipos e utiliza consultas parametrizadas, o que reduz a exposição a injeção SQL sem eliminar a necessidade de validação e controle de acesso. Os controles de pseudonimização e de criptografia aplicados aos dados persistidos estão especificados na Seção 4.7.1.

A combinação entre o modelo assíncrono do Node.js, a modularidade do NestJS e a tipagem do Prisma ORM fornece uma base adequada ao processamento e à mitigação de riscos previstos no escopo. Essa estrutura favorece a separação de responsabilidades, a realização de testes e a evolução controlada dos módulos, mantendo a viabilidade técnica do produto no contexto acadêmico e social proposto.

4.6.3 Banco de Dados: PostgreSQL e Redis

A camada de armazenamento do SafeID foi projetada segundo o princípio de *Privacy by Design*, com estratégia híbrida de persistência relacional e em memória destinada a favorecer desempenho mensurável, segurança e alinhamento com as diretrizes da LGPD.

A persistência de longo prazo é estruturada sobre o sistema gerenciador de banco de dados relacional PostgreSQL (POSTGRESQL GLOBAL DEVELOPMENT GROUP, [s. d.]), escolhido pela integridade transacional, pelo suporte a restrições relacionais e pelos recursos de segurança. Em comparação com o MongoDB, o PostgreSQL é mais aderente às relações entre usuários, históricos e registros de auditoria do SafeID, pois permite impor consistência referencial e transações ACID diretamente no esquema. Academicamente, o PostgreSQL justifica-se por ser um banco de dados de código aberto que implementa de forma sistemática os padrões SQL, oferecendo suporte nativo a transações **ACID** (Atomicidade, Consistência, Isolamento e Durabilidade). No contexto do SafeID, tais propriedades contribuem para que o histórico de consultas de vazamentos, os perfis de usuários cadastrados e os *logs* analíticos sejam consolidados sem riscos de corrupção ou perda de sincronia, pilares fundamentais para a confiabilidade de uma ferramenta de cibersegurança.

Limitação e risco técnico: uma instância relacional central pode representar ponto único de falha caso não haja redundância, replicação e rotinas de recuperação verificadas. Em produção, o Amazon RDS mantém backups automáticos com retenção de 1 dia, o limite do plano gratuito da AWS, e um snapshot manual é feito antes de cada migration, mas a instância não é Multi-AZ; a meta de disponibilidade correspondente está no RNF10, e a adoção de alta disponibilidade está registrada como evolução (Seção 6.3).

Em um projeto focado em Responsabilidade Social Digital, a proteção do dado do cidadão é o elemento central. O banco de produção usa armazenamento criptografado em repouso, e os dados de incidentes recebem, adicionalmente, criptografia na aplicação antes de seguirem para o banco (Seção 4.7.1). A integração com o Prisma ORM assegura que o mapeamento das entidades reflita a lógica de negócio de maneira tipada (*type-safe*), com consultas parametrizadas que reduzem o risco de *SQL Injection*.

Complementando a camada relacional, o sistema integra o Redis como banco de dados chave-valor em memória, focado em três frentes:

1. ***Cache* de consultas:** o Redis armazena resultados recentes (12 horas quando há vazamentos e 24 horas quando não há) e as recomendações da IA (7 dias), reduzindo

chamadas repetidas à HIBP e ao modelo. As chaves usam o HMAC-SHA256 do e-mail (Seção 4.7.1).

2. **Mensageria e filas:** o Redis atua como a infraestrutura de dados que alimenta o **BullMQ**, gerenciando de modo ordenado e com mecanismos de recuperação o enfileiramento das consultas, varreduras e e-mails que aguardam processamento pelos *workers* do *back-end*.
3. **Limitação de requisições distribuída:** o limitador do BullMQ fica no próprio Redis, de modo que o limite de requisições da HIBP vale para todas as instâncias da aplicação ao mesmo tempo. Em produção, o Redis usa a política noeviction, exigida pelo BullMQ para não descartar dados de jobs sob pressão de memória.

A estratégia híbrida entre PostgreSQL e Redis separa a persistência transacional do armazenamento temporário e da infraestrutura de filas. Essa divisão atende às correlações exigidas pelo Motor de Risco, reduz consultas externas repetidas e mantém controles distintos para dados persistentes e voláteis, sem eliminar a necessidade de monitoramento, redundância e testes de recuperação.

4.6.4 Inteligência Artificial e APIs Externas

A capacidade analítica do SafeID depende da integração de serviços externos de inteligência e processamento de linguagem. O sistema utiliza a API Have I Been Pwned (HIBP; HAVE I BEEN PWNED, [s. d.]) para consultar registros de comprometimento de dados provenientes de fontes OSINT (*Open Source Intelligence*). A escolha decorre da abrangência histórica da base e da disponibilidade de metadados estruturados para consulta. Para respeitar o limite de requisições por minuto do plano contratado, lido diariamente na própria API (Seção 3.7), a arquitetura consome os dados de forma assíncrona por meio de uma única fila (**hibp-check**), com limitador global no Redis. A comunicação também utiliza o padrão ***Circuit Breaker (Seção 4.7.2)***, reduzindo o efeito de instabilidades externas sobre o *back-end*.

Complementando a coleta OSINT, o SafeID utiliza um modelo de linguagem hospedado no Azure AI Foundry, acessado pelo SDK oficial da OpenAI (versão 6.41), que é compatível com esse provedor. O modelo em produção é o grok-4-20-non-reasoning, configurável pela variável AI_MODEL sem alteração de código. O projeto previa inicialmente o GPT-4o-mini; a troca de provedor e os critérios da escolha estão registrados na Seção 6.2.2. O modelo recebe metadados de incidentes e devolve, em JSON, um resumo, de 4 a 6 passos de mitigação e um nível de urgência. As chamadas são autenticadas por chave de

API guardada como segredo no AWS Systems Manager (SSM) e injetada no contêiner na inicialização.

Limitação e risco técnico da LLM: o uso de um modelo de terceiros introduz custo variável por volume de requisições e latência sujeita à carga, à rede e aos limites do provedor. Esses fatores podem afetar o orçamento e o tempo de resposta, exigindo limites de consumo, monitoramento, *timeouts*, *cache* quando aplicável e respostas determinísticas de contingência.

Limitação e risco técnico da HIBP: a consulta de novas exposições depende criticamente da disponibilidade da API externa HIBP. Interrupções, mudanças contratuais ou restrições de taxa podem impedir a atualização imediata dos resultados; por isso, o sistema mantém filas, *cache*, *Circuit Breaker*, mensagens de contingência e uma camada de integração desacoplada.

Na integração com a LLM, a temperatura 0,3, a saída obrigatória em JSON, a validação dos campos recebidos e as instruções *fixas do sistema (que vedam inventar fatos, pedir dados ou indicar links)* reduzem a variabilidade das respostas, mas não garantem ausência de alucinações. O *payload* foi projetado para excluir o e-mail e outros dados pessoais desnecessários, transmitindo somente os metadados requeridos para gerar a orientação. Essa minimização reduz a exposição de informações pessoais a terceiros e apoia o princípio de *Privacy by Design*.

A integração entre as APIs externas e o algoritmo de *Risk Score* interno combina dados estruturados, processamento assíncrono e geração de orientações. A governança dessa camada requer anonimização, controle de consumo, observabilidade e mecanismos de contingência para que a indisponibilidade de terceiros não altere o cálculo determinístico do risco. Assim, a aplicação mantém o foco na alfabetização digital e na proteção do usuário B2C sem equiparar seus resultados aos de plataformas corporativas.

4.6.5 Infraestrutura em Nuvem

Situação atual: a arquitetura descrita nesta seção está implantada e em produção. Ela foi projetada na PI-1 como arquitetura alvo, enquanto o MVP rodava em plataformas PaaS (Seção 5.4), e foi provisionada na AWS durante a PI-2 por scripts versionados no repositório. Os pontos em que a implantação atual difere do desenho completo estão indicados ao final da seção.

A adoção de contêineres Docker favorece a portabilidade da aplicação entre diferentes ambientes de execução e reduz a dependência de um fornecedor específico de infraestrutura. A topologia foi projetada para promover isolamento de rede e resiliência dos serviços, além

de apoiar o alinhamento com as diretrizes da LGPD; essas escolhas técnicas, contudo, não representam garantia de fidelidade a uma plataforma nem comprovação absoluta de conformidade, que também depende de governança, processos e avaliação contínua.

A topologia do SafeID baseia-se no ecossistema da Amazon Web Services (AWS) como provedor principal de *cloud computing*. A arquitetura foi desenhada utilizando o conceito de nuvem privada (Amazon VPC), de modo que o banco de dados e o cache permaneçam em sub-redes privadas, sem acesso direto pela internet pública. A arquitetura em produção utiliza os seguintes serviços da AWS:

- **Amazon ECS com AWS Fargate:** em substituição a servidores tradicionais (EC2), o Fargate executa o contêiner do back-end em NestJS sem a necessidade de gerenciar sistemas operacionais. Hoje roda uma task (0,5 vCPU e 1 GB) com API, workers e agendamentos; a imagem fica no Amazon ECR, com uma versão por commit.
- **Application Load Balancer (ALB):** atua como o único ponto de entrada público da API, recebendo o tráfego da internet, realizando a terminação de criptografia SSL/TLS (HTTPS) e distribuindo as requisições entre os contêineres saudáveis.
- **Amazon RDS (PostgreSQL):** banco de dados relacional gerenciado, provisionado em sub-redes isoladas, PostgreSQL 16 em instância db.t3.micro, com armazenamento criptografado e backups automáticos com retenção de 1 dia, o limite do plano gratuito da AWS.
- **Amazon ElastiCache (Redis):** *cache* de memória distribuído alocado na rede privada, que guarda o cache, as filas do BullMQ e o limitador de requisições da HIBP.
- **Amazon S3 e CloudFront:** o *front-end* em React é construído de forma estática, armazenado em bucket S3 privado (lido apenas pelo CloudFront) e distribuído globalmente por meio da rede de distribuição de conteúdo (CDN) CloudFront, o que pode reduzir a latência de distribuição e, em conjunto com controles específicos, contribuir para a mitigação de ataques DDoS.
- **AWS IAM e Security Groups:** políticas de acesso pautadas no princípio do menor privilégio (*least privilege*), em que *firewalls* virtuais controlam o fluxo de dados estritamente entre as camadas autorizadas: o contêiner só aceita conexões do ALB, e o banco e o Redis, só do contêiner. A permissão da aplicação na AWS limita-se a enviar e-mails pelo domínio safeid.app.br.
- **Amazon SES, SSM Parameter Store e CloudWatch:** o SES envia os e-mails de verificação e de alerta; os segredos (chaves de criptografia, de HMAC, da HIBP e da IA) ficam no SSM e são injetados no contêiner; os logs JSON vão para o CloudWatch,

com retenção de 30 dias, filtros de métrica e alarmes enviados à equipe por e-mail (SNS).

Diferenças entre o desenho e a implantação atual: (i) roda uma única task, com API e workers no mesmo processo, sem escalonamento automático (Seção 5.7); (ii) o RDS e o ElastiCache não são Multi-AZ; e (iii) desde 27/10/2026, para poupar os créditos da conta, o NAT Gateway está desligado (“modo economia”): a task roda na sub-rede pública com IP próprio, mas continua inacessível pela internet, porque seu security group só aceita o ALB; o NAT é religado, em cerca de 15 minutos, para apresentações. O diagrama da implantação está na Seção 5.4 (Figura 16).

Estimativa de custo. Segundo a documentação de implantação do repositório, o ambiente custa cerca de US$ 3,50 por dia com o NAT Gateway e US$ 2,45 por dia sem ele, ou seja, entre US$ 75 e US$ 105 por mês. A principal parcela fixa é o NAT (cerca de US$ 1 por dia). O custo do modelo de linguagem é variável: com as premissas do Apêndice C (R$ 0,0025 por chamada), 60 mil chamadas mensais custam cerca de R$ 150, valor reduzido pelo cache de recomendações de 7 dias, que evita nova chamada para o mesmo conjunto de vazamentos. A Tabela 5 (Apêndice C) consolida os custos em reais.

4.7 Segurança e Controle de Acesso

Em um cenário contemporâneo de proliferação de ameaças cibernéticas e vazamentos massivos de dados, a segurança da informação e a salvaguarda da privacidade não podem ser tratadas como meros requisitos adicionais de um sistema. No SafeID, esses elementos constituem a base de toda a arquitetura de software. O desenvolvimento da plataforma pautou-se de forma sistemática pelos conceitos complementares de ***Security by Design*** (Segurança desde o Projeto) e ***Privacy by Design*** (Privacidade desde o Projeto). Essa abordagem integra controles de proteção de dados e mitigação de vulnerabilidades ao código, às rotas de API e ao banco de dados desde o projeto; tais controles reduzem riscos, mas sua eficácia depende de configuração, testes, monitoramento e governança contínua.

4.7.1 Critérios de Segurança e Privacidade

Para operacionalizar os pilares de confidencialidade, integridade e disponibilidade, o SafeID implementa uma série de controles técnicos distribuídos em múltiplos níveis de sua arquitetura:

- **Camada de tráfego e proteção de aplicação:** a comunicação utiliza HTTPS/TLS. No *back-end*, o Helmet reduz a exposição de cabeçalhos e permite configurar políticas de

segurança, mas não previne sozinho XSS; codificação de saída e Política de Segurança de Conteúdo (CSP) são controles complementares. O *class-validator* rejeita entradas que não atendem aos esquemas definidos antes do processamento.

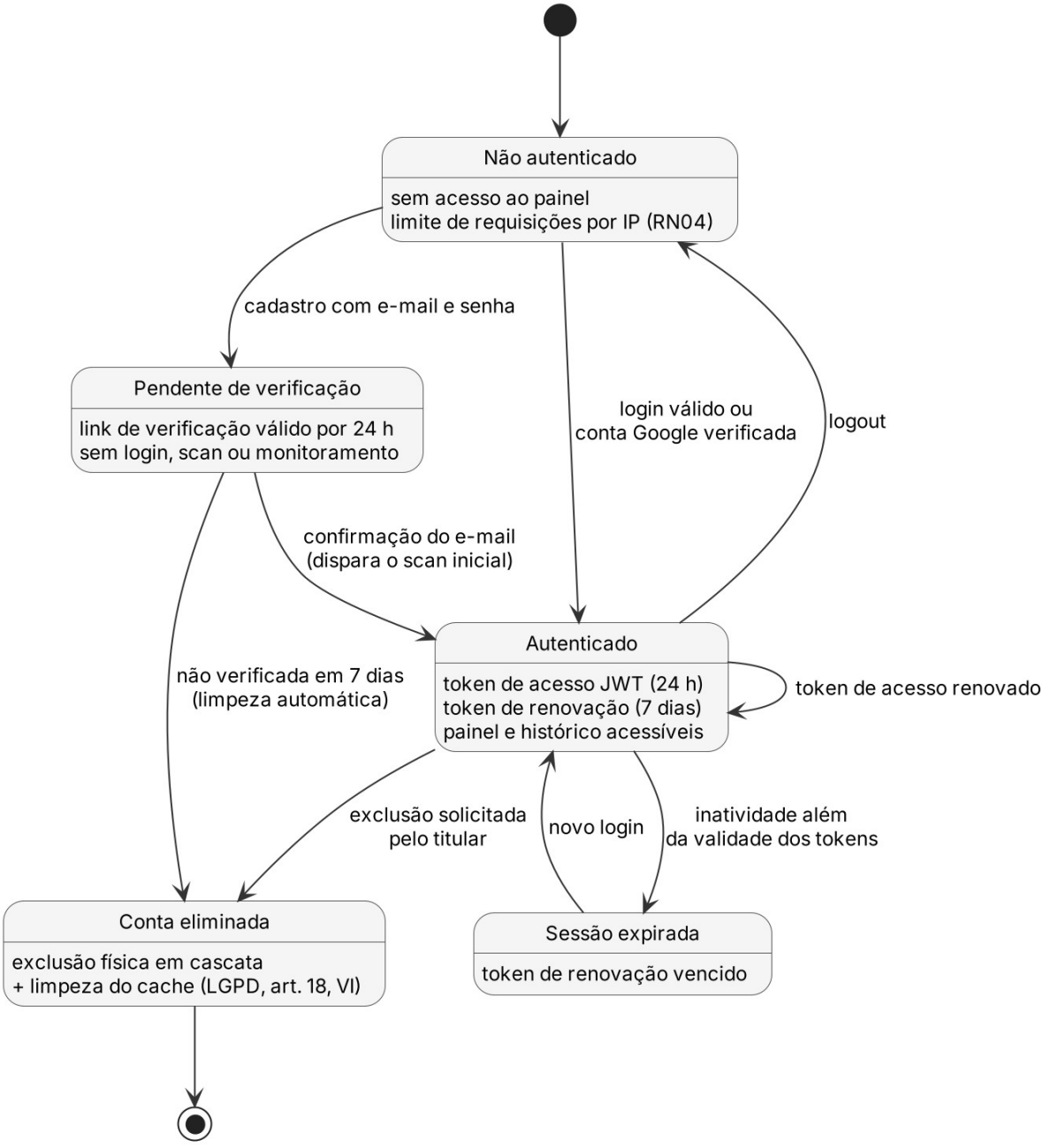
- **Segurança de dados e criptografia (especificação única, referida pelas demais seções):** (i) pseudonimização: o e-mail usado como chave de cache e gravado no histórico é transformado por HMAC-SHA256 com uma chave secreta (pepper), guardada fora do banco, no SSM. Diferentemente de um SHA-256 sem chave, que por ter entrada previsível permite recuperar o e-mail com ataques de dicionário ou tabelas pré-calculadas, o HMAC só pode ser recalculado por quem possui a chave. Se a variável EMAIL_HASH_SECRET não estiver configurada, o código volta ao SHA-256 sem chave e registra um aviso; por isso, a presença da chave em produção deve ser verificada a cada implantação, e em 05/11/2026 a equipe confirmou que ela está configurada no arquivo de ambiente de produção, de onde o script de implantação a envia ao SSM; (ii) criptografia: os dados de incidentes do histórico são cifrados com AES-256-GCM na aplicação, com chave de 32 bytes também guardada no SSM, o que protege confidencialidade e integridade quando chaves, vetores de inicialização e etiquetas de autenticação são geridos corretamente. Os tokens de verificação de e-mail são guardados apenas como hash SHA-256, aceitável por serem valores aleatórios de alta entropia.
- **Minimização de dados no consumo de Inteligência Artificial:** o fluxo de integração foi projetado para excluir o e-mail e outras informações pessoais desnecessárias do *payload*. A LLM recebe somente nome, data e classes de dados dos incidentes, além do score e da classificação, o que reduz a exposição de dados a terceiros, sem caracterizar garantia absoluta de anonimização. Para vazamentos que o HIBP marca como sensíveis, nem o nome do vazamento é enviado à IA ou ao e-mail de alerta; ele aparece apenas no painel do titular.

4.7.2 Autenticação, Autorização e Disponibilidade

O controle de acesso do SafeID exige conta com e-mail verificado para qualquer consulta. O cadastro cria a conta em estado pendente e envia um link de uso único, válido por 24 horas; contas não confirmadas em 7 dias são eliminadas. Após a confirmação (ou no primeiro login com Google), o usuário passa a operar com um token de acesso JWT, válido por 24 horas, e um token de renovação, válido por 7 dias, assinados com segredos distintos. As rotas de dados do usuário exigem o token, e cada usuário acessa apenas os próprios

registros. As senhas são armazenadas com bcrypt, e as rotas sensíveis têm limites por IP (RN04). A exclusão da conta remove os dados definitivamente, conforme a Seção 4.7.4. A Figura 11 representa esses estados de acesso e suas transições. Quem esquece a senha recebe por e-mail um link de uso único, válido por 60 minutos, com a mesma resposta exista a conta ou não; a troca de senha, pelo link ou na conta logada, encerra as demais sessões, o que invalida um token de renovação eventualmente vazado. Desde 03/11/2026, o limite por IP usa o endereço que o balanceador acrescenta ao final do cabeçalho X-Forwarded-For, e não o primeiro valor, que o cliente controla (achado S07 do teste de penetração, Seção 5.5.3.2).

Figura 11 – Diagrama de estados do controle de acesso



Fonte: elaborado pelos autores (2026).

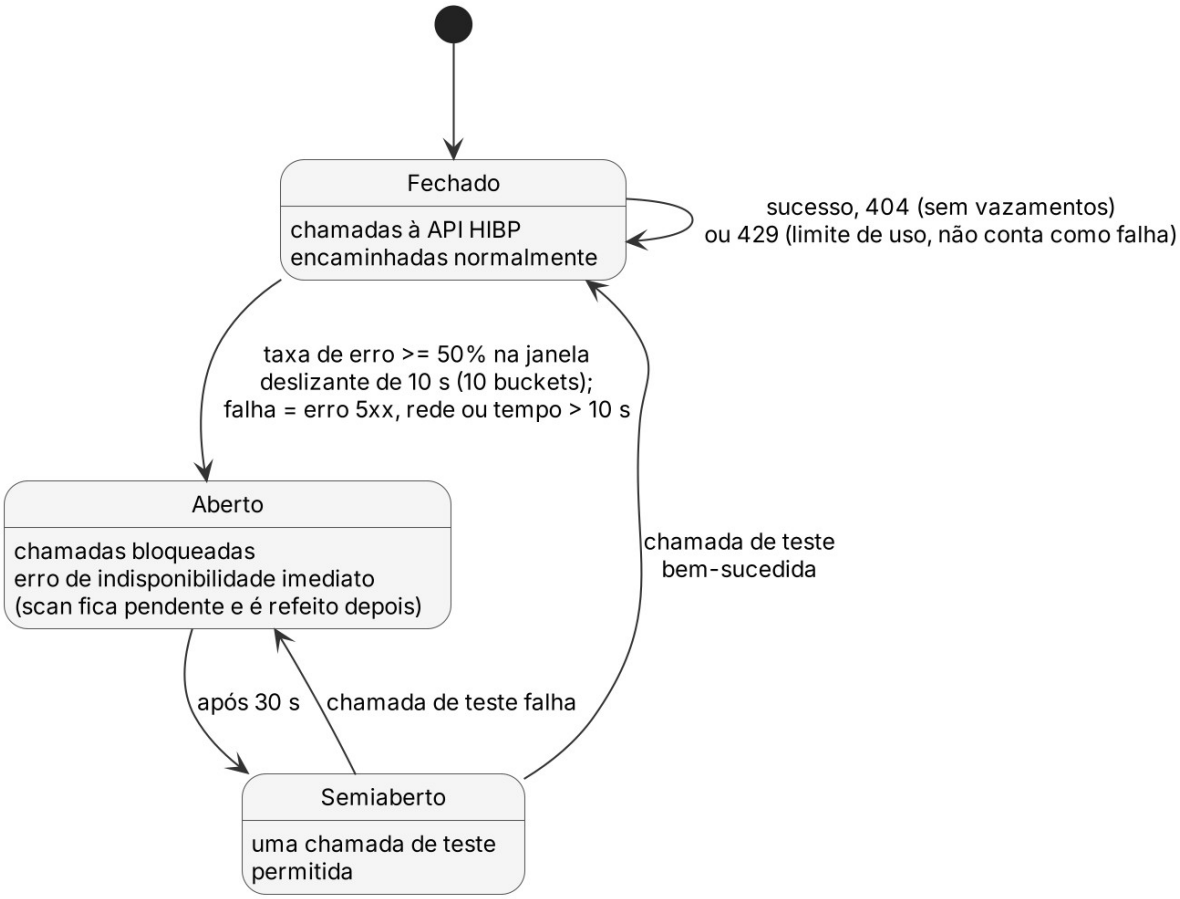
Especificação do Circuit Breaker (referência única para as Seções 4.1, 4.2 e 4.6.4). No pilar da disponibilidade, o disjuntor implementado com Opossum protege o sistema contra falhas em cascata provocadas por instabilidades da HIBP (RNF04), com os seguintes parâmetros, definidos em HibpClient:

- **Janela de observação:** deslizante, de 10 segundos, dividida em 10 intervalos de 1 segundo; a taxa de erro é calculada sobre as chamadas dessa janela.
- **Limiar de abertura:** taxa de erro igual ou superior a 50%. Contam como falha os erros 5xx, as falhas de rede e as chamadas que excedem 10 segundos (tempo-limite da

requisição HTTP e do próprio disjuntor). Respostas 429 (limite de uso) não contam, porque indicam uso acima do plano, e não falha do serviço, e respostas 404 significam ausência de vazamentos.

- **Número mínimo de requisições:** não configurado (padrão da biblioteca, zero), de modo que uma única falha numa janela sem outras chamadas já abre o circuito. Como as chamadas à HIBP são espaçadas pelo limitador, esse comportamento é conservador; a definição de um mínimo de chamadas está registrada como ajuste futuro.
- **Estado aberto:** as chamadas falham imediatamente com erro de indisponibilidade; o scan fica pendente e é refeito pela reconciliação de hora em hora.
- **Estado semiaberto:** após 30 segundos, uma chamada de teste é liberada; o sucesso fecha o circuito e a falha o reabre (Figura 12).
- **Escopo:** o estado do disjuntor é mantido em memória, por processo. Com mais de uma instância, cada uma teria o próprio disjuntor; o limite de requisições, por sua vez, já é global (Seção 4.6.3). Essa limitação está registrada na Seção 5.7.

Figura 12 – Diagrama de estados do *Circuit Breaker* de integração externa



Fonte: elaborado pelos autores (2026).

4.7.3 Registro de Eventos (Logs) e Auditoria

O SafeID adota registro de eventos (logging) estruturado: cada evento é uma linha JSON com data, nível, contexto, nome do evento e campos adicionais, gerada por um logger próprio (server/src/shared/utils/logger.ts), sem biblioteca externa. Em produção, os logs vão para o Amazon CloudWatch, com retenção de 30 dias; filtros de métrica sobre esses eventos alimentam alarmes, por exemplo, para sincronização do catálogo parada, chave da HIBP inválida ou falhas no envio de e-mails, notificados à equipe por e-mail. Trata-se de uma base de observabilidade inicial, e não de uma camada madura: não há métricas de aplicação dedicadas, rastreamento distribuído nem painel centralizado, itens registrados como evolução (Seção 6.3).

Os registros são categorizados por níveis de severidade (debug, info, warn e error), com nível mínimo configurável. A proteção dos dados pessoais nos logs é feita por convenção de código, e não por um interceptador automático: os eventos registram o identificador numérico do usuário (userId) e, quando é preciso referir-se a um e-mail, apenas os 8 primeiros caracteres do seu SHA-256 (função maskEmail), nunca o endereço em claro. Ainda há mensagens de depuração legadas (console.log) no caso de uso que registram o HMAC do e-mail, completo ou truncado, e não o endereço; sua migração para o logger estruturado está pendente.

Correlação de eventos em uma investigação. O identificador de correlação persistente é o userId, que liga logs, histórico (ScanHistory), vazamentos conhecidos e notificações sem expor o e-mail. Para investigar um titular a partir do endereço informado por ele, a equipe localiza o userId na tabela de usuários, com acesso restrito ao banco pelo bastion, e filtra os eventos por esse campo; o maskEmail permite conferir eventos anteriores ao cadastro. Como o maskEmail usa SHA-256 sem chave, um prefixo de 8 caracteres pode ser associado a um e-mail conhecido por quem tiver acesso aos logs; a troca por um HMAC truncado está registrada como ajuste futuro. O acesso aos logs é restrito à conta AWS da equipe.

4.7.4 Observância à Legislação

O SafeID foi projetado em alinhamento com as diretrizes da Lei Geral de Proteção de Dados Pessoais (LGPD — Lei nº 13.709/2018; BRASIL, 2018). Os controles técnicos descritos apoiam a conformidade, mas não constituem, isoladamente, comprovação jurídica integral, que também depende de governança, processos, bases legais e avaliação

especializada. A abordagem considera os princípios do art. 6º e o direito do titular descritos a seguir:

- **Princípio da Transparência e Livre Acesso:** muitas vezes, os relatórios de vazamentos corporativos utilizam termos estritamente técnicos que impedem a real compreensão por parte do cidadão leigo. O SafeID foi projetado em alinhamento com o princípio da transparência ao utilizar o modelo de linguagem natural para traduzir metadados complexos em planos de ação claros e acessíveis, com o objetivo de permitir ao titular entender com precisão o impacto real da exposição de seus dados.
- **Princípio da Necessidade e Minimização de Dados:** o sistema opera sob o escopo estrito da minimização, coletando e processando apenas os dados estritamente necessários para a execução de sua finalidade (a verificação de vulnerabilidades cibernéticas). Não há captação secundária de informações de navegação, perfis de consumo ou enriquecimento ilícito de bases de dados. A exigência de e-mail verificado impede o tratamento de e-mails de terceiros.
- **Princípio da Segurança e Prevenção:** a combinação de criptografia em nível de aplicação (AES-256-GCM), barramento de validações via *Class-validator* (NestJS) e isolamento de redes internas adota controles voltados a atender à exigência legal de adotar medidas técnicas aptas a proteger os dados pessoais de acessos não autorizados e de situações acidentais ou ilícitas de destruição, perda ou alteração.
- **Direito à eliminação dos dados (art. 18, VI):** a eliminação não é um princípio do art. 6º, mas um direito do titular, previsto no art. 18, VI, para os dados tratados com o seu consentimento. O SafeID o atende pela exclusão da conta no próprio painel: o back-end remove o usuário com exclusão física em cascata do histórico, dos vazamentos conhecidos, das notificações e dos tokens, e apaga o resultado guardado no cache do Redis. O titular também pode revogar o consentimento do monitoramento a qualquer momento (art. 18, IX), pelas preferências ou pelo link de descadastramento dos e-mails, observadas as hipóteses legais de conservação.
- Direitos de acesso e portabilidade (art. 18, II e V): nas configurações da conta, o titular baixa um arquivo JSON com seus dados de cadastro, análises (com os dados de incidentes decifrados), vazamentos conhecidos e notificações, sem depender de pedido manual à equipe.
- **Princípio da Responsabilização e Prestação de Contas (art. 6º, X):** o projeto registra as medidas que adota para demonstrar conformidade: o aceite da Política de Privacidade e o consentimento do monitoramento são gravados com data no cadastro

(termsAcceptedAt e monitoringConsentAt); a Política de Privacidade e os Termos de Uso estão publicados na aplicação e no repositório (PRIVACY.md e TERMS.md), o cadastro exige o aceite dos dois textos por maiores de 18 anos e grava a versão aceita (termsVersion); as decisões de segurança e as mudanças no código são rastreáveis por Pull Requests revisados; e os alarmes de operação registram falhas relevantes. O relatório de impacto à proteção de dados e o plano de resposta a incidentes envolvendo o próprio SafeID foram concluídos e aprovados pelo Product Owner em 04/11/2026 e estão publicados em docs/lgpd no repositório; a equipe se declara agente de tratamento de pequeno porte, o que conta em dobro os prazos de comunicação à ANPD.

5 IMPLEMENTAÇÃO

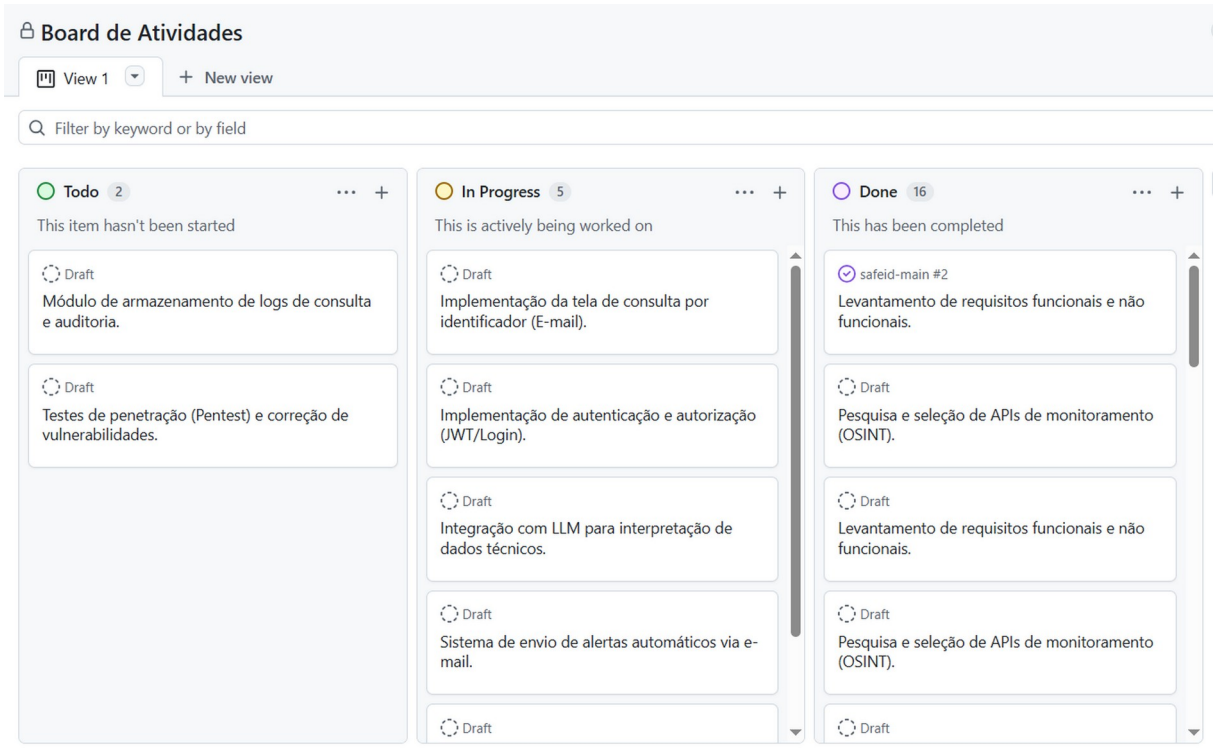
Este capítulo apresenta a materialização da solução projetada em *software*, descrevendo o ambiente utilizado, a organização da aplicação, a implementação dos componentes, sua integração e os procedimentos de teste. Sua função é demonstrar como o projeto definido nos capítulos anteriores foi transformado em uma solução executável, na PI-1 e na PI-2, e quais resultados foram obtidos.

5.1 Ambiente de Desenvolvimento

A gestão de configuração e versão do código-fonte é realizada por meio do **Git** como sistema de controle de versão distribuído, utilizando o **GitHub** como plataforma de hospedagem. Essa escolha justifica-se pela facilidade de colaboração entre a equipe e pela integração contínua (CI/CD) com ferramentas de implantação. O projeto adota o modelo de **monorepo**, centralizando dados, documentação e código-fonte em um único repositório gerenciado em uma organização no GitHub, disponível em https://github.com/Safe-ID/safeid-main. O repositório possui um arquivo README.md na raiz, que orienta a navegação e o processo de instalação, e as entregas de cada *sprint* são evidenciadas pelo histórico de *commits* e pelos registros de atividades dos membros, permitindo a auditoria das contribuições individuais e validando a evolução do MVP.

O *Product Backlog*, o planejamento das *Sprints* e o acompanhamento das atividades são gerenciados no **GitHub Projects**, integrado ao repositório. A ferramenta organiza os itens por estado, prioridade, responsável, fase e ciclo de entrega, além de relacionar tarefas a *issues*, *pull requests* e *commits*. Essa vinculação contribui para a rastreabilidade entre requisitos, responsabilidades, implementação, revisão e entrega. A Figura 13 apresenta um recorte ampliado do quadro de atividades, capturado durante a PI-1. No momento da captura, o quadro tinha 23 itens: 2 em “Todo” (armazenamento de logs de auditoria e testes de penetração), 5 em “In Progress” (entre eles a tela de consulta, a autenticação, a integração com a LLM e os alertas por e-mail) e 16 em “Done”. A situação atual de cada item do backlog está no Quadro 9 (Apêndice A), e a metodologia de gestão adotada é detalhada no mesmo apêndice.

Figura 13 – Quadro de atividades do SafeID no GitHub Projects (recorte)



Fonte: elaborado pelos autores (2026). Disponível em: https://github.com/orgs/Safe-ID/projects/1/views/1.

O ambiente de desenvolvimento e execução é composto pelas seguintes ferramentas e configurações:

- **Linguagens e plataformas:** TypeScript em modo estrito sobre Node.js 20 no *back-end*; JavaScript (JSX) com React 19, Vite e Tailwind CSS no front-end; NestJS e Prisma ORM no servidor;
- **Serviços de dados:** PostgreSQL e Redis, provisionados localmente por Docker Compose; no pipeline de integração contínua, os testes de integração e E2E usam um contêiner PostgreSQL efêmero, e o Redis e a HIBP são simulados;
- **Empacotamento e implantação:** contêineres Docker com *Multi-stage Build*, pipeline de CI no GitHub Actions e implantação na AWS por scripts versionados (deploy.ps1), com rollback automático do ECS;
- **Prototipação:** Figma, para o protótipo de alta fidelidade da interface;
- **Qualidade:** Jest no back-end (unitários, integração e E2E da API), Vitest com React Testing Library no front-end e fast-check para testes baseados em propriedades (fuzzing) do Motor de Risco, além das ferramentas de análise estática e de segurança descritas a seguir. O Grafana k6, o Playwright e o OWASP ZAP foram usados, respectivamente, nos testes de carga, de jornada no navegador e de segurança (Seção 5.5.3), com scripts versionados em tests/load, e2e e tests/security.

A análise estática atua como a primeira linha de defesa no *pipeline* de qualidade, avaliando o código-fonte sem a necessidade de executá-lo. O SafeID combina ferramentas de qualidade de código e de segurança:

1. **Compilação estrita do TypeScript:** o compilador é configurado em modo estrito (*strict: true*), o que mitiga em tempo de desenvolvimento erros comuns de tipagem, referências nulas (*null pointers*) e comportamentos indefinidos em tempo de execução.
2. **ESLint:** utilizado para aplicar regras estritas de padronização de código (*code linting*), identificar padrões inadequados de programação, variáveis não utilizadas e potenciais falhas de lógica antes da geração do artefato final (*build*).
3. **Prettier:** contribui para a formatação automática unificada do código entre os membros da equipe, reduzindo conflitos de estilo e aumentando a legibilidade e a manutenibilidade do repositório a longo prazo.
4. **Análise estática de segurança (SAST):** o CodeQL, do GitHub, analisa o código JavaScript e TypeScript a cada Pull Request, a cada push na main e semanalmente, procurando padrões de vulnerabilidade, como injeção e uso inseguro de dados de entrada.
5. **Análise de composição de software (SCA):** o Dependabot verifica semanalmente as dependências do back-end e do front-end e abre Pull Requests de atualização, inclusive de segurança, e o Dependency Review bloqueia Pull Requests que introduzam dependência com vulnerabilidade conhecida de severidade alta ou crítica.
6. **Proteção da branch principal:** a main exige Pull Request revisado por outro desenvolvedor e aprovação de todos os checks obrigatórios do CI antes da integração.

A padronização e a uniformidade estilística do código-fonte constituem premissas essenciais para viabilizar a manutenibilidade de longo prazo e mitigar o endividamento técnico (*technical debt*). O SafeID estabelece uma convenção de código (*code convention*) baseada nas diretrizes consolidadas para ecossistemas de TypeScript e React.js, auditada automaticamente pelo ESLint e formatada pelo Prettier, com regras como:

- uso obrigatório de *camelCase* para nomenclatura de funções e variáveis, e *PascalCase* para componentes funcionais do React.js;
- desestímulo ao uso do tipo genérico any no TypeScript: o ESLint emite aviso a cada ocorrência, e alguns pontos de integração ainda o utilizam, como os parâmetros de cache e fila do caso de uso;

- organização de pastas em camadas inspirada na Clean Architecture, com núcleo de regras (core), infraestrutura (infra) e módulos de entrada (modules), conforme a Seção 4.2.

Essa uniformidade contribui para que o código-fonte possua alta legibilidade, facilitando a integração de novos desenvolvedores e assegurando que o projeto atenda aos critérios de engenharia e sustentabilidade tecnológica demandados nos âmbitos acadêmico e de mercado.

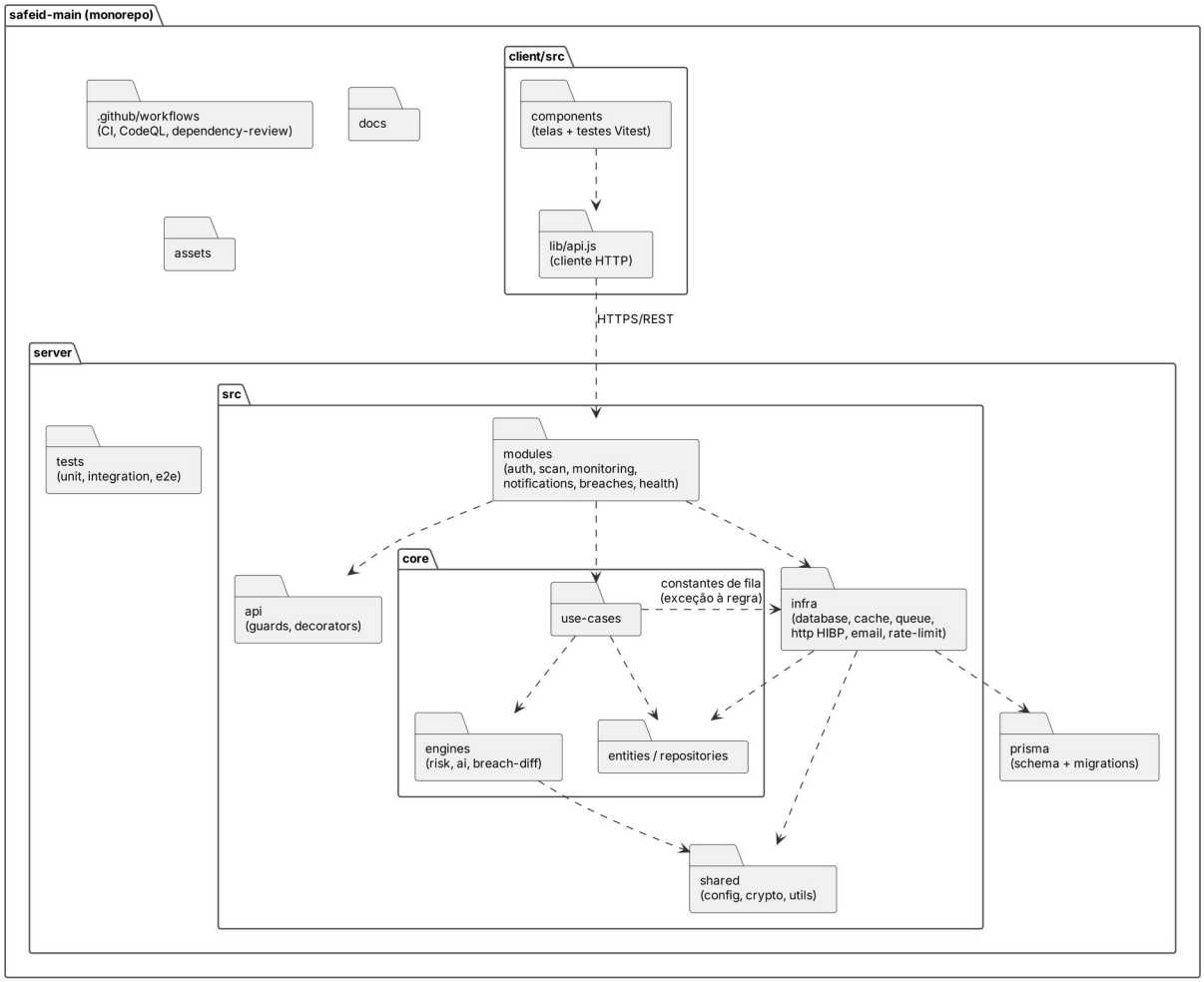
5.2 Estrutura da Aplicação

O monorepo do SafeID organiza-se nos seguintes diretórios:

- **/client:** código-fonte da interface web desenvolvida em React.js;
- **/server:** lógica de negócio, integração com APIs OSINT e motor de IA em Node.js;
- **/database:** material de apoio à modelagem (exemplos de esquemas e a ferramenta brModelo); as migrations do SafeID ficam em /server/prisma/migrations, geradas pelo Prisma;
- **/docs:** documentação oficial do projeto (incluindo os arquivos Word e PDF) e registros de requisitos;
- **/assets:** diagramas de arquitetura, logotipos e recursos visuais do sistema.

No diretório do servidor, o código segue as camadas descritas na Seção 4.2: modules (módulos NestJS de autenticação, consulta, monitoramento, notificações, vazamentos e saúde, com controladores, DTOs e serviços), core (caso de uso ExecuteRiskScanUseCase, motores de risco e de IA, entidades e repositórios abstratos), infra (banco, cache, filas, cliente HIBP, e-mail e limite de requisições) e shared (configuração, criptografia e utilitários). As dependências apontam das camadas externas para o núcleo, com as duas exceções registradas na Seção 4.2. A Figura 14 apresenta o diagrama de pacotes correspondente.

Figura 14 – Diagrama de pacotes da aplicação



Fonte: elaborado pelos autores (2026).

5.3 Implementação dos Componentes

Na PI-1, foram implementados os componentes do fluxo de consulta e diagnóstico, correspondentes aos itens 01 a 12 do *backlog* (Apêndice A); na PI-2, até a data desta versão, foram implementados os itens 13 a 15, 17 e 19, além da implantação na AWS. Os aspectos técnicos mais relevantes são descritos a seguir.

- **Motor de Risco:** implementado como função de domínio pura, aplica a tabela de pesos (Tabela 1), o fator temporal em degrau (RN03), a soma ponderada, a normalização com e a classificação nas faixas LOW, MODERATE, HIGH e CRITICAL. O valor contínuo é arredondado ao inteiro mais próximo antes da classificação, de modo que a faixa sempre corresponde ao valor persistido no campo riskScore (RN08); os subscores de cada incidente, com duas casas decimais, ficam disponíveis para auditoria.

- **Consulta assíncrona à fonte externa:** não há rota pública de consulta; o ScanService aciona o caso de uso na confirmação do e-mail, no monitoramento e nas novas tentativas. O caso de uso consulta o cache e, quando necessário, enfileira um job na fila hibp-check, aguardando sua conclusão por até 20 s. O worker aplica o limite do plano da HIBP de forma global (Seção 4.6.3).
- **Resiliência:** o HibpClient encapsula o disjuntor implementado com Opossum, com os parâmetros especificados na Seção 4.7.2, e o tratamento dos códigos HTTP da HIBP.
- **Proteção de dados:** os identificadores são pseudonimizados com HMAC-SHA256 com chave secreta antes do uso como chave de *cache e no histórico*; os dados de incidentes são cifrados com AES-256-GCM na aplicação antes da gravação no PostgreSQL; e os *logs* usam userId ou maskEmail no lugar do e-mail (Seções 4.7.1 e 4.7.3).
- **Autenticação e histórico:** o AuthService implementa cadastro com aceite da política, verificação de e-mail, login com senha ou Google, renovação de tokens e exclusão de conta; o ScanService registra o histórico, o resultado exibido no painel e os vazamentos já conhecidos.
- **Interface:** as telas de cadastro e login e o *painel* com o termômetro de risco, o plano de ação, o histórico e as notificações foram desenvolvidos em React e Tailwind CSS, com estados de carregamento e de contingência.
- **Assistente cognitivo:** o AIEngine envia ao modelo de linguagem, no Azure AI Foundry, apenas metadados dos incidentes, com instruções fixas, temperatura 0,3, saída em JSON e tempo-limite de 10 s; valida a resposta e, em caso de falha, usa uma resposta padrão determinística. As recomendações válidas ficam em cache por 7 dias (Seção 4.6.4).
- **Monitoramento e alertas:** jobs agendados do BullMQ sincronizam o catálogo da HIBP a cada 6 horas, verificam de hora em hora se surgiu vazamento novo, conferem os usuários monitorados, criam notificações sem duplicidade e enviam os alertas pelo Amazon SES, com descadastramento em um clique; uma rotina de reconciliação recupera, de hora em hora, o que se perdeu no Redis e refaz consultas que falharam.

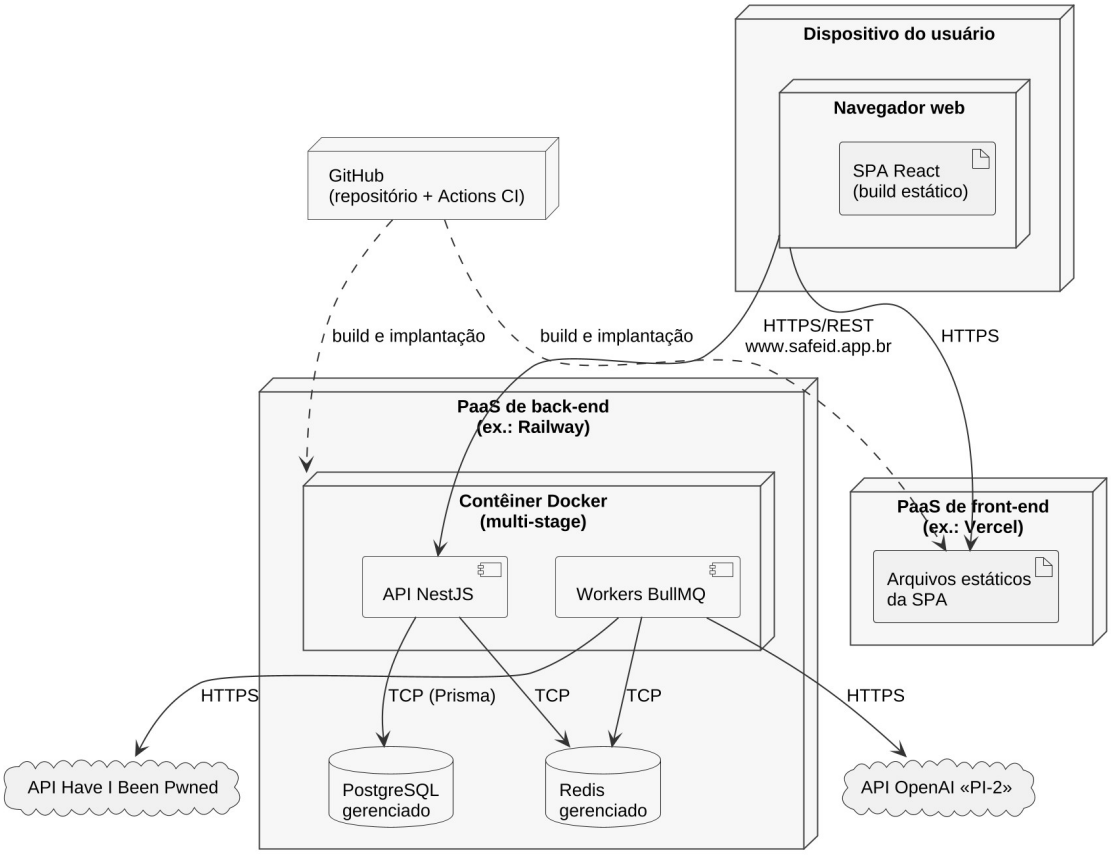
5.4 Integração dos Módulos

A integração dos módulos ocorre no interior de um único processo NestJS (monólito modular), no qual os módulos se comunicam por injeção de dependência, enquanto as consultas à HIBP, as varreduras e os e-mails são distribuídos por filas assíncronas geridas por

Redis e BullMQ. O front-end comunica-se com o back-end exclusivamente por HTTPS/REST, e o back-end comunica-se com o PostgreSQL por meio do Prisma ORM e com o Redis para cache, filas e limite de requisições. O desacoplamento por filas protege a requisição do usuário da latência e dos limites da fonte externa, mas não isola falhas de processo: em produção, a API HTTP e os workers rodam no mesmo processo Node.js (APP_ROLE=all), em uma única task, de modo que uma falha desse processo interrompe simultaneamente o atendimento e o processamento. O ECS recria a task em 1 a 2 minutos, e os jobs pendentes são retomados a partir do Redis, mas esse ponto único de falha está registrado na Seção 5.7. O código já permite separar os papéis sem alteração (APP_ROLE=api e APP_ROLE=worker), em services ECS distintas.

Na fase de MVP da PI-1, a equipe adotou uma estratégia de implantação incremental e financeiramente sustentável: em vez de provisionar imediatamente os recursos da AWS, optou-se pelo empacotamento agnóstico via contêineres Docker Multi-stage e pela hospedagem temporária em plataformas PaaS (Railway e Vercel), o que deu agilidade ao ciclo de desenvolvimento a custo reduzido. A Figura 15 registra essa implantação da PI-1.

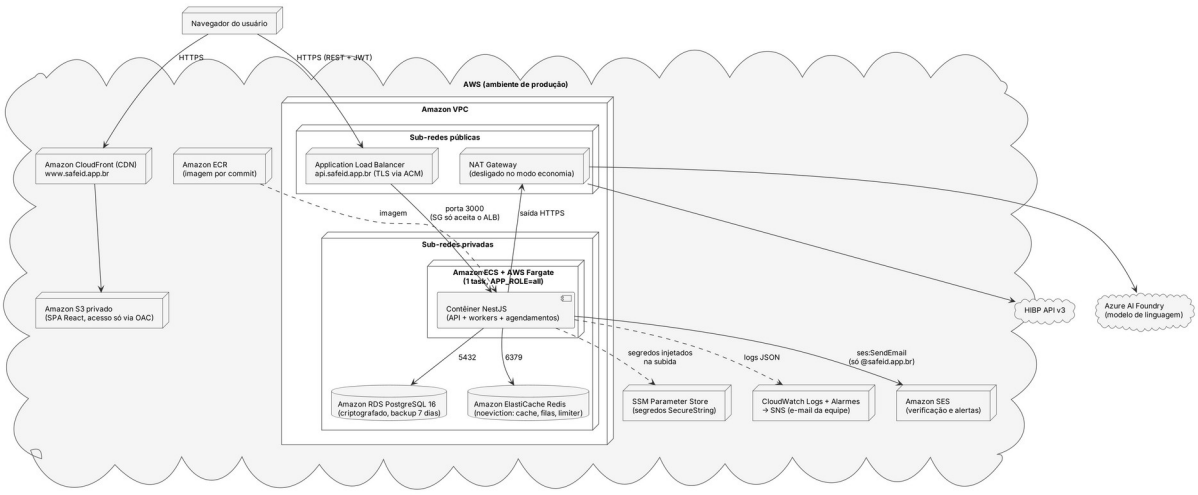
Figura 15 – Diagrama de implantação do MVP da PI-1 (PaaS)



Fonte: elaborado pelos autores (2026).

Como o código e o banco de dados foram estruturados de forma portável, a migração para a AWS foi feita durante a PI-2, sem alterações no código da aplicação, e a aplicação está disponível em https://www.safeid.app.br (front-end no CloudFront) e https://api.safeid.app.br (API no ALB). A migração seguiu um roteiro registrado no repositório (DEPLOY_PRODUCAO.md), com critérios de aceite e estratégia de rollback que a Seção 4.6.5 resume: (i) preparação única, em ordem: rede, NAT, banco, Redis, certificados, SES, segredos no SSM, ECS e front-end; (ii) critérios de aceite: health check do ALB respondendo com banco e Redis disponíveis, agendamentos com próximo disparo definido e sincronização recente do catálogo; (iii) rollback: o ECS usa deployment circuit breaker, que volta sozinho à versão anterior se a nova não ficar saudável, e qualquer imagem anterior, identificada pelo commit, pode ser reimplantada; antes de cada migration do banco, é feito um snapshot manual. A Figura 16 representa a implantação atual descrita na Seção 4.6.5.

Figura 16 – Diagrama de implantação da arquitetura em produção na AWS



Fonte: elaborado pelos autores (2026).

5.5 Testes Realizados

A garantia da qualidade, da resiliência e da manutenibilidade de um ecossistema focado em cibersegurança e privacidade de dados como o SafeID exige uma estratégia rigorosa de testes automatizados. Em conformidade com a separação em *camadas (Seção 4.2)*, o isolamento da lógica de negócio em relação às dependências externas de infraestrutura permite criar uma suíte de testes previsível, rápida e desacoplada. Esta seção detalha o planejamento e a execução das diferentes camadas de validação que mitigam o risco de regressões de código, falhas de segurança e indisponibilidades sistêmicas.

5.5.1 Plano de Testes

O Plano de Testes do SafeID fundamenta-se no conceito da **Pirâmide de Testes**, priorizando uma base ampla de testes unitários rápidos e de baixo custo computacional, seguidos por testes de componente e de integração estruturados e por uma camada enxuta de testes de ponta a ponta (*End-to-End*).

A meta de qualidade estabelecida para o projeto estipula cobertura mínima (*code coverage*) de 80% de *statements*, *branches* e *lines* e de 85% de *functions* para permitir a integração de novas funcionalidades. Na medição de 04/11/2026, a suíte unitária do *back-end* alcançou 98,98% de *statements*, 90,9% de *branches*, 97,18% de *functions* e 99,13% de *lines*, atingindo todas as metas, conforme detalhado na Seção 5.5.4.2. A execução do plano ocorre de maneira substancialmente automatizada por meio do *pipeline* de Integração Contínua (CI) no **GitHub Actions**. Sempre que um desenvolvedor submete um *Pull Request* direcionado à ramificação principal (*main*), o ambiente de nuvem isola a aplicação, provisiona um contêiner PostgreSQL efêmero e executa, em sequência, lint, build e testes unitários do back-end, testes de integração e E2E da API e lint, testes e build do front-end; o CodeQL e o Dependency Review rodam em paralelo. Caso qualquer teste falhe ou algum limite de cobertura configurado no Jest não seja cumprido, a integração é bloqueada, impedindo que códigos defeituosos alcancem o ambiente produtivo.

5.5.2 Testes Funcionais

Os testes funcionais verificam se o comportamento do SafeID atende de forma sistemática aos requisitos de negócio estabelecidos nas histórias de usuário e em seus critérios de aceite.

5.5.2.1 Testes unitários

Os testes unitários têm como escopo a validação isolada das menores unidades de código lógico do sistema, especificamente as entidades e os casos de uso (*use cases*) do *back-end* localizados no domínio central. Utilizando *o* **Jest**, todas as dependências externas — como consultas ao PostgreSQL ou requisições HTTP para as APIs externas — são substituídas por objetos simulados (*mocks*).

O principal foco dos testes unitários é o **Motor de Risco** (*Risk Engine*). Para protegê-lo, foram implementados testes baseados em propriedades (fuzzing) com a biblioteca fast-check, que geram automaticamente milhares de conjuntos de incidentes, datas e nomes de classes, inclusive malformados, e verificam invariantes do modelo: o score é sempre inteiro

entre 0 e 100, a faixa obedece à RN08, o resultado não depende da ordem dos incidentes nem da grafia das classes, e expor uma classe a mais ou um incidente a mais nunca reduz o score. Esses testes encontraram um defeito real: nomes de classes como “__proto__” acessavam propriedades herdadas e tornavam o score inválido, o que levou à troca da estrutura de pesos por um Map.

5.5.2.2 Testes de componente

Focados na camada de apresentação, os testes de componente usam o Vitest com a React Testing Library e simulam a interação do usuário com os componentes, sem navegador real. As seis suítes atuais cobrem o cadastro e o login (envio do consentimento de alertas, tela de confirmação de e-mail e mensagens de erro), o sino de notificações, o painel (tradução da classificação para o português, aviso de que o scan que falhou é refeito sozinho, ausência de botão para refazer o scan), o termômetro de risco no estado pendente, as configurações da conta (troca de senha, cópia dos dados e exclusão), a criação de nova senha pelo link do e-mail e as rotas da aplicação (textos legais e página não encontrada). Esses testes verificam texto e comportamento visíveis ao usuário, e não apenas classes CSS.

Limitação reconhecida: os testes de componente não detectam regressões visuais (diferenças de layout ou de cor) nem verificam automaticamente a acessibilidade, como atributos ARIA e contraste. Para complementar essa camada, estão previstos testes de regressão visual por captura de tela e verificação automática de acessibilidade com a biblioteca axe-core (Seção 6.3).

5.5.2.3 Testes de integração

Os testes de integração validam a comunicação e o contrato entre múltiplos componentes e subsistemas que dependem uns dos outros para executar um fluxo de trabalho completo. No SafeID, essa camada valida a consistência de dados entre as rotas da API NestJS, o mapeamento do Prisma ORM e os serviços de infraestrutura. Durante a execução, o pipeline de CI levanta um contêiner PostgreSQL real, aplica as migrations e executa os testes com o Jest; o Redis e a HIBP são substituídos por simulações. Os testes de integração verificam, por exemplo, que:

- o ScanService submeta uma consulta e persista o snapshot e o histórico, e que o histórico e o detalhe sejam recuperados pela camada de persistência;
- o fluxo de monitoramento defina a base de vazamentos conhecidos sem notificar e, quando um vazamento novo é publicado, notifique uma única vez;

- a notificação seja recuperada quando o processamento falha depois de registrar o vazamento.

A criptografia AES-256-GCM e os estados do Circuit Breaker, inclusive a não abertura do circuito por respostas 429, são verificados por testes unitários dedicados.

5.5.2.4 Testes de ponta a ponta (E2E)

No estado atual, os testes de ponta a ponta exercitam a API completa por HTTP, com a aplicação NestJS em execução e o banco real do CI, sem navegador. Eles cobrem o cadastro com verificação do e-mail pelo link até o acesso a /auth/me, a renovação de sessão, o bloqueio do login antes da verificação, a exclusão da conta, o limite de tentativas de login por IP (inclusive atrás de proxy), as notificações e preferências de monitoramento, o descadastramento em um clique, a verificação de saúde e as rotas de histórico, além de garantir que não existe rota pública de consulta. Desde 03/11/2026, a jornada completa também é verificada no navegador com o Playwright (PLAYWRIGHT, 2026): oito testes cobrem do cadastro à exclusão da conta, passando pelo painel, pelo plano de ação, pela exportação em PDF, pela cópia dos dados e pela troca e recuperação de senha, além das páginas públicas, dos textos legais, da página não encontrada, da ausência de rolagem horizontal e de erros de Content-Security-Policy. Na campanha de 03/11, as 32 execuções passaram nos quatro ambientes (Chromium, Firefox, WebKit e Chromium com emulação de celular). A suíte roda em uma cópia local, com HIBP, IA e e-mail simulados, e ainda não faz parte do CI, porque depende dessa cópia no ar (relatório em docs/testes/relatorio-e2e-playwright.pdf).

5.5.3 Testes Não Funcionais

Os testes não funcionais do SafeID tratam de desempenho, capacidade, estabilidade, resiliência e segurança. A resiliência é verificada por testes unitários do disjuntor; o desempenho e a carga, por uma campanha com o Grafana k6 (GRAFANA LABS, 2026) em produção; e a segurança, por um teste de penetração automatizado com o OWASP ZAP (ZAP, 2026) e scripts próprios, ambos executados em 03/11/2026. Os scripts estão no repositório (tests/load e tests/security), e os relatórios e dados brutos, em docs/testes.

A abordagem segue as características de qualidade da norma ABNT NBR ISO/IEC 25010 (ABNT, 2011), publicada com base nas normas da Organização Internacional para Padronização (ISO) e da Comissão Eletrotécnica Internacional (IEC). Também considera as práticas de segurança do NIST (2025), as orientações da Autoridade Nacional de Proteção de

Dados (ANPD, 2022), as diretrizes de governo digital da Secretaria de Governo Digital (SGD) e, como referência comparativa internacional, o arcabouço de proteção de dados da União Europeia (UE).

5.5.3.1 Estratégia integrada de validação

No nível unitário, o Jest (JEST, 2026) valida as funções do Motor de Risco, os fatores temporais, os limites de normalização, o tratamento de valores ausentes e as transições de estado do disjuntor. Nos componentes React, o Vitest com a React Testing Library verifica renderização, estados de carregamento e mensagens de falha. Na integração e no E2E, o Jest exercita os serviços e os controladores NestJS por HTTP, com validação de entrada, autenticação, persistência e respostas de contingência. No navegador, o Playwright percorre a jornada completa (Seção 5.5.2.4), e o Grafana k6 mede os cenários de linha de base, carga progressiva, pico e recuperação.

5.5.3.2 Validação de desempenho, carga, resiliência e segurança

A campanha de carga, executada em 03/11/2026 contra a produção (commit fbc4e58), seguiu o plano: aquecimento, patamar de 100 usuários virtuais por 5 minutos, pico de 300 e recuperação, repetidos três vezes, com publicação da mediana e da amplitude do P95. Foram 204.487 requisições, com 0,00% de erro, nenhum erro 5xx e nenhum alarme disparado. Duas observações orientam a leitura da Tabela 2. Primeiro, a carga incidiu sobre GET /api/health, que consulta o banco e o Redis, e sobre a página inicial: com a retirada da consulta manual, não existe rota pública de consulta, o Motor de Risco não tem rota própria e o enfileiramento só ocorre no cadastro e no monitoramento, de modo que as metas desses três cenários não são mensuráveis por carga externa e permanecem verificadas por testes automatizados. Segundo, a meta de vazão de 500 req/s não é alcançável com o roteiro de um acesso por segundo por usuário virtual: a vazão observada foi de cerca de 90 req/s no patamar e de 233 req/s no pico.

Tabela 2 – Metas e resultados de desempenho, carga e resiliência (k6 e Opossum)

Componente / cenário	**Meta**	**Resultado (mediana do P95 em 3 execuções)**	**Situação**
Linha de base e patamar de até 100 usuários virtuais	P95 < 750 ms; erro < 2%	144 a 145 ms; 0,00% de erro; até 90 req/s	Atingida
Pico de 300 usuários virtuais	P95 < 1,5 s; erro < 2%	216 ms; 0,00% de erro; 233 req/s	Atingida
Recuperação (20 usuários virtuais)	Retorno ao patamar da linha de base	144 ms; 0,00% de erro	Atingida
Consulta com acerto no cache Redis	P95 < 50 ms; ≥ 500 req/s	Sem rota pública de consulta	Não mensurável por carga externa
Motor de Risco sem chamada	P95 < 100 ms; ≥ 250 req/s	O motor não tem rota própria	Coberto por testes

Componente / cenário	Meta	Resultado (mediana do P95 em 3 execuções)	Situação
externa			unitários
API com fila BullMQ (enfileiramento)	P95 < 750 ms para aceitar e enfileirar	Enfileiramento só no cadastro e no monitoramento	Coberto por testes de integração
Consulta nova à HIBP	Limitada pelo plano (≈ 9 por minuto no plano de entrada); concluída ou pendente em até 20 s	Plano Core 1; limitador global e tempo-limite do job	Verificado em código
Falha da fonte externa (Opossum)	Abrir com ≥ 50% de erro; falha imediata com circuito aberto; semiabrir após 30 s	Testes unitários do HibpClient	Comportamento verificado

Fonte: elaborado pelos autores (2026).

Na AWS, a CPU do contêiner atingiu 73% no pico, a do RDS, 7%, e a do Redis, 1,35%, e o tempo médio de resposta do alvo no balanceador foi de 5 ms; a maior parte dos cerca de 140 ms medidos pelo gerador corresponde à rede entre o Brasil e a região us-east-2. Com uma única task e sem escalonamento automático, a margem para rotas mais pesadas é pequena. Como limitações, as rotas autenticadas, com descriptografia e IA, não foram medidas, porque respondem 429 após 100 requisições a cada 15 minutos por IP; todo o tráfego partiu de um único computador; e não houve teste de longa duração nem busca do ponto de ruptura (relatório em docs/testes/relatorio-carga-k6.pdf).

Segurança. O teste de penetração de 03/11/2026, autorizado pela equipe e restrito aos domínios e à infraestrutura do projeto, combinou varreduras passivas do OWASP ZAP e verificações dirigidas em produção com varredura ativa, testes de autorização entre usuários e fuzzing de entrada em uma cópia local descartável. Foram aprovadas 50 de 50 verificações em produção (transporte, cabeçalhos, CORS, autenticação com JWT forjado, injeção SQL e NoSQL, exposição de arquivos, enumeração de contas e limite de tentativas) e 11 de 11 testes de autorização, e o fuzzing enviou 400 requisições com 22 tipos de carga maliciosa sem provocar nenhum erro 5xx. Os dois achados acionáveis foram corrigidos e reconfirmados em produção no mesmo dia: S01, seis dependências de severidade alta no back-end (PR #123), e S07, de severidade média, o limite de tentativas de login contornável por um cabeçalho X-Forwarded-For forjado (PR #127). Os demais achados foram aceitos ou ficaram como recomendação de baixo risco, por exemplo, o style-src com unsafe-inline na política de segurança de conteúdo, exigido pelo React e pelo Tailwind, e o Redis sem TLS na rede privada; o critério de nenhum achado alto ou crítico em aberto foi, portanto, cumprido. O teste é automatizado e conduzido pela própria equipe: não substitui um pentest manual por terceiros nem prova a ausência de falhas (relatório em docs/testes/relatorio-pentest.pdf).

Alerta real. No mesmo dia, um teste em produção comprovou de ponta a ponta a US08 e o RF09: em uma conta da equipe, um vazamento já conhecido foi retirado da base de vazamentos conhecidos e uma verificação foi enfileirada; o monitoramento o detectou como novo, criou uma única notificação e o Amazon SES entregou o e-mail em cerca de 3 segundos. O gatilho foi manual, e o disparo agendado por novidade no catálogo segue coberto pelos testes de integração (relatório em docs/testes/relatorio-alerta-email.pdf).

5.5.3.3 Matriz do *score* de risco e regras RN02 e RN03

A regra de negócio RN02 define pesos determinísticos de 1 a 10 conforme o potencial de dano da classe exposta. A regra RN03 aplica fator temporal integral para incidentes com até 24 meses, fator 0,7 entre 24 e 60 meses e fator 0,4 acima de 60 meses. A Tabela 3 consolida os pesos (RN02) e os fatores temporais (RN03) utilizados como base de verificação nos testes do *score* de risco.

Tabela 3 – Matriz de pesos e criticidade do *score* de risco (RN02 e RN03)

Categoria do Dado Exposto	**Nível de Criticidade**	**Peso Aplicado (1 a 10)**	**Multiplicador Temporal (≤ 24 meses)**
Dados financeiros e senhas (Passwords)	Altíssima	10	1,0
Documentos oficiais, dados parciais de cartão, biometria, tokens e *perguntas* de segurança	Alta	8	1,0
Dicas de senha (Password hints)	Alta	7	1,0
Telefone e endereço residencial	Média	5	1,0
Data de nascimento	Média	4	1,0
Nome, nome de usuário e classes não mapeadas (peso padrão)	Baixa	3	1,0
E-mail, endereço IP e geolocalização	Baixa	2	1,0
User-Agent e telemetria de dispositivo	Baixa	1	1,0

Fonte: elaborado pelos autores (2026).

5.5.3.4 Exemplo prático de cálculo para verificação

Para um e-mail presente em múltiplos vazamentos, o *RiskScore* corresponde à soma dos pesos das classes expostas, ajustados pelo fator temporal, normalizada por (RN02 e RN03):

Considerem-se três incidentes: (a) senha (Passwords) há 6 meses, peso 10 e fator 1,0; (b) telefone há 36 meses, peso 5 e fator 0,7; e (c) nome completo há 72 meses, peso 3 e fator 0,4. O cálculo auditável é:

1. pontuação ajustada do incidente A: ;

2. pontuação ajustada do incidente B: ;
3. pontuação ajustada do incidente C: ;
4. soma ponderada: ;
5. normalização SafeID: .

O resultado 29,4 é arredondado para 29 antes da classificação (RN08) e persistido no campo inteiro riskScore (Quadro 5), o que enquadra o usuário na faixa de risco médio. Os testes devem reproduzir esse valor e também cobrir os limites 0 e 100, as fronteiras entre faixas (25/26, 50/51 e 75/76), a ausência de incidentes, duplicidades e as transições temporais em 24 e 60 meses. Os testes atuais verificam essas propriedades de forma genérica (fuzzing); casos com os valores exatos deste exemplo e do exemplo da Seção 2.4.1.4 devem ser acrescentados à suíte.

5.5.3.5 Matriz de rastreabilidade

O Quadro 6 relaciona os requisitos aos tipos de teste e às ferramentas efetivamente utilizadas, permitindo rastrear a verificação de cada requisito; as verificações executadas fora do CI estão indicadas entre parênteses.

Quadro 6 – Matriz de rastreabilidade (requisitos *versus* testes)

Requisito / US (Seção 3.3)	**Critério de aceite**	**Tipo de teste**	**Ferramenta**
RN02, RN03, RN08 e RF03 / US02	Calcular score determinístico, aplicar temporalidade e classificar na faixa correta.	Unitário (propriedades) e componente	Jest com fast-check; Vitest
RF01 e RF07 / US01 e US06	Verificar o e-mail pelo link, disparar a consulta e apresentar retorno ou estado pendente.	Integração e E2E da API	Jest (PostgreSQL real no CI)
RF04 / US01	Abrir o disjuntor com 50% de erro, sem abrir por 429, e falhar rápido.	Unitário	Jest
RF05 / US02	Exibir score, faixa em português e estado pendente.	Componente	Vitest e React Testing Library
RF06 / US03 e US04	Gerar diagnóstico e plano sem transmitir identificador pessoal; resposta padrão em falha.	Unitário	Jest
RF08 e RF11 / US05 e US06	Autenticar, renovar sessão, proteger rotas, persistir histórico cifrado e excluir a conta.	Unitário, integração e E2E da API	Jest
RF09 / US08	Notificar uma única vez por vazamento novo; descadastramento em um clique.	Integração e E2E da API	Jest
RN04 / RNF09	Bloquear tentativas de login acima do limite, por cliente.	E2E da API	Jest
RNF02, RNF03 e RNF04	Atingir as metas mensuráveis da Tabela 2.	Desempenho e carga (produção, 03/11/2026)	Grafana k6
Jornada completa / US01 a US04	Cadastro, confirmação, painel e plano no navegador.	E2E no navegador (cópia local, manual)	Playwright
Segurança / RNF06, RNF07 e RNF09	Nenhum achado alto ou crítico em aberto; nenhum usuário acessa dados de outro.	Teste de penetração (produção e cópia local)	OWASP ZAP e scripts em tests/security

Fonte: elaborado pelos autores (2026).

5.5.3.6 Governança, conformidade e manutenibilidade

Cada execução deve registrar versão do código, massa anonimizada, parâmetros, duração, ambiente e artefatos. Dados reais não devem integrar a suíte automatizada. Essa prática apoia os princípios de necessidade, segurança e prevenção da LGPD e as orientações da ANPD (2022), além de favorecer a repetibilidade. ISO, NIST, União Europeia e SGD são referências complementares de governança e segurança, sem substituir os requisitos legais brasileiros.

5.5.4 Testes Automatizados

A automação completa do ciclo de testes é tratada no SafeID como requisito indispensável de Engenharia de Software para assegurar a manutenibilidade contínua do código. A arquitetura de automação assenta-se sobre o *pipeline* de Integração Contínua do **GitHub Actions**, eliminando o fator de falha humana na validação da qualidade dos artefatos. O fluxo automatizado rege-se pelo princípio do *Fail-Fast* (falha rápida): qualquer alteração introduzida no repositório aciona a esteira de análise estática, testes unitários, de integração e E2E, geração de relatórios de cobertura e análise de segurança (CodeQL e Dependency Review). A automação assegura o determinismo do ecossistema, uma vez que o ambiente de teste é destruído e recriado a cada execução, estabelecendo como meta que efeitos colaterais de testes anteriores não mascarem falhas reais na lógica de negócio.

5.5.4.1 Fluxo de desenvolvimento orientado a testes

O SafeID adota os testes como instrumento de segurança, qualidade e velocidade de entrega, e não como etapa burocrática ao final do desenvolvimento. O Guia de Testes e Fluxo de Implementação, mantido na documentação do repositório pelo desenvolvedor responsável pela suíte, estabelece que os testes evitam regressões, explicitam o comportamento esperado de cada funcionalidade e orientam a implementação, reduzindo o retrabalho. Cada regra é verificada no nível mais adequado da Pirâmide de Testes descrita na Seção 5.5.1:

- **testes unitários** (npm run test:unit): validam regras isoladas, como o cálculo de risco, a validação e a sanitização de *payloads* e a criptografia e descriptografia, sem dependência de banco de dados;

- **testes de integração** (npm run test:integration): verificam a comunicação entre módulos e serviços de infraestrutura, como o salvamento do *snapshot* do usuário após o *scan* via Prisma, o registro de histórico e a integração com a fila de monitoramento;
- **testes E2E** (npm run test:e2e): validam o comportamento da API em fluxos HTTP reais, sem depender do navegador, como cadastro, verificação de e-mail e *login*, as rotas de histórico (GET /api/v1/scan/history) e a verificação de saúde da aplicação (GET /api/health).
- testes de componente do front-end (npm test, na pasta client): executam o Vitest com a React Testing Library.

Toda nova funcionalidade segue um fluxo de sete etapas: (i) definir o requisito de negócio, incluindo casos válidos, inválidos e de erro; (ii) escrever o teste que descreve o comportamento esperado; (iii) confirmar que o teste falha; (iv) implementar a solução mínima; (v) executar o teste específico e, em seguida, a suíte do nível impactado; (vi) validar regressões; e (vii) documentar a mudança, quando necessário. Como regra, uma funcionalidade sem teste não está pronta para integrar o fluxo de entregas. Complementam o fluxo as seguintes boas práticas: limitar os *mocks* ao necessário, testar o comportamento real do código e não apenas a chamada de funções, evitar testes fracos, separar as camadas de teste, adotar nomes claros e priorizar cenários simples.

5.5.4.2 Cobertura de testes

A cobertura de código é medida pelo comando npm run test:cov na suíte unitária do *back-end* e verificada automaticamente no *pipeline* de Integração Contínua. Na medição de 04/11/2026, feita pelo CI sobre o commit fdc2284 da main, foram executadas 36 suítes e 380 testes, todos aprovados e sem uso de *snapshots*. A Tabela 4 compara os resultados obtidos com os limites configurados no Jest e com as metas do projeto.

Tabela 4 – Cobertura da suíte unitária do *back-end* (medição de 04/11/2026)

Métrica	Resultado	Limite configurado no Jest	Meta do projeto	Situação
Statements	98,98%	80%	80%	Atingida
Branches	90,90%	80%	80%	Atingida
Functions	97,18%	85%	85%	Atingida
Lines	99,13%	80%	80%	Atingida

Fonte: elaborado pelos autores a partir do registro da execução do CI no GitHub Actions (2026). A suíte unitária usa respostas simuladas da API HIBP.

Na contagem instrumentada da medição anterior, de 17/10/2026, havia 811 *branches*, dos quais 732 cobertos e 79 não, acima do mínimo de 649 necessários para atingir 80%; a medição de 04/11 manteve a cobertura acima de 90%, apesar das funções incluídas desde então. O limite de *branches* configurado no Jest, que começou em 68%, foi elevado para 80% depois da expansão da suíte, e os limites de *statements* (80%), *functions* (85%) e *lines* (80%) correspondem às metas do projeto. Esses limites estão definidos no arquivo jest.unit.config.js. Se qualquer um deles não for cumprido, o *job* “Backend lint, build and unit tests” do GitHub Actions, que executa a instalação das dependências (npm ci), o *lint*, o *build* e o comando npm run test:cov, falha, e o *Pull Request* não pode ser integrado à ramificação *main*. A cobertura global é gerada apenas pela suíte unitária; os testes de integração e E2E validam fluxos mais amplos e não substituem esse critério.

Convém ressaltar que cobertura mede quais caminhos do código foram executados, e não a ausência de defeitos. Como a suíte automatizada usa respostas simuladas da HIBP, cenários que dependem do comportamento real da API, como variedade de vazamentos, limites de taxa em produção e falhas intermitentes, são exercitados apenas por simulação; em produção, esses eventos são acompanhados pelos alarmes descritos na Seção 4.7.3.

Os branches ainda não exercitados concentram-se em caminhos difíceis de reproduzir em teste unitário: o ExecuteRiskScanUseCase (espera de jobs, tempo-limite e falhas de persistência), a inicialização e os workers (papéis de APP_ROLE, encerramento e reconexão do Redis), o HibpClient e o HibpPlanService (combinações de erros HTTP e estados do disjuntor) e os serviços de monitoramento (falhas parciais durante a varredura e a reconciliação). Esses pontos devem ser cobertos com testes de comportamento, evitando testar detalhes internos desnecessários.

Como evolução incremental, a equipe pretende manter todos os testes atuais aprovados e avançar gradualmente rumo a 90% de cobertura de branches como limite configurado no CI, sem trocar testes de comportamento por testes de detalhes internos. O relatório detalhado é gerado no diretório server/coverage, com uma versão navegável (lcov-report/index.html) e o arquivo lcov.info, enviado ao Codecov a cada execução do CI.

5.6 Resultados Obtidos

Até a data desta versão, o SafeID está em produção em https://www.safeid.app.br, implantado na AWS. A aplicação verifica a titularidade do e-mail no cadastro, consulta as exposições registradas na fonte HIBP por processamento assíncrono e resiliente, calcula o *Risk Score* segundo o modelo determinístico das regras RN02, RN03 e RN08, gera o *plano* de

ação com o modelo de linguagem, mantém o histórico cifrado e monitora continuamente os usuários que autorizam, com alertas por e-mail e no painel. A suíte unitária do *back-end* atingiu todas as metas de cobertura (Tabela 4); em 03/11/2026, a campanha de carga atingiu as metas mensuráveis da Tabela 2 com 0,00% de erro, o teste de penetração não deixou achado alto ou crítico em aberto, a jornada no navegador passou nos quatro ambientes e um alerta real foi entregue por e-mail em produção (Seção 5.5.3).

A verificação dos objetivos específicos definidos na Seção 1.2.2 apresenta o seguinte resultado:

- **Concluído na Fase PI-1** — investigar, selecionar e consumir APIs OSINT;
- **Concluído na Fase PI-1** — modelar e implementar o algoritmo de Risk Score;
- **Concluído na Fase PI-1** — desenvolver a interface web responsiva e a visualização do risco;
- **Concluído parcialmente (PI-1 e PI-2)** — projetar a arquitetura de persistência e implementar o histórico e as funções de conta; a gestão de múltiplos identificadores (US07) ficou fora da entrega final;
- **Concluído na Fase PI-2** — integrar a LLM como assistente cognitivo;
- **Concluído na Fase PI-2** — aplicar Privacy by Design e as diretrizes da LGPD ao ciclo operacional: controles técnicos, consentimento registrado, Política de Privacidade e Termos de Uso publicados, cópia dos dados e exclusão de conta implementadas; relatório de impacto e plano de resposta a incidentes aprovados em 04/11/2026; a conformidade jurídica integral depende também de avaliação especializada.

Nenhum objetivo específico foi classificado como “não iniciado”. Em relação às necessidades identificadas na Seção 2.3.3, os resultados atendem às necessidades N1 (conhecer as exposições), N2 (compreender a gravidade), N3 (saber como agir), por meio do plano de ação gerado pela IA, e N4 (acompanhar a evolução), por meio do histórico e do monitoramento contínuo; a necessidade N5 (preservar a privacidade) é atendida pelos controles técnicos de pseudonimização, criptografia e exigência de conta verificada, com a ressalva de que a conformidade jurídica depende também dos itens de governança ainda pendentes.

5.7 Limitações da Implementação

Nesta etapa, o SafeID apresenta as seguintes limitações principais:

- dependência atual de uma única API de *threat intelligence*, a Have I Been Pwned (HIBP);
- cobertura de *branches* de 90,9% na suíte unitária, com caminhos de infraestrutura e de integrações externas ainda não exercitados, e respostas da HIBP apenas simuladas na suíte automatizada;
- ausência, até o momento, de testes com usuários reais e de validação empírica de usabilidade;
- escopo restrito ao e-mail como identificador primário;
- testes de carga restritos à rota de saúde e à página inicial, a partir de um único computador, sem rotas autenticadas nem teste de longa duração; jornada no navegador fora do CI, executada manualmente em cópia local; e teste de penetração automatizado, conduzido pela própria equipe, sem pentest manual por terceiros;
- funcionalidades ainda não implementadas: gestão de múltiplos identificadores (o esquema atual não a comporta), painel administrativo e tema claro;
- alinhamento integral à LGPD ainda em consolidação, uma vez que os controles técnicos implementados apoiam, mas não comprovam isoladamente, a conformidade jurídica.
- ponto único de falha de processo: a API HTTP e os workers rodam no mesmo processo Node.js, em uma única task; uma falha desse processo interrompe ao mesmo tempo o atendimento e o processamento, até que o ECS recrie a task (1 a 2 minutos). O código já permite separar os papéis (APP_ROLE), o que está priorizado na Seção 6.3;
- estado local do Circuit Breaker: o disjuntor da HIBP é mantido em memória, por processo; com várias instâncias, cada uma teria o próprio estado, e uma instância poderia continuar chamando a API enquanto outra já detectou a falha. O limite de requisições à HIBP já é coordenado entre instâncias pelo Redis, de modo que a escala horizontal não viola o limite do plano; o disjuntor distribuído está no roteiro de evolução;
- banco de dados e Redis sem redundância Multi-AZ e sem acordo de nível de serviço formal (RNF10);
- observabilidade inicial: logs estruturados e alarmes, sem métricas de aplicação dedicadas nem rastreamento distribuído.

6 CONCLUSÃO

Este capítulo encerra o documento avaliando o projeto como um todo e relacionando os resultados alcançados aos objetivos inicialmente estabelecidos. São sintetizadas as principais conclusões, registradas as dificuldades encontradas, com as escolhas e os descartes delas decorrentes, e indicadas as possibilidades de continuidade do trabalho.

6.1 Considerações Finais

O SafeID foi concebido para enfrentar a assimetria informacional entre a existência de registros técnicos sobre vazamentos de dados e a capacidade do cidadão comum de interpretá-los e agir. A partir do contexto caracterizado no Capítulo 2 — volume expressivo de ataques no Brasil, longo ciclo entre exposição e contenção, fadiga de segurança e insuficiência da comunicação técnica sem mediação —, o projeto propôs e implementou uma plataforma que articula detecção pós-incidente, pontuação de risco explicável e apoio cognitivo à decisão.

Esta versão consolida as entregas da PI-1 e da PI-2, além do amadurecimento técnico e analítico da equipe de desenvolvimento. O projeto cumpriu os objetivos da PI-1 — seleção e consumo da API OSINT, modelagem e implementação do *Risk Score*, interface responsiva e base de persistência — e, na PI-2, integrou a LLM, implementou contas, histórico, monitoramento contínuo e alertas e implantou a aplicação na AWS; concluiu a validação por testes de carga, de segurança e de jornada no navegador e formalizou a governança da LGPD; a gestão de múltiplos identificadores ficou fora da entrega final (Seção 5.6). O modelo de risco, com pesos fundamentados em referências normativas e técnicas e parâmetros auditáveis, constitui contribuição central do trabalho, por oferecer ao usuário uma medida comparável e explicável de sua exposição. Do ponto de vista arquitetural, a combinação de monólito modular com processamento assíncrono por filas mostrou-se adequada às restrições de taxa da fonte externa e ao porte do projeto.

Ressalva-se, contudo, que o desempenho sob carga foi medido apenas em rotas simples e a partir de um único gerador, que a cobertura de testes mede caminhos executados, e não ausência de defeitos, e que a efetividade comunicacional da solução ainda depende de validação com usuários reais. Em síntese, o SafeID demonstra a viabilidade técnica de tornar operacional, para o cidadão, o direito à informação sobre a exposição de seus dados, desde que acompanhado de governança, controles técnicos e observância contínua das diretrizes da LGPD.

6.2 Dificuldades Encontradas

O processo de desenvolvimento de um ecossistema pautado em *Security by Design* exigiu constantes avaliações de impacto entre as dimensões de custos, desempenho, segurança e prazos. Diante disso, o grupo deparou-se com desafios que exigiram escolhas arquiteturais rígidas e o descarte consciente de soluções inicialmente planejadas.

6.2.1 Dificuldades Técnicas

As principais dificuldades técnicas enfrentadas durante o desenvolvimento e as soluções adotadas são descritas a seguir:

- **Gerenciamento da taxa de limite (*rate limiting*) das APIs externas:** a principal barreira operacional residiu na integração com a API *Have I Been Pwned*. Por tratar-se de um serviço comercial com limite de requisições por minuto definido pelo plano contratado, sob pena de bloqueio da chave, tornou-se inviável adotar uma arquitetura de chamadas síncronas diretas a partir do *front-end*. A solução desse impasse exigiu uma reestruturação profunda do *back-end*, introduzindo uma camada de mensageria assíncrona baseada em filas (**BullMQ** e **Redis**) para cadenciar e suavizar o fluxo de consultas em conformidade com as diretrizes do provedor OSINT. Na PI-2, o limite passou a ser lido do próprio plano e aplicado de forma global no Redis, válido para todas as instâncias.
- **Conformidade com a LGPD:** na PI-2, foram implementados o consentimento registrado, a Política de Privacidade e os Termos de Uso, a verificação de titularidade do e-mail, a cópia dos dados e a exclusão da conta; a formalização da governança (relatório de impacto e resposta a incidentes) foi concluída com o relatório de impacto e o plano de resposta a incidentes, aprovados em 04/11/2026. Os controles técnicos descritos apoiam a conformidade, mas não constituem, isoladamente, comprovação jurídica integral, que também depende de governança, processos, bases legais e avaliação especializada.
- **Determinismo e custos em Inteligência Artificial:** o controle da imprevisibilidade das respostas exigiu instruções fixas detalhadas, temperatura baixa (0,3), saída obrigatória em JSON com validação dos campos e uma resposta padrão determinística para falhas; o custo foi contido com cache das recomendações por 7 dias, que evita nova chamada para o mesmo conjunto de vazamentos.

- **Acesso restrito à base de dados real:** como a API HIBP é paga, os testes automatizados limitam-se a *respostas simuladas* e aos e-mails de teste do provedor, o que exigiu a simulação de cenários de volume e de falha e restringiu a cobertura dos *branches* de integração externa (Seção 5.5.4.2).
- **Revisões de decisões internas de modelagem e priorização:** nem todas as dificuldades foram externas; algumas decisões da própria equipe precisaram ser revistas: (i) a primeira versão da tabela de pesos (Tabela 1) previa categorias, como hashes Bcrypt ou MD5, que o HIBP não informa, e foi realinhada às classes de dados que a API efetivamente devolve, com um peso padrão para classes não mapeadas; (ii) o limite de cobertura de branches foi fixado em 68% enquanto a suíte era pequena e só foi elevado a 80% depois que a cobertura real o superou com folga, decisão que evitou bloquear o CI, mas adiou a cobrança da meta; (iii) a consulta anônima de visitantes, prevista na PI-1, foi abandonada na PI-2 por permitir consultar e-mails de terceiros, o que contrariava o princípio da necessidade e abria espaço para enumeração; (iv) pelo mesmo motivo de simplicidade e de proteção da cota da HIBP, a consulta manual sob demanda foi substituída por consultas automáticas no cadastro e no monitoramento; e (v) o modelo de linguagem previsto, GPT-4o-mini, foi substituído por um modelo hospedado no Azure AI Foundry. Em retrospectiva, a equipe priorizou entregar o fluxo completo em produção antes de medir desempenho, o que explica a ausência, nesta versão, dos testes de carga.

6.2.2 Escolhas e Descartes

Esta subseção sistematiza as principais decisões tomadas durante a concepção e a implementação do SafeID, considerando sua aderência aos requisitos, às restrições do projeto e aos objetivos acadêmicos definidos.

As escolhas arquiteturais consolidadas foram as seguintes:

- **Adoção de uma arquitetura em camadas inspirada na Clean Architecture:** a escolha por isolar as regras de negócio (core domain) das ferramentas de infraestrutura (bancos de dados e APIs de IA) mostrou-se a decisão mais determinante do projeto. Essa dissociação permitiu que os testes unitários e de *fuzzing* fossem executados de forma isolada e rápida no *pipeline* do **GitHub Actions**, protegendo o sistema contra regressões sem a necessidade de acionar serviços externos pagos a cada *build*.
- **Escolha do modelo de linguagem:** o projeto previa o GPT-4o-mini e, na PI-2, adotou o grok-4-20-non-reasoning, hospedado no Azure AI Foundry e acessado pelo mesmo

SDK da OpenAI. A troca teve dois motivos: (a) custo, porque o custo por chamada do modelo adotado no Azure AI Foundry é compatível com a premissa do Apêndice C; e (b) capacidade de processamento, porque o provedor comporta o volume de chamadas do serviço e o modelo, sem etapa de raciocínio, responde diretamente dentro do tempo-limite de 10 s do RNF03. Como condições técnicas, o modelo também precisava produzir saída em JSON e ser compatível com o SDK já integrado, o que permitiu a troca por variável de ambiente, sem mudança de código. A equipe não registrou um comparativo medido entre modelos, o que fica como limitação da decisão.

Em contrapartida, as alternativas descartadas e as razões do descarte são descritas a seguir:

- **Implantação imediata da infraestrutura AWS (adiada na PI-1 e concluída na PI-2):** para a fase de MVP, a equipe adotou empacotamento agnóstico via contêineres Docker Multi-stage e hospedagem temporária em plataformas PaaS (Railway e Vercel), evitando custos mínimos em dólar e complexidade precoce de redes virtuais; a migração para a AWS ocorreu na PI-2, sem mudança de código (Seção 5.4).
- **Fragmentação prematura em microsserviços (descartada):** considerou-se construir o SafeID com os módulos de Inteligência Artificial, autenticação e notificação em serviços isolados. A decisão foi tomada com critérios objetivos: (a) custo: cada serviço adicional no Fargate, no porte atual (0,5 vCPU e 1 GB), custa cerca de US$ 9 por mês, de modo que três serviços somariam cerca de US$ 18 por mês a um ambiente de US$ 75 a 105 por mês, aumento de 17% a 24%, sem contar balanceamento e observabilidade adicionais; (b) esforço: três imagens, três pipelines de implantação e contratos de API entre serviços, para uma equipe de quatro pessoas com 10 a 12 horas semanais cada; (c) carga: o volume de consultas é limitado pelo plano da HIBP (cerca de 9 por minuto no plano de entrada), muito abaixo do que exigiria escalar partes do sistema de forma independente; e (d) risco: chamadas de rede entre serviços acrescentariam latência e pontos de falha, além de exigir rastreamento distribuído. O monólito modular com filas atende a carga atual, e a separação do worker em serviço próprio (APP_ROLE=worker), já suportada pelo código, é o primeiro passo previsto quando a varredura passar de cerca de uma hora ou a latência da API subir durante as varreduras.

6.3 Trabalhos Futuros

O projeto se encerra com a apresentação final, em 13/11/2026, e não há etapa de desenvolvimento posterior. Os itens obrigatórios (Must) foram todos concluídos antes dessa data, e os demais, priorizados pelo método MoSCoW, ficam registrados como recomendações para uma eventual continuidade do SafeID, sem responsável nem prazo.

Must (obrigatório até a apresentação final, em 13/11/2026): (i) testes de penetração e correção das vulnerabilidades graves — concluído em 03/11/2026, sem achado alto ou crítico em aberto (S01 e S07 corrigidos e reconfirmados em produção); (ii) campanha de carga com o k6 — concluída em 03/11/2026, com as metas mensuráveis atingidas e os desvios explicados na Seção 5.5.3.2; (iii) testes com os valores exatos dos exemplos das Seções 2.4.1.4 e 5.5.3.4 — o exemplo de 74 já tinha teste, e os casos de 85 e de 29 foram incluídos em 04/11/2026; (iv) verificação da presença de EMAIL_HASH_SECRET em produção — concluída em 05/11/2026: a chave está definida no arquivo de ambiente de produção e é enviada ao SSM pelo script de implantação; (v) relatório de impacto à proteção de dados e plano de resposta a incidentes — concluídos e aprovados pelo Product Owner em 04/11/2026, publicados em docs/lgpd.

Should (recomendado para uma continuidade**):** gestão de múltiplos identificadores (US07); separação do worker em serviço próprio e Circuit Breaker com estado compartilhado no Redis, com número mínimo de chamadas configurado; troca do maskEmail por HMAC truncado e migração dos logs legados para o logger estruturado; validação de usabilidade com pelo menos 10 participantes leigos, com a meta de que ao menos 80% interpretem corretamente a faixa de risco e a primeira ação do plano; instrução específica para biometria no plano de ação e reavaliação da tabela de pesos com dados de uso; inclusão da suíte do Playwright no CI e medição de rotas autenticadas com carga distribuída; e as recomendações de baixo risco do teste de penetração (cabeçalhos COOP e CORP no site e resposta 404 para a análise de outro usuário, S03 e S04).

Could (oportuno): segunda fonte de threat intelligence para reduzir a dependência da HIBP; painel administrativo com métricas agregadas (US10); tema claro com verificação automática de acessibilidade (axe-core) e testes de regressão visual; verificação automática da regra de dependência entre camadas no CI; métricas de aplicação e rastreamento distribuído; alta disponibilidade (Multi-AZ) do banco e do Redis.

Won’t (fora do escopo desta fase, mantidos como linhas de produto): o monitoramento de chaves PIX, que depende de uma fonte de dados que o HIBP não oferece e

corresponde à classe de dados financeiros da Tabela 1; e as funcionalidades B2B2C, previstas na parceria com a Enetsec (Seção 2.1.4), que dependem de acordo comercial e cujo financiamento poderia vir do reinvestimento discutido no Apêndice C. Ambos dependem de uma continuidade do projeto e só fariam sentido depois dos itens Should.

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR ISO/IEC 25010**: engenharia de sistemas e software — requisitos e avaliação da qualidade de produto de software (SQuaRE). Rio de Janeiro: ABNT, 2011. Disponível em: https://www.abntcatalogo.com.br/. Acesso em: 15 set. 2026.

AUTORIDADE NACIONAL DE PROTEÇÃO DE DADOS. **Guia orientativo para definições dos agentes de tratamento de dados pessoais e do encarregado**. Versão 2.0. Brasília, DF: ANPD, 2022. Disponível em: https://www.gov.br/anpd/pt-br/centrais-de-conteudo/materiais-educativos-e-publicacoes/guia-orientativo-para-definicoes-dos-agentes-de-tratamento-de-dados-pessoais-e-do-encarregado. Acesso em: 15 set. 2026.

BRASIL. **Lei nº 13.709, de 14 de agosto de 2018**. Dispõe sobre a proteção de dados pessoais e altera a Lei nº 12.965, de 23 de abril de 2014 (Marco Civil da Internet). Brasília, DF: Presidência da República, 2018. Disponível em: http://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm. Acesso em: 15 set. 2026.

CHARTMOGUL; GROWTH UNHINGED; PRODUCTLED. The conversion report: benchmarks for trials, freemium, and conversion in 2026. [S. l.]: ChartMogul, 2026. Disponível em: https://chartmogul.com/reports/saas-conversion-report/. Acesso em: 2 nov. 2026.

CHEN, Jing; HENRY, Elaine; JIANG, Xi. Is cybersecurity risk factor disclosure informative? Evidence from disclosures following a data breach. **Journal of Business Ethics**, v. 187, n. 1, p. 199-224, 2023.

CHEN, Qiang; WANG, Lun. Social response and management of cybersecurity incidents. **Academic Journal of Sociology and Management**, v. 2, n. 4, p. 49-56, 2024.

CONTI, Mauro; DARGAHI, Tooska; DEHGHANTANHA, Ali. Cyber threat intelligence: challenges and opportunities. **Cyber Threat Intelligence**, [s. l.], p. 1-6, 2018a.

CONTI, Mauro *et al*. Shadow attacks based on password reuses: a quantitative empirical analysis. **IEEE Transactions on Dependable and Secure Computing**, v. 15, n. 2, p. 309-320, 2018b.

FIRST — FORUM OF INCIDENT RESPONSE AND SECURITY TEAMS. **Common Vulnerability Scoring System v3.1**: specification document. [S. l.]: FIRST, 2019. Disponível em: https://www.first.org/cvss/v3.1/specification-document. Acesso em: 15 set. 2026.

FIRST — FORUM OF INCIDENT RESPONSE AND SECURITY TEAMS. Common Vulnerability Scoring System v4.0: specification document. [S. l.]: FIRST, 2023. Disponível em: https://www.first.org/cvss/v4.0/specification-document. Acesso em: 2 nov. 2026.

FORTINET. **Brasil é o segundo país que mais sofre ataques cibernéticos na América Latina**. São Paulo, 18 ago. 2022. Disponível em: https://www.fortinet.com/br/corporate/about-us/newsroom/press-releases/2022/brasil-e-o-segundo-pais-que-mais-sofre-ataques-ciberneticos-na-a. Acesso em: 8 out. 2026.

GRAFANA LABS. **Grafana k6 documentation**. [S. l.]: Grafana Labs, [2026]. Disponível em: https://grafana.com/docs/k6/latest/. Acesso em: 15 set. 2026.

HAVE I BEEN PWNED. **API documentation**: version 3. [S. l.], [s. d.]. Disponível em: https://haveibeenpwned.com/API/v3. Acesso em: 5 maio 2026.

IBM SECURITY. **Cost of a data breach report 2023**. [S. l.]: IBM, 2023. Disponível em: https://www.ibm.com/security/data-breach. Acesso em: 18 abr. 2026.

JEST. **Getting started**. [S. l.]: Jest, 2026. Disponível em: https://jestjs.io/docs/getting-started. Acesso em: 15 set. 2026.

META. **React**: a JavaScript library for building user interfaces. [S. l.], [s. d.]. Disponível em: https://react.dev/. Acesso em: 5 maio 2026.

NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY. The NIST Cybersecurity Framework (CSF) 2.0. Gaithersburg: NIST, 2024. (NIST CSWP 29). DOI: https://doi.org/10.6028/NIST.CSWP.29. Acesso em: 2 nov. 2026.

NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY. **NIST Special Publication 800-63B-4**: digital identity guidelines — authentication and authenticator management. Gaithersburg: NIST, 2025. DOI: https://doi.org/10.6028/NIST.SP.800-63B-4. Acesso em: 2 nov. 2026.

NEMEC ZLATOLAS, Lili; WELZER, Tatjana; LHOTSKA, Lenka. Data breaches in healthcare: security mechanisms for attack mitigation. **Cluster Computing**, v. 27, n. 7, p. 8639-8654, 2024.

NODE.JS. **Node.js**: JavaScript runtime built on Chrome’s V8 JavaScript engine. [S. l.], [s. d.]. Disponível em: https://nodejs.org/. Acesso em: 5 maio 2026.

NODESHIFT. **Opossum**: Node.js circuit breaker. [S. l.]: NodeShift, 2026. Disponível em: https://github.com/nodeshift/opossum/blob/main/README.md. Acesso em: 15 set. 2026.

OBJECT MANAGEMENT GROUP. **OMG Unified Modeling Language (OMG UML)**: version 2.5.1. Needham: OMG, 2017. Disponível em: https://www.omg.org/spec/UML/2.5.1. Acesso em: 12 out. 2026.

OWASP FOUNDATION. OWASP risk rating methodology. [S. l.]: OWASP, [s. d.]. Disponível em: https://owasp.org/www-community/OWASP_Risk_Rating_Methodology. Acesso em: 2 nov. 2026.

PLAYWRIGHT. **Installation**: Playwright Test. [S. l.]: Microsoft, 2026. Disponível em: https://playwright.dev/docs/intro. Acesso em: 15 set. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL**: the world’s most advanced open source relational database. [S. l.], [s. d.]. Disponível em: https://www.postgresql.org/. Acesso em: 5 maio 2026.

PRISMA. **Prisma**: next-generation ORM for Node.js & TypeScript. [S. l.], [s. d.]. Disponível em: https://www.prisma.io/. Acesso em: 5 maio 2026.

RECURLY. Churn rate benchmarks. [S. l.]: Recurly, [2026]. Disponível em: https://recurly.com/research/churn-rate-benchmarks/. Acesso em: 2 nov. 2026.

REUBEN-OWOH, Blessing; HAIG, Ella. A systematic review of voluntary cybersecurity standards and frameworks. **International Journal of Information Security**, v. 24, n. 5, p. 206, 2025.

SANTOS, J. F.; PINTO, G. Privacidade e segurança em sistemas de informação: uma análise dos aspectos psicológicos e cognitivos envolvidos na percepção e na adoção de medidas de segurança. **Revista Interface Tecnológica**, v. 21, n. 1, p. 288-299, 2025.

TAILWIND LABS. **Tailwind CSS**: rapidly build modern websites without ever leaving your HTML. [S. l.], [s. d.]. Disponível em: https://tailwindcss.com/. Acesso em: 5 maio 2026.

VON SOLMS, Rossouw; VAN NIEKERK, Johan. From information security to cyber security. **Computers & Security**, v. 38, p. 97-102, 2013.

ZAP. **ZAP documentation**. [S. l.]: ZAP Dev Team, [2026]. Disponível em: https://www.zaproxy.org/docs/. Acesso em: 4 nov. 2026.

APÊNDICE A – GESTÃO DO PROJETO

Este apêndice reúne os elementos de gestão do projeto produzidos pelos autores — organização da equipe, metodologia, gestão de riscos, *backlog* do produto e planejamento das *Sprints* —, cuja inclusão integral no corpo do texto prejudicaria sua fluidez.

A.1 Organização da Equipe

A equipe do SafeID é composta por quatro membros, distribuídos em funções que cobrem o ciclo completo de desenvolvimento, desde a concepção do produto até a implementação e a persistência dos dados:

- **Matheus Pereira Pereira:** *Product Owner* e *Scrum Master*, responsável pela visão do produto, pela gestão do *backlog* e pela facilitação dos ritos ágeis;
- **Lucas Vieira de Souza:** desenvolvedor *back-end*, responsável pela lógica de servidor, pela integração com APIs externas e pela inteligência artificial;
- **Joao Antônio Chamusca Martins:** desenvolvedor *front-end*, responsável pela interface do usuário, pela responsividade e pela experiência do usuário (UX);
- **Kelvyn Gabriel Milu de Oliveira:** desenvolvedor e administrador de banco de dados (DBA), responsável pela modelagem, pela integridade e pelo desempenho das estruturas de dados.

As atividades foram distribuídas de forma colaborativa por meio de uma matriz de responsabilidades (Quadro 7), que relaciona os papéis no Scrum, as especialidades técnicas, as principais entregas e a disponibilidade estimada de cada integrante. Para assegurar equilíbrio na participação, adotou-se uma dedicação uniforme de 10 a 12 horas semanais por membro. Somadas, as quatro dedicações formam a capacidade da equipe, de 40 a 48 horas semanais (4 × 10 a 12 h); cada integrante responde, portanto, por um quarto (25%) dessa capacidade. A base de cálculo não é uma jornada integral de 40 horas por pessoa: em relação a ela, a dedicação individual equivale a 25% a 30%. Ao longo dos nove meses com desenvolvimento do cronograma (cerca de 36 semanas, excluído o mês de transição), a capacidade total estimada é de 1.440 a 1.730 horas, base usada para conferir o volume do backlog (Seção A.4). O PO/*Scrum Master* atua na interface com os requisitos de negócio, enquanto o time de desenvolvimento concentra-se na implementação técnica e na integração coesa entre *front-end*, *back-end* e banco de dados.

Quadro 7 – Matriz de responsabilidades e atribuições da equipe

Integrante	Papel no Scrum	Especialidade Técnica	Atribuições e Entregas	Dedicação Estimada
Matheus Pereira	*Scrum Master* / PO	Gestão e Dados	Liderança da documentação acadêmica, gestão do *backlog*, curadoria dos dados das APIs e validação do cálculo de risco (*score*).	10 a 12 h/semana (25%)
Kelvyn Gabriel	Desenvolvedor / DBA	Cibersegurança	Modelagem do banco de dados (PostgreSQL), segurança da arquitetura, definição dos pesos de criticidade dos vazamentos e conformidade com a LGPD.	10 a 12 h/semana (25%)
Lucas Vieira	Desenvolvedor	*Back-end* / APIs	Construção da API do SafeID (Node.js), integração com o *Have I Been Pwned*, conexão com a LLM e lógica de processamento de metadados.	10 a 12 h/semana (25%)
João Antônio	Desenvolvedor	*Front-end* / UX	*Design* da interface (Figma), desenvolvimento dos componentes em React, implementação do termômetro de risco e validação da responsividade.	10 a 12 h/semana (25%)

Fonte: elaborado pelos autores (2026).

A gestão do projeto foi desenhada para que a construção conceitual e a técnica caminhem juntas:

- **Documentação coletiva:** embora Matheus lidere a redação acadêmica, Kelvyn, Lucas e João são responsáveis por documentar as especificações técnicas (dicionário de dados, diagrama de classes e documentação da API no GitHub);
- **Repositório único (monorepo):** optou-se por um repositório centralizado no GitHub, o que facilita o rastreamento da autoria de cada funcionalidade e contribui para que a documentação técnica permaneça próxima ao código (README.md e *Wiki*);
- **Integração de conhecimentos:** a experiência profissional de Kelvyn em cibersegurança alimenta diretamente a lógica desenvolvida por Lucas nas APIs, enquanto Matheus traduz essa complexidade técnica para a linguagem acessível da documentação e da interface projetada por João.

A.2 Metodologia de Gestão e Desenvolvimento (Scrum)

O projeto adota o **Scrum** como metodologia ágil de gestão, permitindo entregas incrementais e adaptação rápida a mudanças de requisitos. O Scrum foi selecionado por sua eficácia em projetos de tecnologia nos quais a integração de diferentes camadas (IA, APIs e banco de dados) exige ciclos constantes de inspeção e adaptação. As atividades foram organizadas em ciclos quinzenais (*Sprints*), iniciados em março de 2026:

- **Planejamento:** no início de cada *Sprint*, a equipe define os itens do *backlog* a serem desenvolvidos;

- **Construção e documentação:** paralelamente ao código, a documentação técnica é atualizada para garantir o registro da arquitetura;
- **Reuniões de alinhamento:** realizadas para identificar impedimentos técnicos, especialmente na integração com as APIs de monitoramento de incidentes.
- Sprint Review e Retrospectiva: não foram realizadas como cerimônias formais. A inspeção do incremento ocorreu na revisão obrigatória dos Pull Requests e nas reuniões de alinhamento, e as adaptações de processo foram decididas nessas reuniões, sem registro próprio. A ausência desses eventos é uma limitação do uso do Scrum neste projeto, porque reduziu a oportunidade de avaliar cada incremento com o parceiro e de registrar as lições aprendidas a cada ciclo.

O acompanhamento é realizado no GitHub Projects, conforme descrito na Seção 5.1 e ilustrado na Figura 13.

A.3 Gestão de Riscos do Projeto

A gestão de riscos do SafeID é conduzida de forma contínua ao longo das *Sprints*, com identificação, avaliação numérica de probabilidade e impacto, definição de responsáveis e acompanhamento das ações de resposta no GitHub Projects. A priorização considera riscos capazes de comprometer a qualidade técnica, o cronograma, a disponibilidade da equipe ou serviços externos essenciais. Sempre que um risco se materializa, o item correspondente é reavaliado no planejamento da *Sprint* e recebe uma ação de contingência compatível com o impacto observado. Os principais riscos estão consolidados no Quadro 8. Probabilidade (P) e impacto (I) são pontuados de 1 (muito baixo) a 5 (muito alto), e a severidade é o produto P × I, de 1 a 25, classificada como baixa (1 a 6), média (8 a 12) ou alta (15 a 25), o que permite ordenar objetivamente os riscos. Além dos riscos de execução, a matriz inclui riscos próprios do domínio de segurança, já que o SafeID trata precisamente de exposição de dados.

Quadro 8 – Matriz de riscos do projeto SafeID

Categoria	Risco	P	I	P×I	Estratégia de mitigação	Responsável	Última revisão
Segurança	Violação de dados do próprio SafeID (banco, segredos, logs ou conta de nuvem).	2	5	10	Dados de incidentes cifrados, e-mails pseudonimizados com HMAC, segredos no SSM, banco em rede privada, menor privilégio, CodeQL e Dependabot, pentest; formalizar plano de resposta a incidentes, com comunicação à ANPD e aos titulares (LGPD, art. 48).	Kelvyn Gabriel	02/11/2026
Câmbio	Variação do dólar sobre custos de AWS, HIBP e IA, cotados em moeda estrangeira.	4	3	12	Acompanhar o câmbio mensalmente; análise de sensibilidade (Apêndice C); modo economia sem NAT; cache de recomendações; plano mínimo da HIBP.	Matheus Pereira	02/11/2026
Técnico	Falhas na integração entre front-end, back-end, banco	3	4	12	Contratos de API, testes automatizados de integração e E2E e integração contínua para	Lucas Vieira	24/09/2026

Categoria	Risco	P	I	P×I	Estratégia de mitigação	Responsável	Última revisão
	de dados e motor de risco.				detectar regressões antecipadamente.		
Prazo	Subestimação da complexidade ou acúmulo de itens críticos no fim da Sprint.	3	4	12	Refinar o backlog antes do planejamento, limitar o trabalho em andamento, acompanhar marcos semanalmente, priorizar o caminho crítico e reservar sprint de contingência (Seção A.6).	Matheus Pereira	02/11/2026
Dependência de terceiros	Indisponibilidade, limitação de taxa ou mudança de contrato da API HIBP.	3	4	12	Tempo-limite, novas tentativas, Circuit Breaker, cache, filas com limite global, mensagem de contingência e camada de integração desacoplada para permitir fonte alternativa.	Lucas Vieira	24/09/2026
Técnico / IA	Inadequação ou indisponibilidade das respostas da LLM.	3	3	9	Instruções fixas, saída em JSON validada, resposta padrão determinística e diagnóstico de risco independente da LLM.	Lucas Vieira	02/11/2026
Equipe	Indisponibilidade temporária de integrante ou concentração de conhecimento em uma única função.	3	3	9	Documentação técnica atualizada, revisão cruzada obrigatória de Pull Requests, pareamento e compartilhamento de conhecimento sobre componentes críticos.	Matheus Pereira	24/09/2026
Uso indevido	Uso do sistema para enumerar e-mails ou apoiar engenharia social.	2	4	8	Sem consulta pública: resultado só para o titular verificado; resposta idêntica no reenvio de verificação, exista a conta ou não; limites por IP.	Lucas Vieira	02/11/2026
Legal / LGPD	Tratamento de e-mails de terceiros sem base legal.	1	4	4	Verificação obrigatória de titularidade, consentimento registrado, Política de Privacidade publicada e exclusão de conta pelo titular.	Matheus Pereira	02/11/2026

Fonte: elaborado pelos autores (2026).

As respostas previstas combinam prevenção, redução de probabilidade, limitação de impacto e contingência. Em especial, o desacoplamento de integrações externas evita que a indisponibilidade do HIBP ou falhas da LLM interrompam as funções centrais do SafeID. Pela pontuação, os riscos prioritários são a violação de dados do próprio SafeID, cujo impacto é o mais alto, e os riscos de severidade 12, entre eles o cambial. A matriz deve ser revisada nas reuniões de alinhamento e atualizada quando houver alteração de probabilidade, impacto, responsável ou estratégia de tratamento.

A.4 *Backlog* do Produto

O *backlog* do SafeID foi categorizado para permitir uma visão clara do que compõe o protótipo (PI-1) e a versão final (PI-2), conforme o Quadro 9, que traz também a situação de cada item na data desta versão e uma estimativa de esforço em horas de trabalho da equipe. As estimativas são de ordem de grandeza: somam cerca de 1.100 horas, abaixo da capacidade de 1.440 a 1.730 horas calculada na Seção A.1, o que deixa margem de 25% a 35% para retrabalho e imprevistos. O item 24 foi incluído nesta versão para registrar a implantação na AWS.

Quadro 9 – *Backlog* do produto (PI-1 e PI-2)

ID	Categoria	Descrição do item	Fase	Prioridade	Esforço (h)	Situação
01	Requisito	Levantamento de requisitos funcionais e não funcionais.	PI-1	Essencial	40	Concluído
02	Requisito	Pesquisa e seleção de APIs de monitoramento (OSINT).	PI-1	Essencial	24	Concluído
03	Arquitetura	Definição da stack tecnológica (React, Node.js, banco de dados).	PI-1	Essencial	12	Concluído
04	Banco de Dados	Modelagem do Diagrama Entidade-Relacionamento (DER).	PI-1	Essencial	16	Concluído
05	Banco de Dados	Implementação do esquema de tabelas para histórico e usuários.	PI-1	Alta	16	Concluído
06	Front-end	Prototipação de alta fidelidade (Figma).	PI-1	Alta	40	Concluído
07	Back-end	Desenvolvimento do servidor API (Node.js + NestJS).	PI-1	Essencial	60	Concluído
08	Integração	Implementação do módulo de consumo da API externa (HIBP).	PI-1	Essencial	40	Concluído
09	Lógica	Desenvolvimento do algoritmo de cálculo de Risk Score.	PI-1	Essencial	40	Concluído
10	Front-end	Implementação da tela de consulta por identificador (e-mail).	PI-1	Alta	24	Concluído (substituída pelo cadastro na PI-2)
11	Front-end	Desenvolvimento do dashboard de visualização de vazamentos.	PI-1	Alta	48	Concluído
12	Documentação	Redação da documentação de gestão e análise do projeto.	PI-1	Essencial	80	Concluído
13	Segurança	Implementação de autenticação e autorização (JWT/login).	PI-2	Alta	60	Concluído
14	Inteligência	Integração com LLM para interpretação de dados técnicos.	PI-2	Alta	40	Concluído
15	Inteligência	Geração automática de plano de ação preventivo (via IA).	PI-2	Alta	32	Concluído
16	Back-end	Módulo de armazenamento de logs de consulta e auditoria.	PI-2	Média	24	Parcial (logs estruturados e alarmes)
17	Segurança	Criptografia de dados sensíveis em repouso (LGPD).	PI-2	Essencial	24	Concluído
18	Front-end	Implementação de temas (dark/light mode) e acessibilidade.	PI-2	Baixa	24	Não realizado (fora da entrega final)
19	Notificação	Sistema de envio de alertas automáticos via e-mail.	PI-2	Média	80	Concluído
20	Qualidade	Realização de testes de integração e carga.	PI-2	Alta	60	Concluído (carga com k6 em 03/11)
21	Segurança	Testes de penetração (pentest) e correção de vulnerabilidades.	PI-2	Essencial	60	Concluído (03/11; S01 e S07 corrigidos)
22	Documentação	Finalização da monografia e revisão para banca final.	PI-2	Essencial	120	Concluído (04/11)
23	Banco de Dados	Evolução do esquema e do painel para múltiplos identificadores por usuário (US07).	PI-2	Média	60	Não realizado (fora da entrega final)
24	Infraestrutura	Implantação na AWS (rede, ECS Fargate, RDS, ElastiCache, S3/CloudFront, SES e alarmes).	PI-2	Alta	80	Concluído
		Total estimado			1.104	

Fonte: elaborado pelos autores (2026).

A.5 Detalhamento das *Sprints* (PI-1 – março a junho)

O cronograma da PI-1 concentrou-se na fundamentação teórica e na construção do protótipo funcional (MVP), cobrindo os itens 01 a 12 do *backlog*:

- ***Sprint* 1 – Concepção e Engenharia de Requisitos (março):** objetivo de estabelecer a base conceitual e técnica para o monitoramento de vazamentos; itens 01, 02 e 12; artefatos: Documento de Visão, Levantamento de Requisitos e Estudo de APIs (OSINT); entregas: definição do escopo, referencial teórico e configuração do repositório.
- ***Sprint* 2 – Modelagem e Design de Experiência (abril):** objetivo de traduzir os requisitos em arquitetura de dados e interface visual; itens 03, 04, 05 e 06; artefatos: DER, dicionário de dados e protótipo de alta fidelidade (Figma); entregas: arquitetura do banco de dados e *layout* da interface homem-máquina.
- ***Sprint* 3 – Desenvolvimento do Core e Lógica de Risco (maio):** objetivo de implementar o motor de busca e o algoritmo de análise; itens 07, 08 e 09; artefatos: servidor NestJS funcional, módulo de integração HIBP e algoritmo de cálculo de *Risk Score*; entregas: *back-end* operacional capaz de processar e categorizar incidentes.
- ***Sprint* 4 – Integração *Front-end* e Entrega do Protótipo (junho):** objetivo de consolidar a Prova de Conceito (PoC) para apresentação à banca; itens 10, 11 e 12; artefatos: *dashboard* em React integrado ao *back-end* e relatório técnico parcial; entregas: protótipo funcional com consulta e visualização de risco e documentação da PI-1 finalizada.

A.6 Planejamento para a PI-2 (agosto a novembro)

Para o segundo semestre, o *roadmap* foca na evolução do protótipo para um sistema mais completo, com integração de inteligência artificial e reforço de segurança (itens 13 a 24). A situação de cada item está no Quadro 9; o planejamento de novembro foi revisto nesta versão, porque o projeto se encerra com a apresentação final em 13/11/2026:

- **Agosto e setembro – Autenticação e Inteligência Cognitiva:** foco em garantir a privacidade do usuário e a camada de IA; itens 13, 14, 15, 16 e 23; atividades: implementação de autenticação JWT, integração com a API da LLM para interpretação dos vazamentos e geração do plano de ação preventivo personalizado, além da evolução do esquema de dados para múltiplos identificadores por usuário

(US07). Situação: autenticação, IA e plano de ação concluídos; logs de auditoria parciais; múltiplos identificadores adiados.

- **Outubro – Conformidade, Segurança e Notificações:** foco no alinhamento com a LGPD e no monitoramento ativo; itens 17, 18 e 19; atividades: implementação de criptografia em repouso, desenvolvimento do sistema de alertas via e-mail e ajustes de acessibilidade e interface (*dark*/*light mode*). Situação: criptografia, alertas e implantação na AWS concluídos; temas adiados.
- **Novembro – Qualidade, Resiliência e Encerramento (revisto):** na primeira semana, o teste de penetração, a campanha de carga com o k6, a jornada no navegador com o Playwright e o teste de alerta real (itens 20 e 21), concluídos em 03/11/2026, com os dois achados acionáveis corrigidos no mesmo dia; na segunda semana, a consolidação dos resultados, a revisão do trabalho escrito, a preparação dos slides (item 22) e a apresentação final, em 13/11/2026, sem novas funcionalidades.

APÊNDICE B – CRONOGRAMA DO PROJETO

O cronograma adota o seguinte mapeamento para 2026: M1 = março, M2 = abril, M3 = maio, M4 = junho, M5 = julho, M6 = agosto, M7 = setembro, M8 = outubro e M9 = novembro. A PI-1 corresponde a M1–M4 (março a junho); a PI-2 corresponde a M6–M9 (agosto a novembro), com a apresentação final em 13/11/2026; e M5 (julho) permanece como período de transição, sem desenvolvimento principal, reservado ao recesso, à revisão do protótipo da PI-1, à consolidação documental, ao refinamento do *backlog* e à preparação do ambiente para a PI-2. Embora o Quadro 8 classifique o risco de prazo como de severidade média-alta, o mês não foi tratado como folga: coincide com o recesso do calendário do IFSP, quando a disponibilidade da equipe é reduzida, e foi usado em atividades que destravam o desenvolvimento seguinte (refinamento do backlog e preparação de ambientes). Em retrospectiva, a PI-2 iniciou com o backlog pronto, e a etapa de qualidade foi antecipada para o início de novembro (Seção A.6).

Para viabilizar essa construção de forma sustentável, adotou-se uma **abordagem metodológica híbrida**. O projeto é governado por um macroplanejamento preditivo, dividido em fases sequenciais de engenharia (Concepção, Elaboração, Construção e Transição), enquanto a execução interna ocorre sob o *framework* ágil **Scrum**, com iterações quinzenais (*Sprints* de 14 dias) para entrega contínua de valor. O tempo total do desenvolvimento do MVP funcional e auditado do SafeID foi de **9 meses, de março a novembro de 2026**.

A distribuição temporal das atividades foi calculada com base na complexidade de acoplamento das APIs externas e na arquitetura de mensageria distribuída. O cronograma divide-se em seis macrofases:

1. **Concepção e Requisitos (M1 e M2):** análise de viabilidade técnica, estudo da documentação da API *Have I Been Pwned*, mapeamento legal dos artigos da LGPD aplicáveis ao produto e engenharia de requisitos por meio do levantamento de histórias de usuário e critérios de aceite.
2. **Arquitetura e Prototipagem (M3 e M4):** projeto da *Clean Architecture* do *back-end*, modelagem lógica e física do banco de dados PostgreSQL e do *cache* Redis, além da criação do protótipo de alta fidelidade da interface (UI/UX) no Figma, estabelecendo a identidade visual baseada no Tailwind CSS.
3. **Transição e Preparação Técnica (M5):** intervalo destinado ao recesso, à revisão do *backlog*, à preparação dos ambientes e ao planejamento das entregas da PI-2, sem atribuição de desenvolvimento principal ao período.

4. **Desenvolvimento e Integração com IA (M6 a M8):** evolução do *back-end* (API NestJS, Prisma ORM, Redis, BullMQ e criptografia AES-256-GCM), desenvolvimento das telas em React.js e Tailwind CSS com integração de APIs e integração com o modelo de linguagem (Azure AI Foundry), incluindo engenharia de *prompts*, parametrização e anonimização dos dados enviados.
5. **Qualidade e Testes Avançados (M9):** estabilização na primeira semana de novembro, com testes automatizados de integração e E2E da API, campanha de carga com o k6, teste de penetração, jornada no navegador com o Playwright e testes de fuzzing no motor de risco.
6. **Implantação e Encerramento (M7 a M9):** provisionamento da infraestrutura na AWS e implantação em produção em setembro e outubro (M7 e M8), em paralelo ao desenvolvimento, e, em novembro (M9), consolidação dos resultados e apresentação final em 13/11/2026. A implantação, prevista originalmente para o último mês, foi antecipada, o que reduziu o risco concentrado no encerramento.

O Quadro 10 apresenta o cronograma consolidado, mapeando o encadeamento lógico das atividades ao longo dos nove meses de execução, e a Figura 17 resume esse planejamento em formato de *roadmap*.

Dependências entre fases paralelas: no Quadro 10 e na Figura 17, o desenvolvimento back-end e o front-end da PI-2 aparecem em paralelo nos mesmos meses, mas não são independentes, porque cada tela depende dos endpoints correspondentes. Para reduzir o risco de atraso em cascata, a equipe trabalhou por fatias verticais: o contrato de cada rota foi definido primeiro (documentado no Swagger e na matriz de consumo da API do repositório), o front-end avançou com respostas simuladas e a integração ocorreu ao fim de cada sprint. As dependências críticas foram: autenticação e verificação de e-mail antes do painel do usuário; integração com a IA antes da tela do plano de ação; e monitoramento antes do sino de notificações. Um atraso no back-end desloca a tela dependente para a sprint seguinte, sem bloquear as demais.

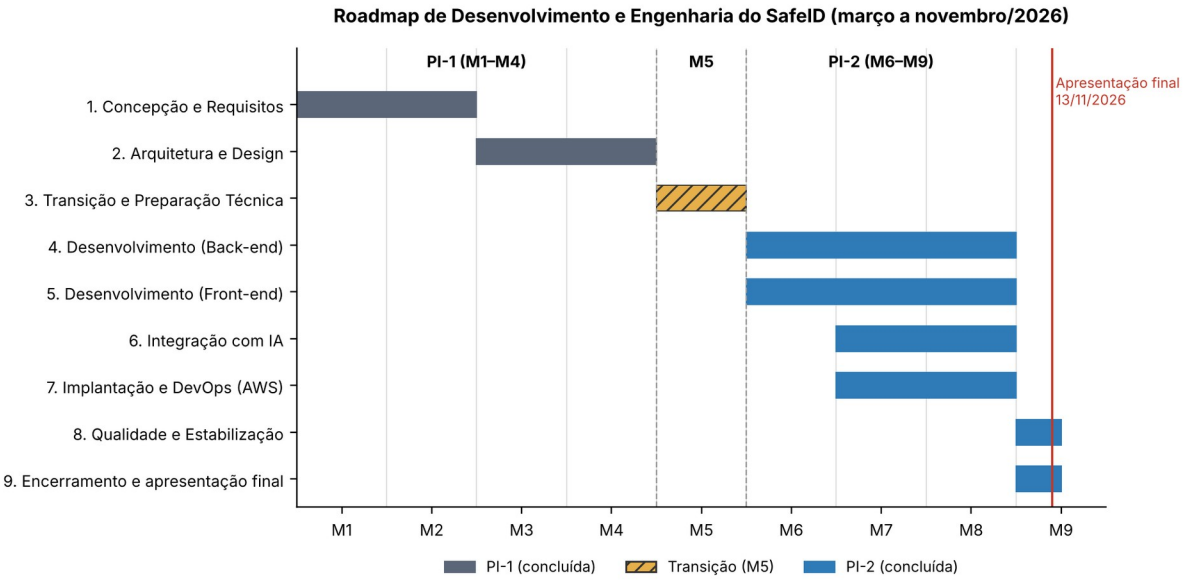
Quadro 10 – Cronograma consolidado e macrofases de engenharia do projeto

Macrofase	Atividades e entregáveis	M1	M2	M3	M4	M5	M6	M7	M8	M9
1. Concepção e Requisitos (PI-1)	Pesquisa OSINT, engenharia de requisitos e compliance LGPD	X	X							
2. Arquitetura e Design (PI-1)	Diagramação de infraestrutura, modelagem de dados e UI/UX			X	X					
3. Transição e Preparação Técnica (M5)	Revisão do protótipo da PI-1, consolidação documental, refinamento do backlog e preparação do ambiente para a PI-2					X				

Macrofase	Atividades e entregáveis	M1	M2	M3	M4	M5	M6	M7	M8	M9
4. Desenvolvimento Back-end (PI-2)	API NestJS, Prisma ORM, Redis, BullMQ e AES-256-GCM						X	X	X	
5. Desenvolvimento Front-end (PI-2)	Telas React.js, Tailwind CSS e integração de APIs						X	X	X	
6. Integração com IA (PI-2)	Prompt engineering com o modelo do Azure AI Foundry (SDK OpenAI 6.41), parametrização e anonimização							X	X	
7. Implantação e DevOps (PI-2)	Conteinerização Docker, CI/CD GitHub Actions e implantação na AWS							X	X	
8. Qualidade e Estabilização (PI-2)	Carga com k6, pentest, E2E no navegador e alerta real (03/11)									X
9. Encerramento (PI-2)	Revisão do trabalho escrito, slides e apresentação final (13/11/2026)									X

Nota: as células sombreadas e marcadas com “X” indicam os meses de execução de cada macrofase. Fonte: elaborado pelos autores (2026).

Figura 17 – *Roadmap* de desenvolvimento das atividades do SafeID



Fonte: elaborado pelos autores (2026).

APÊNDICE C – VIABILIDADE FINANCEIRA

A análise de viabilidade financeira fundamenta a transição do SafeID de uma Prova de Conceito (PoC) acadêmica para um produto de mercado sustentável e escalável. O modelo de negócios adotado estrutura-se sob o formato de **SaaS (*Software as a Service*) *Freemium***, direcionado ao mercado B2C. Essa estratégia concilia o propósito de Responsabilidade Social Digital do projeto — oferecendo consultas básicas gratuitas de utilidade pública para a população — com a monetização recorrente por meio de recursos avançados de monitoramento proativo.

C.1 Custos

Os custos operacionais do SafeID dividem-se em infraestrutura tecnológica, APIs e despesas comerciais e operacionais. Os valores em dólar são convertidos pela premissa de câmbio de R$ 5,20 por dólar, próxima da cotação de setembro e outubro de 2026; a sensibilidade a essa premissa está na Tabela 7. A base comum de dimensionamento é uma infraestrutura com capacidade para até 50.000 usuários cadastrados; essa capacidade não equivale ao número de usuários ativos mensais, que determina o volume efetivo de consultas, notificações e consumo. Os cenários financeiros devem, portanto, ser comparados por usuários ativos mensais e assinantes pagantes, sem tratar a capacidade cadastrada como base ativa. A Tabela 5 projeta os custos operacionais mensais (OpEx) para a faixa inicial de infraestrutura; trata-se de um limite de capacidade contratada, não de uma projeção de 50.000 usuários ativos mensais.

Tabela 5 – Custos operacionais mensais (OpEx) estimados por categoria

Categoria de Custo	Especificação Técnica / Provedor	Custo Mensal Estimado	Natureza do Custo
Hospedagem e banco	AWS em produção: ECS Fargate, ALB, RDS, ElastiCache, NAT, S3/CloudFront e SES (US$ 75 a 105 por mês; considera-se o teto, com NAT)	R$ 550,00	Fixo (escalonável)
API Have I Been Pwned	Assinatura comercial da API de consulta OSINT corporativa	R$ 25,00 (US$ 4,00)	Fixo
Modelo de linguagem (IA)	Consumo de *tokens* no Azure AI Foundry (média de consultas)	R$ 150,00	Variável (por demanda)
Serviços de alerta	Disparos de SMS e WhatsApp do plano Pro (Infobip/Twilio); e-mails pelo SES já incluídos na AWS	R$ 200,00	Variável (por demanda)
Marketing e tráfego	Campanhas de aquisição orgânica e tráfego pago (insumo do CAC, Seção C.6)	R$ 400,00	Flexível
Manutenção e suporte	Licenças de *software* de suporte e provisionamento de CI/CD	R$ 100,00	Fixo

Categoria de Custo	Especificação Técnica / Provedor	Custo Mensal Estimado	Natureza do Custo
Total mensal	**Custo operacional de sustentação inicial**	**R$ 1.425,00**	—

Fonte: elaborado pelos autores (2026).

Memória de cálculo auditável: custo de IA = número de consultas à IA × custo médio por consulta; com as premissas existentes, 60.000 × R$ 0,0025 = R$ 150,00/mês. Custo de mensagens = número de notificações × custo unitário; com as premissas existentes, 1.000 × R$ 0,20 = R$ 200,00/mês. HIBP = número de chaves ou planos necessários para a faixa de volume × preço unitário do plano; o orçamento atual considera uma assinatura de R$ 25,00, devendo a quantidade ser revista por faixa de volume. Infraestrutura/hospedagem = custo do ambiente AWS em produção, entre US$ 2,45 e US$ 3,50 por dia segundo a documentação de implantação, ou seja, US$ 75 a 105 por mês, convertido em R$ 550,00 pelo teto; uma segunda faixa, de igual valor, é considerada acima de 50.000 usuários ativos mensais. Com esse valor, a parcela fixa mensal (hospedagem, HIBP, marketing e manutenção) é de R$ 1.075,00, e a variável, de cerca de R$ 0,023 por usuário ativo (IA e mensagens), o que reproduz o total de R$ 1.425,00 para 15.000 usuários ativos. Como referência, 60.000 consultas mensais equivalem a 4 consultas por usuário ativo para uma base de 15.000 usuários ativos mensais (Cenário Realista); para outras bases, o custo de IA deve ser recalculado como usuários ativos × consultas por usuário × R$ 0,0025, e o de mensagens como usuários ativos notificados × R$ 0,20. São premissas de planejamento, sujeitas ao volume efetivo, ao contrato, ao câmbio e, no caso da IA, à composição de *tokens* de entrada e saída.

Nota de otimização arquitetural: o custo com o modelo de linguagem é mitigado pela camada de cache em memória (Redis): recomendações válidas ficam guardadas por 7 dias, identificadas pelo conjunto de vazamentos e pela classificação, de modo que usuários com os mesmos vazamentos não disparam novas chamadas ao modelo.

C.2 Receitas

A geração de receita do SafeID apoia-se na conversão de usuários da base gratuita para o plano recorrente **SafeID Pro**. O valor da assinatura mensal foi precificado em **R$ 11,90/mês**, patamar competitivo e acessível para a realidade socioeconômica do cidadão brasileiro leigo, posicionando o sistema como um serviço de proteção essencial de baixo custo. Na fase acadêmica, todas as funcionalidades implementadas são oferecidas gratuitamente; a divisão entre as camadas de serviço abaixo é a proposta comercial:

- **Plano Gratuito (Social):** verificação do e-mail da conta, com o indicador visual do Risk Score, a lista de vazamentos, o plano de ação gerado pela IA e alertas de novos vazamentos por e-mail;
- **Plano Premium (SafeID Pro – recorrente):** R$ 11,90 mensais; inclui monitoramento de até 3 identificadores (dependente *da US07)*, envio imediato de alertas via WhatsApp/SMS assim que um novo vazamento for detectado e acesso ilimitado aos manuais de ação didáticos e personalizados gerados pela Inteligência Artificial.

C.3 Cenário Realista

Os três cenários usam agora a mesma linha do tempo e as mesmas premissas de aquisição de usuários, variando apenas a taxa de conversão, para que sejam diretamente comparáveis. A base de usuários ativos mensais (MAU) é a mesma em todos: 1.500 no M1, 3.000 no M2, 4.500 no M3, 8.000 no M4, 11.500 no M5 e 15.000 no M6, crescendo 2.500 por mês até o M24 e 1.000 por mês até o M36. O custo mensal segue a fórmula da Seção C.1 (R$ 1.075,00 fixos + R$ 0,023 por usuário ativo). A receita é MAU × taxa de conversão × R$ 11,90. As taxas são referenciadas em benchmarks de mercado: em produtos freemium, uma conversão de 3% a 5% é considerada boa, e um quarto dos produtos converte menos de 2,5% (CHARTMOGUL; GROWTH UNHINGED; PRODUCTLED, 2026). O cenário realista adota 2%, no quartil inferior; o otimista, 4,5%, dentro da faixa considerada boa; e o pessimista, 0,6%, abaixo do quartil inferior. Os marcos M1 a M36 referem-se a meses de operação comercial, e não ao cronograma acadêmico do Apêndice B.

Cenário realista (conversão de 2%): o equilíbrio ocorre no M4, quando a base supera cerca de 5.000 usuários ativos; no M6, 300 assinantes geram R$ 3.570,00 para um custo de R$ 1.425,00, resultado de R$ 2.145,00.

C.4 Cenário Otimista

Cenário otimista (conversão de 4,5%), sobre a mesma base: o equilíbrio ocorre no M2; no M6, 675 assinantes geram R$ 8.032,50, resultado de R$ 6.607,50; no M12, com 30.000 usuários ativos, 1.350 assinantes geram R$ 16.065,00, resultado de R$ 14.290,00.

O superávit financeiro desse cenário viabilizaria o reinvestimento em Pesquisa e Desenvolvimento (P&D), direcionado às evoluções descritas na Seção 6.3 (Trabalhos Futuros).

C.5 Cenário Pessimista

Cenário pessimista (conversão de 0,6%), sobre a mesma base: no M6, 90 assinantes geram R$ 1.071,00 para um custo de R$ 1.425,00, déficit de R$ 354,00; o equilíbrio só ocorre no M9, com cerca de 22.500 usuários ativos, e o primeiro ano fecha com prejuízo acumulado de cerca de R$ 4 mil.

O custo mensal de R$ 1.425,00 representa a faixa inicial com 15.000 usuários ativos e inclui componentes variáveis; não é integralmente fixo. A hospedagem parte do custo-base da faixa de capacidade e deve ser revista quando CPU, memória, armazenamento ou transferência ultrapassarem o limite contratado. IA e mensagens variam com consultas e notificações, e o HIBP varia com o número de chaves ou planos exigidos pela faixa de volume.

Para reverter o déficit do cenário pessimista e assegurar a sobrevivência do produto sem comprometer a qualidade de entrega, a equipe gestora do SafeID estabelece as seguintes ações contingenciais:

1. **Redução de infraestrutura (*downscaling*):** migração temporária dos servidores lógicos para planos mínimos de computação em nuvem e limitação da retenção de dados temporários de histórico no PostgreSQL, como o modo economia sem NAT já documentado, que reduz o custo diário do ambiente de cerca de US$ 3,50 para US$ 2,45.
2. **Otimização de custos de IA:** redução do limite de *tokens* de saída (*max tokens*) do modelo e aumento da agressividade das políticas de *cache* do Redis, estabelecendo como meta que a inteligência artificial só seja acionada em casos críticos e estritamente inéditos.

A Tabela 6 consolida os três cenários sobre a mesma base, e a Tabela 7 apresenta a projeção de 36 meses e a análise de sensibilidade do cenário realista no M12.

Tabela 6 – Cenários financeiros sobre base comum de usuários ativos

Marco	MAU	Custo mensal	Pessimista (0,6%): assinantes / resultado	Realista (2%): assinantes / resultado	Otimista (4,5%): assinantes / resultado
M1	1.500	R$ 1.110,00	9 / −R$ 1.002,90	30 / −R$ 753,00	68 / −R$ 300,80
M3	4.500	R$ 1.180,00	27 / −R$ 858,70	90 / −R$ 109,00	202 / R$ 1.223,80
M6	15.000	R$ 1.425,00	90 / −R$ 354,00	300 / R$ 2.145,00	675 / R$ 6.607,50
M12	30.000	R$ 1.775,00	180 / R$ 367,00	600 / R$ 5.365,00	1.350 / R$ 14.290,00
Equilíbrio	—	—	M9	M4	M2

Fonte: elaborado pelos autores (2026). Assinantes arredondados ao inteiro; não considera impostos sobre a receita nem taxas de pagamento.

Tabela 7 – Projeção de 36 meses e análise de sensibilidade

Item	Pessimista (0,6%)	Realista (2%)	Otimista (4,5%)
Resultado acumulado no ano 1 (M1–M12)	−R$ 3.959,60	R$ 27.028,00	R$ 82.374,90
Resultado acumulado no ano 2 (M13–M24)	R$ 11.577,00	R$ 104.040,00	R$ 269.152,50
Resultado acumulado no ano 3 (M25–M36)	R$ 18.857,20	R$ 151.804,00	R$ 389.209,00
Sensibilidade do cenário realista no M12 (base: R$ 5.365,00)	Variação	Resultado no M12	Efeito
Câmbio	+20% / −20%	R$ 5.250,00 / R$ 5.480,00	±R$ 115,00
Conversão	1% / 3%	R$ 1.795,00 / R$ 8.935,00	±R$ 3.570,00
Custo de IA por chamada	3 vezes maior	R$ 4.765,00	−R$ 600,00
Churn mensal (efeito no LTV)	5% / 6,8%	LTV de R$ 166,60 / R$ 122,50	−26% no LTV

Fonte: elaborado pelos autores (2026). MAU de 72.000 no M36; segunda faixa de hospedagem acima de 50.000 MAU.

A projeção mostra que o resultado é muito mais sensível à taxa de conversão do que ao câmbio ou ao custo de IA: um ponto percentual de conversão vale cerca de R$ 3.570,00 por mês no M12, contra R$ 115,00 de uma variação cambial de 20%. A premissa mais frágil, porém, é a trajetória de usuários ativos, que pressupõe crescimento contínuo com verba de marketing de apenas R$ 400,00 por mês; por isso, os resultados dos anos 2 e 3 devem ser lidos como limite superior, a ser revisto com dados reais de aquisição.

C.6 Métricas SaaS: CAC e LTV

CAC (Custo de Aquisição de Cliente) = verba de *marketing* atribuível ao período ÷ novos clientes pagantes adquiridos no mesmo período. Com a verba mensal de R$ 400,00 prevista na Tabela 5 e a trajetória de assinantes do Cenário Realista (Seção C.3), obtêm-se um CAC projetado de R$ 13,33 entre M1 e M3 (R$ 400,00 ÷ 30 novos assinantes por mês) e de R$ 5,71 entre M4 e M6 (R$ 400,00 ÷ 70 novos assinantes por mês). Trata-se de estimativa de projeção, que atribui toda a verba de *marketing* às novas assinaturas e desconsidera cancelamentos; o valor real deverá ser apurado por canal quando houver dados de aquisição observados.

LTV (*Lifetime Value*) = receita média mensal por assinante × margem bruta × tempo médio de permanência, com permanência ≈ 1 ÷ *churn* mensal. Como ainda não existem dados observados de *churn* e de margem, adota-se a hipótese ilustrativa de *churn* mensal de 5% (permanência média de 20 meses) e margem bruta de 70%, o que resulta em LTV ≈ R$ 11,90 × 0,70 × 20 = R$ 166,60 por assinante. Essas hipóteses são de planejamento e devem ser substituídas por valores observados após o lançamento. O churn de 5% é otimista frente ao

benchmark de assinaturas B2C, de 6,8% ao mês (RECURLY, [2026]); com esse valor, a permanência média cai para cerca de 15 meses e o LTV, para cerca de R$ 122,50.

Razão LTV/CAC. Com os insumos já apresentados, a razão LTV/CAC resulta em cerca de 12,5 (R$ 166,60 ÷ R$ 13,33) entre M1 e M3 e 29,2 (R$ 166,60 ÷ R$ 5,71) entre M4 e M6; com o churn do benchmark B2C, os valores caem para 9,2 e 21,5. Todos ficam muito acima da referência usual de mercado, de pelo menos 3, o que deve ser lido como sinal de premissas otimistas, e não como evidência de viabilidade. O CAC está provavelmente subestimado: atribui toda a verba de R$ 400,00 a 30 ou 70 novos assinantes por mês, sem custos de aquisição indiretos (conteúdo, parcerias, tempo da equipe) e sem descontar cancelamentos. Como o SafeID ainda não opera comercialmente, CAC, LTV e churn devem ser medidos por canal após o lançamento, e a razão deve ser recalculada com dados reais antes de qualquer decisão de investimento.

ANEXOS

Não há anexos nesta versão do documento. Todos os materiais complementares — diagramas, quadros de gestão, cronograma e análise financeira — foram produzidos pelos próprios autores e encontram-se nos Apêndices A, B e C.